DuocUC® | ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Informe *Ecosistema de Microservicios - LARTDUNISS*

Curso: Fullstack I
Sección: 004D
Docente: Maria Angeles Robinson
Alumnas: Denisse Lazcano / Victoria Rivera

Resumen Negocio

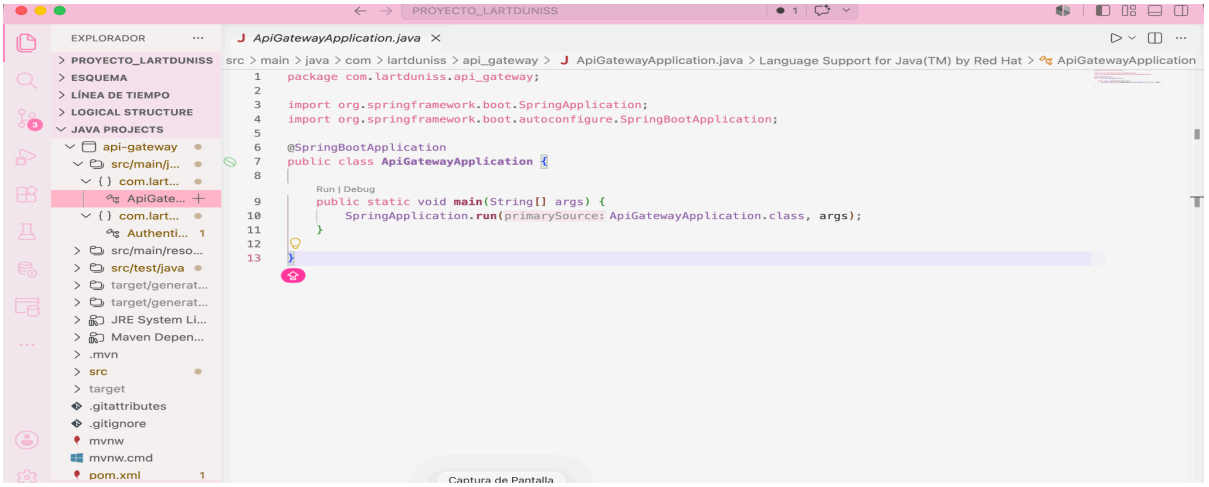
El sistema soporta integralmente el ecosistema de negocio de L'art du niss, una pastelería. Permite la gestión de catálogos, inventarios físicos, procesamiento de pedidos en tiempo real y validación de pagos mediante una arquitectura desacoplada de 10 servicios.

Listado de los Microservicios Implementados

- API GATEWAY:

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **package com.lartduniss.api_gateway;** CLASE -> **ApiGatewayApplication**

```
package com.lartduniss.api_gateway;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ApiGatewayApplication {

   public static void main(String[] args) {
       SpringApplication.run(ApiGatewayApplication.class, args);
   }

}
```



RUTA -> SRC/MAIN/JAVA
PACKAGE -> **package com.lartduniss.api_gateway.filter;** CLASE -> **AuthenticationFilter**

```
package com.lartduniss.api_gateway.filter;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.cloud.gateway.filter.GatewayFilter;
import
org.springframework.cloud.gateway.filter.factory.AbstractGatewayFilterFactory;
import org.springframework.core.io.buffer.DataBuffer;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.server.reactive.ServerHttpRequest;
import org.springframework.http.server.reactive.ServerHttpResponse;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
import reactor.core.publisher.Mono;
import java.nio.charset.StandardCharsets;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Objects;

@Component
public class AuthenticationFilter extends
AbstractGatewayFilterFactory<AuthenticationFilter.Config> {

   @Value("${jwt.secret}")
   private String secreto;
```

```
    public AuthenticationFilter() {
        super(Config.class);
    }

    public static class Config {

    }

    @Override
    public GatewayFilter apply(Config config) {
        return (exchange, chain) -> {
            String path = exchange.getRequest().getURI().getPath();


            if (path.contains("api-docs") ||
                path.contains("/swagger-ui") ||
                path.contains("/webjars") ||
                path.startsWith("/api/v1/usuarios/registrar") ||
                path.startsWith("/api/v1/usuarios/login")) {
                return chain.filter(exchange);
            }


            String authHeader =
exchange.getRequest().getHeaders().getFirst(HttpHeaders.AUTHORIZATION);

            if (authHeader == null || !authHeader.startsWith("Bearer ")) {
                return onError(exchange, "Token de autenticación faltante o con
formato inválido.", HttpStatus.UNAUTHORIZED);
            }

            String token = authHeader.substring(7);

            try {
                Claims claims = Jwts.parserBuilder()

.setSigningKey(Keys.hmacShaKeyFor(secreto.getBytes(StandardCharsets.UTF_8)))
                        .build()
                        .parseClaimsJws(token)
                        .getBody();

                String username = claims.getSubject();

                List<?> rolesRaw = claims.get("roles", List.class);
                List<String> roles = rolesRaw.stream()
```

```
                        .map(Object::toString)
                        .toList();

                String method = exchange.getRequest().getMethod().name();


                if (path.startsWith("/opiniones") &&
                    (method.equals("PUT") || method.equals("DELETE")) &&
                    roles.contains("CLIENTE")) {

                    return onError(exchange,
                        "Lo sentimos, tu cuenta no tiene permisos para realizar
acciones de moderación o alteración estructural en el sistema de opiniones. Esta
función está reservada para ADMINISTRADORES.",
                        HttpStatus.FORBIDDEN);
                }

                if (path.startsWith("/api/v1/inventario") &&
                    (method.equals("POST") || method.equals("DELETE")) &&
                    roles.contains("CLIENTE")) {

                    return onError(exchange,
                        "Lo sentimos, tu cuenta no tiene permisos para registrar o
purgar elementos del inventario directamente. Esta función está reservada para
ADMINISTRADORES.",
                        HttpStatus.FORBIDDEN);
                }

                String rolesStr = String.join(",", roles);


                ServerHttpRequest requestMutada = exchange.getRequest().mutate()
                        .header("X-User-Username", username)
                        .header("X-Username", username)
                        .header("X-User-Roles", rolesStr)
                        .build();
                return
chain.filter(exchange.mutate().request(requestMutada).build());

            } catch (Exception e) {
                return onError(exchange, "El token JWT ha expirado o su firma es
inválida.", HttpStatus.UNAUTHORIZED);
            }
        };
    }
```

```
   @SuppressWarnings("null")
   private Mono<Void> onError(ServerWebExchange exchange, String err, HttpStatus
httpStatus) {
       ServerHttpResponse response = exchange.getResponse();
       response.setStatusCode(httpStatus);
       response.getHeaders().add(HttpHeaders.CONTENT_TYPE,
"application/json;charset=UTF-8");

       String jsonResponse = String.format(
           "{\"timestamp\": \"%s\", \"status\": %d, \"error\": \"%s\", \"mensaje\":
\"%s\"}",
           LocalDateTime.now(),
           httpStatus.value(),
           httpStatus.getReasonPhrase(),
           err
       );

       byte[] bytes =
Objects.requireNonNull(jsonResponse.getBytes(StandardCharsets.UTF_8));
       DataBuffer buffer =
Objects.requireNonNull(response.bufferFactory().wrap(bytes));
       Mono<DataBuffer> body = Objects.requireNonNull(Mono.just(buffer));
       return response.writeWith(body);
   }
}
```

RUTA -> SRC/MAIN/RESOURCES
APPLICATION.YML

```
server:
 port: 9090

jwt:
 secret: "493322258340235739058340598340598340598340598340598340598"

spring:
 application:
   name: api-gateway
 cloud:
   gateway:
     globalcors:
       cors-configurations:
         '[/**]':
           allowedOrigins: "*"
```

```yaml
        allowedMethods:
          - GET
          - POST
          - PUT
          - DELETE
          - OPTIONS
        allowedHeaders: "*"

  routes:
    # 1. Clientes
    - id: clientes-docs
      uri: http://localhost:8091
      predicates:
        - Path=/api/v1/clientes/v3/api-docs
    - id: clientes-service
      uri: http://localhost:8091
      predicates:
        - Path=/api/v1/clientes/**
      filters:
        - AuthenticationFilter

    # 2. Productos
    - id: productos-docs
      uri: http://localhost:8092
      predicates:
        - Path=/api/v1/productos/v3/api-docs
    - id: productos-service
      uri: http://localhost:8092
      predicates:
        - Path=/api/v1/productos/**
      filters:
        - AuthenticationFilter

    # 3. Pagos
    - id: pagos-docs
      uri: http://localhost:8093
      predicates:
        - Path=/api/v1/pagos/v3/api-docs
    - id: pagos-service
      uri: http://localhost:8093
      predicates:
        - Path=/api/v1/pagos/**
      filters:
        - AuthenticationFilter
```

```yaml
      # 4. Pedidos
      - id: pedidos-docs
        uri: http://localhost:8094
        predicates:
          - Path=/api/v1/pedidos/v3/api-docs
      - id: pedidos-service
        uri: http://localhost:8094
        predicates:
          - Path=/api/v1/pedidos/**
        filters:
          - AuthenticationFilter

      # 5. Usuarios
      - id: usuarios-docs
        uri: http://localhost:8095
        predicates:
          - Path=/api/v1/usuarios/v3/api-docs
      - id: usuarios-registro
        uri: http://localhost:8095
        predicates:
          - Path=/api/v1/usuarios/registrar
      - id: usuarios-login
        uri: http://localhost:8095
        predicates:
          - Path=/api/v1/usuarios/login
      - id: usuarios-service
        uri: http://localhost:8095
        predicates:
          - Path=/api/v1/usuarios/**
        filters:
          - AuthenticationFilter

      # 6. Despachos
      - id: despachos-docs
        uri: http://localhost:8097
        predicates:
          - Path=/api/v1/despachos/v3/api-docs
      - id: servicio-despachos
        uri: http://localhost:8097
        predicates:
          - Path=/api/v1/despachos/**
        filters:
          - AuthenticationFilter

      # 7. Inventario
```

```yaml
      - id: inventario-docs
        uri: http://localhost:8099
        predicates:
          - Path=/api/v1/inventario/v3/api-docs
      - id: servicio-inventario
        uri: http://localhost:8099
        predicates:
          - Path=/api/v1/inventario/**
        filters:
          - AuthenticationFilter

      # 8. Notificaciones
      - id: notificaciones-docs
        uri: http://localhost:8071
        predicates:
          - Path=/api/v1/notificaciones/v3/api-docs
      - id: servicio-notificaciones
        uri: http://localhost:8071
        predicates:
          - Path=/api/v1/notificaciones/**
        filters:
          - AuthenticationFilter

      # 9. Reportes
      - id: reportes-docs
        uri: http://localhost:8096
        predicates:
          - Path=/api/v1/reportes/v3/api-docs
      - id: servicio-reportes
        uri: http://localhost:8096
        predicates:
          - Path=/api/v1/reportes/**
        filters:
          - AuthenticationFilter

      # 10. Opiniones
      - id: opiniones-docs
        uri: http://localhost:8098
        predicates:
          - Path=/api/v1/opiniones/v3/api-docs
      - id: servicio-opiniones
        uri: http://localhost:8098
        predicates:
          - Path=/api/v1/opiniones/**
        filters:
```

```
            - AuthenticationFilter

eureka:
 client:
   register-with-eureka: false
   fetch-registry: false

springdoc:
 swagger-ui:
   urls:
     - name: 1. Servicio Clientes
       url: /api/v1/clientes/v3/api-docs
     - name: 2. Servicio Productos
       url: /api/v1/productos/v3/api-docs
     - name: 3. Servicio Pagos
       url: /api/v1/pagos/v3/api-docs
     - name: 4. Servicio Pedidos
       url: /api/v1/pedidos/v3/api-docs
     - name: 5. Servicio Usuarios
       url: /api/v1/usuarios/v3/api-docs
     - name: 6. Servicio Despachos
       url: /api/v1/despachos/v3/api-docs
     - name: 7. Servicio Inventario
       url: /api/v1/inventario/v3/api-docs
     - name: 8. Servicio Notificaciones
       url: /api/v1/notificaciones/v3/api-docs
     - name: 9. Servicio Reportes
       url: /api/v1/reportes/v3/api-docs
     - name: 10. Servicio Opiniones
       url: /api/v1/opiniones/v3/api-docs
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> **package com.lartduniss.api_gateway;**
CLASE -> **ApiGatewayApplicationTest**

```
package com.lartduniss.api_gateway;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.ActiveProfiles;


@SpringBootTest(
   webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT,
   properties = {
```

```
"jwt.secret=claveSecretaSuperSeguraDeMuchosCaracteresParaElEcosistemaLartduNiss2026
"
    }
)
class ApiGatewayApplicationTest {

    @Test
    void contextLoads() {

    }

}
```

POM.XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.5.14</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.lartduniss</groupId>
    <artifactId>api-gateway</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>api-gateway</name>
    <description/>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
```

```
    </scm>
    <properties>
        <java.version>21</java.version>
        <spring-cloud.version>2025.0.2</spring-cloud.version>
    </properties>
    <dependencies>
        <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-gateway-server-webflux</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webflux-ui</artifactId>
        <version>2.7.0</version>
    </dependency>
    <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
<dependency>
```

```
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>

</dependencies>
    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-cloud-dependencies</artifactId>
                <version>${spring-cloud.version}</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>

    <build>
        <plugins>
        <plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <configuration>
        <mainClass>com.lartduniss.api_gateway.ApiGatewayApplication</mainClass>
        <excludes>
            <exclude>
                <groupId>org.projectlombok</groupId>
                <artifactId>lombok</artifactId>
            </exclude>
        </excludes>
    </configuration>
</plugin>


            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>

</project>
```

- 1. DESPACHOS:

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **package config**; CLASE -> **CustomAccessDeniedHandler**

```
import org.springframework.security.web.access.AccessDeniedHandler;
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;

public class CustomAccessDeniedHandler implements AccessDeniedHandler {

   @Override
   public void handle(HttpServletRequest request, HttpServletResponse response,
                      org.springframework.security.access.AccessDeniedException
accessDeniedException)
           throws IOException, ServletException {

       response.setStatus(HttpStatus.FORBIDDEN.value());
       response.setContentType(MediaType.APPLICATION_JSON_VALUE);

       Map<String, Object> data = new HashMap<>();
       data.put("statusCode", HttpStatus.FORBIDDEN.value());
       data.put("message", "No tienes los permisos necesarios para acceder a este
recurso.");
       data.put("timestamp", System.currentTimeMillis());

       ObjectMapper objectMapper = new ObjectMapper();
       response.getOutputStream().println(objectMapper.writeValueAsString(data));
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **package config**; CLASE -> **OpenApiConfig**

```
package config;
```

```
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.servers.Server;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import java.util.List;

@Configuration
public class OpenApiConfig {

   @Bean
   public OpenAPI despachosOpenAPI() {
       return new OpenAPI()
           .info(new Info()
               .title("API de Gestión de Despachos - Lartduniss")
               .version("1.0")
               .description("Documentación interactiva de los endpoints del
microservicio de Despachos"))
           .servers(List.of(
               new Server().url("http://localhost:9090").description("Servidor a
través del Gateway de la Pastelería")
           ));
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **package config**; CLASE -> **RequestHeaderAuthenticationFilter**

```
package config;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.filter.OncePerRequestFilter;
import org.springframework.lang.NonNull;
import java.io.IOException;
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
```

```
public class RequestHeaderAuthenticationFilter extends OncePerRequestFilter {

   @Override
   protected void doFilterInternal(@NonNull HttpServletRequest request, @NonNull
HttpServletResponse response, @NonNull FilterChain filterChain)
           throws ServletException, IOException {

       String username = request.getHeader("X-Username");
       if (username == null) {
           username = request.getHeader("X-User-Username");
       }
       String rolesHeader = request.getHeader("X-User-Roles");

       if (username != null && rolesHeader != null &&
!rolesHeader.trim().isEmpty()) {
           List<SimpleGrantedAuthority> authorities =
Arrays.stream(rolesHeader.split(","))
                   .flatMap(rol -> {
                       String normalized = rol.trim().toUpperCase();
                       return java.util.stream.Stream.of(
                               new SimpleGrantedAuthority(normalized),
                               new SimpleGrantedAuthority("ROLE_" + normalized)
                       );
                   })
                   .collect(Collectors.toList());

           UsernamePasswordAuthenticationToken auth =
                   new UsernamePasswordAuthenticationToken(username, null,
authorities);

           SecurityContextHolder.getContext().setAuthentication(auth);
       }

       filterChain.doFilter(request, response);
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **package config**; CLASE -> **SecurityConfig**

```
package config;

import org.springframework.context.annotation.Bean;
```

```java
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.access.AccessDeniedHandler;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilte
r;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import java.util.List;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

   @Bean
   public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
       return http
           .csrf(csrf -> csrf.disable())
           .cors(cors -> cors.configurationSource(corsConfigurationSource()))
           // Insertamos nuestro filtro de cabeceras antes del de Spring
           .addFilterBefore(new RequestHeaderAuthenticationFilter(),
UsernamePasswordAuthenticationFilter.class)
           .authorizeHttpRequests(auth -> auth
               // Endpoints públicos de Documentación
               .requestMatchers("/api/v1/despachos/v3/api-docs/**",
"/swagger-ui/**", "/swagger-ui.html").permitAll()

               // Restricciones basadas en Roles del Ecosistema
               .requestMatchers(HttpMethod.GET,
"/api/v1/despachos/**").hasAnyRole("ADMIN", "CLIENTE", "EMPLEADO")
               .requestMatchers(HttpMethod.POST,
"/api/v1/despachos/**").hasAnyRole("ADMIN", "EMPLEADO")
               .requestMatchers(HttpMethod.PUT,
"/api/v1/despachos/**").hasAnyRole("ADMIN", "EMPLEADO")
               .requestMatchers(HttpMethod.DELETE,
"/api/v1/despachos/**").hasRole("ADMIN")

               .anyRequest().authenticated()
           )
```

```
            .exceptionHandling(exception -> exception
                .accessDeniedHandler(accessDeniedHandler())
            )
            .build();
    }

    @Bean
    public AccessDeniedHandler accessDeniedHandler() {
        return new CustomAccessDeniedHandler();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();
        configuration.setAllowedOrigins(List.of("http://localhost:8080",
"http://localhost:9090"));
        configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE",
"OPTIONS"));
        configuration.setAllowedHeaders(List.of("*"));
        configuration.setAllowCredentials(true);
        UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);
        return source;
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **package config**; CLASE -> **WebConfig**

```
package config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.lang.NonNull;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(@NonNull CorsRegistry registry) {
        registry.addMapping("/**")
            .allowedOrigins("http://localhost:9090") // Puerto 9090 ya que el 8080
si no me equivoco esta bloqueada
```

```
            .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
            .allowedHeaders("*")
            .allowCredentials(true);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **package controller**; CLASE -> **DespachoController**

```
package controller;

import java.util.List;
import java.util.Objects;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;
import model.Despacho;
import service.DespachoService;

// ANOTACIONES SWAGGER!!!!!!!!!!
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.tags.Tag;

@RestController
@RequestMapping("/api/v1/despachos")
@Tag(name = "Controlador de Despachos", description = "Endpoints para la gestión,
actualización y eliminación de envíos")
public class DespachoController {

    private final DespachoService despachoService;

    public DespachoController(DespachoService despachoService) {
        this.despachoService = despachoService;
    }

    @GetMapping
    @Operation(summary = "Listar todos los despachos", description = "Retorna una
lista completa de todos los despachos registrados en el sistema")
    @ApiResponse(responseCode = "200", description = "Lista obtenida con éxito")
    public ResponseEntity<List<Despacho>> listar() {
        List<Despacho> lista = despachoService.obtenerTodos();
        return new ResponseEntity<>(lista, HttpStatus.OK);
    }
```

```
    @PostMapping
    @Operation(summary = "Crear un nuevo despacho", description = "Registra un
despacho en el sistema y valida sus campos")
    @ApiResponse(responseCode = "201", description = "Despacho creado exitosamente")
    @ApiResponse(responseCode = "400", description = "Datos de entrada inválidos")
    public ResponseEntity<Despacho> crear(@Valid @RequestBody Despacho despacho) {
        Despacho nuevo =
Objects.requireNonNull(despachoService.guardarDespacho(despacho), "La respuesta de
guardarDespacho no puede ser null");
        return new ResponseEntity<>(nuevo, HttpStatus.CREATED);
    }
// Añadimos el runtime exception para manejar el caso de que no se encuentre el
despacho, y así devolver un mensaje más claro al cliente
    @GetMapping("/{id}")
    @Operation(summary = "Obtener un despacho por ID", description = "Busca un
despacho específico en la base de datos a partir de su ID")
    @ApiResponse(responseCode = "200", description = "Despacho encontrado")
    @ApiResponse(responseCode = "404", description = "Despacho no encontrado",
                 content = @io.swagger.v3.oas.annotations.media.Content(schema =
@io.swagger.v3.oas.annotations.media.Schema(implementation =
exception.ErrorResponse.class)))
    public ResponseEntity<Despacho> obtenerUno(@PathVariable Long id) {
        Despacho despacho = despachoService.buscarPorId(id)
                .orElseThrow(() -> new RuntimeException("El despacho con ID " + id +
" no existe en el sistema."));
        return new ResponseEntity<>(despacho, HttpStatus.OK);
    }

    @PutMapping("/{id}")
    @Operation(summary = "Actualizar un despacho existente", description = "Modifica
los detalles de un despacho existente buscando por su ID")
    @ApiResponse(responseCode = "200", description = "Despacho actualizado con
éxito")
    @ApiResponse(responseCode = "404", description = "Despacho no encontrado para
actualizar")
    public ResponseEntity<Despacho> actualizar(@PathVariable Long id, @Valid
@RequestBody Despacho despacho) {
        return despachoService.actualizarDespacho(id, despacho)
                .map(d -> new ResponseEntity<>(d, HttpStatus.OK))
                .orElse(new ResponseEntity<>(HttpStatus.NOT_FOUND));
    }

    @DeleteMapping("/{id}")
```

```
    @Operation(summary = "Eliminar un despacho", description = "Elimina
permanentemente un despacho del sistema a partir de su ID")
    @ApiResponse(responseCode = "200", description = "Despacho eliminado
correctamente")
    @ApiResponse(responseCode = "404", description = "Despacho no encontrado")
    public ResponseEntity<?> eliminar(@PathVariable Long id) {
        if (despachoService.eliminarDespacho(id)) {
            return new ResponseEntity<>("Despacho eliminado correctamente",
HttpStatus.OK);
        }
        return new ResponseEntity<>("No se encontró el despacho a eliminar",
HttpStatus.NOT_FOUND);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **package exception**; CLASE -> **ErrorResponse**

```
package exception;

import lombok.Data;
import lombok.AllArgsConstructor;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class ErrorResponse {
    private int statusCode;
    private String message;
    private long timestamp;
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **package exception**; CLASE -> **GlobalExceptionHandler**

```
package exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
```

```
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {


   @ExceptionHandler(RuntimeException.class)
   public ResponseEntity<ErrorResponse> handleRuntimeException(RuntimeException ex)
{
       ErrorResponse error = new ErrorResponse(
               HttpStatus.NOT_FOUND.value(),
               ex.getMessage(),
               System.currentTimeMillis()
       );
       return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
   }

   @ExceptionHandler(MethodArgumentNotValidException.class)
   public ResponseEntity<Map<String, String>>
handleValidationExceptions(MethodArgumentNotValidException ex) {
       Map<String, String> errors = new HashMap<>();
       ex.getBindingResult().getAllErrors().forEach((error) -> {
           String fieldName = ((FieldError) error).getField();
           String errorMessage = error.getDefaultMessage();
           errors.put(fieldName, errorMessage);
       });
       return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **package lartduniss.proyecto.despachos**; CLASE -> **DespachosApplication**

```
package lartduniss.proyecto.despachos;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.autoconfigure.domain.EntityScan;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;

@SpringBootApplication
```

```java
@ComponentScan(basePackages = {
  "lartduniss.proyecto.despachos",
  "controller",
  "service",
  "config",
  "exception"
})
@EntityScan(basePackages = {"model"})
@EnableJpaRepositories(basePackages = {"repository"})
public class DespachosApplication {
  public static void main(String[] args) {
      SpringApplication.run(DespachosApplication.class, args);
  }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **package model**; CLASE -> **Despacho**

```java
package model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import java.time.LocalDate;
import lombok.Data;
import lombok.AllArgsConstructor;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "despachos")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Despacho {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @NotBlank(message = "La dirección de entrega es obligatoria")
   private String direccionEntrega;
```

```
    @NotBlank(message = "El estado del despacho es obligatorio")
    private String estado;

    @NotNull(message = "La fecha programada es obligatoria")
    private LocalDate fechaProgramada;

    @NotNull(message = "El costo de envío es obligatorio")
    private Double costoEnvio;
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **package repository**; INTERFAZ -> **DespachoRepository**

```
package repository;

import model.Despacho;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface DespachoRepository extends JpaRepository<Despacho, Long> {
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **package service**; CLASE -> **DespachoService**

```
package service;

import model.Despacho;
import repository.DespachoRepository;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Objects;
import java.util.Optional;

@Service
public class DespachoService {

    private final DespachoRepository despachoRepository;

    public DespachoService(DespachoRepository despachoRepository) {
```

```
        this.despachoRepository = despachoRepository;
    }

    public List<Despacho> obtenerTodos() {
        return despachoRepository.findAll();
    }

    public Despacho guardarDespacho(Despacho despacho) {
        return Objects.requireNonNull(despachoRepository.save(despacho), "El
despacho guardado no puede ser null");
    }

    public Optional<Despacho> buscarPorId(Long id) {
        return despachoRepository.findById(id);
    }

    public Optional<Despacho> actualizarDespacho(Long id, Despacho
despachoActualizado) {
        return despachoRepository.findById(id).map((Despacho despachoExistente) -> {

despachoExistente.setDireccionEntrega(despachoActualizado.getDireccionEntrega());
            despachoExistente.setEstado(despachoActualizado.getEstado());

despachoExistente.setFechaProgramada(despachoActualizado.getFechaProgramada());
            despachoExistente.setCostoEnvio(despachoActualizado.getCostoEnvio());
            return despachoRepository.save(despachoExistente);
        });
    }

    public boolean eliminarDespacho(Long id) {
        return despachoRepository.findById(id).map((Despacho despacho) -> {
            despachoRepository.delete(despacho);
            return true;
        }).orElse(false);
    }
}
```

RUTA -> SRC/MAIN/RESOURCES
APPLICATION.PROPERTIES

```
spring.application.name=servicio-despachos
server.port=8097
spring.datasource.url=jdbc:mysql://localhost:3306/db_lart_despachos?createDatabaseI
fNotExist=true
spring.datasource.username=root
```

```
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# CONFIGURACIÓN SWAGGER
springdoc.api-docs.path=/api/v1/despachos/v3/api-docs
springdoc.swagger-ui.path=/swagger-ui.html

# APAGAR REPORTE INFINITO DE CONDICIONES DE ERROR DE LA CONSOLA
logging.level.org.springframework.boot.autoconfigure=ERROR
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> **package lartduniss.proyecto.despachos**; CLASE -> **DespachoApplicationTests**

```
package lartduniss.proyecto.despachos;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Configuration;

@SpringBootTest(classes = DespachosApplicationTests.MockConfig.class)
class DespachosApplicationTests {

    @Configuration
    static class MockConfig {
    }

    @Test
    void contextLoads() {
    }
}
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> **package service**; CLASE -> **DespachoServiceTest**

```
package service;

import model.Despacho;
```

```java
import repository.DespachoRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import java.time.LocalDate;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class DespachoServiceTest {

    @Mock
    private DespachoRepository despachoRepository;

    @InjectMocks
    private DespachoService despachoService;

    @Test
    void guardarDespachoTest() {
        // 1. Arrange
        Despacho despachoMock = new Despacho(null, "Av. Vitacura 1234", "PENDIENTE",
LocalDate.now(), 4500.0);
        Despacho despachoGuardado = new Despacho(1L, "Av. Vitacura 1234",
"PENDIENTE", LocalDate.now(), 4500.0);

        when(despachoRepository.save(despachoMock)).thenReturn(despachoGuardado);

        // 2. Act
        Despacho resultado = despachoService.guardarDespacho(despachoMock);

        // 3. Assert
        assertNotNull(resultado);
        assertEquals(1L, resultado.getId());
        assertEquals("Av. Vitacura 1234", resultado.getDireccionEntrega());
        verify(despachoRepository, times(1)).save(despachoMock);
    }

    @Test
    void buscarPorIdTest() {
        // 1. Arrange
        Long id = 1L;
```

```
        Despacho despachoGuardado = new Despacho(id, "Av. Vitacura 1234",
"PENDIENTE", LocalDate.now(), 4500.0);


when(despachoRepository.findById(id)).thenReturn(Optional.of(despachoGuardado));

        // 2. Act
        Optional<Despacho> resultado = despachoService.buscarPorId(id);

        // 3. Assert
        assertTrue(resultado.isPresent());
        assertEquals("PENDIENTE", resultado.get().getEstado());
        verify(despachoRepository, times(1)).findById(id);
    }
}
```

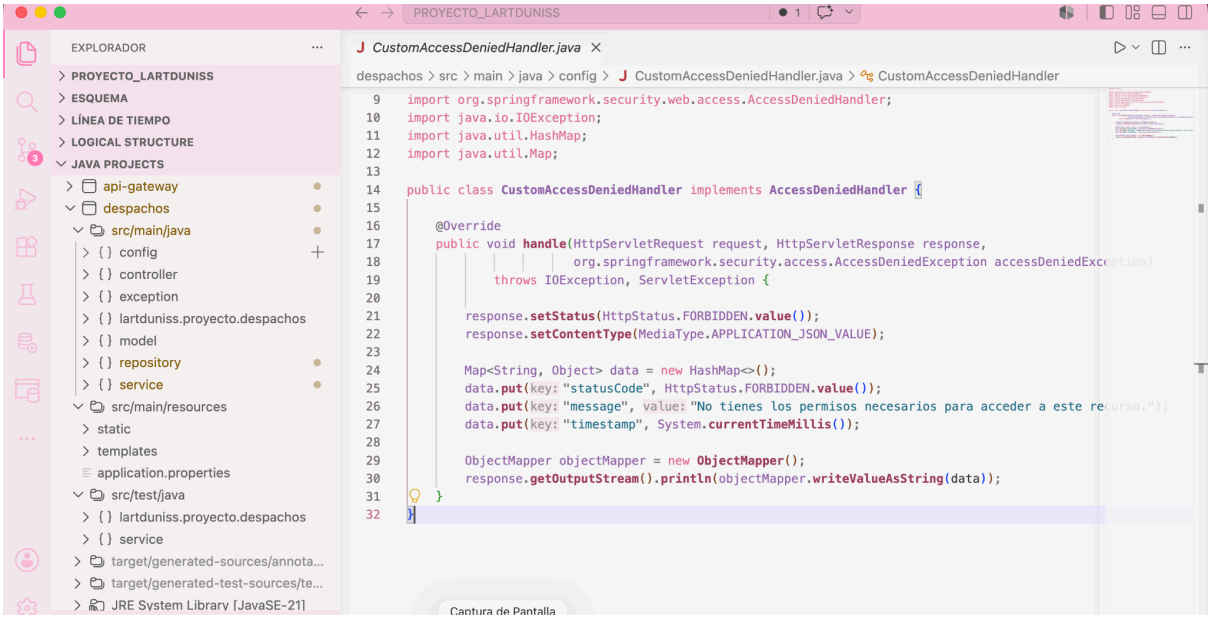
POM.XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>3.5.15</version>
<relativePath/> </parent>
<groupId>lartduniss.proyecto</groupId>
<artifactId>despachos</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>despachos</name>
<description/>
<url/>
<licenses>
<license/>
</licenses>
<developers>
<developer/>
</developers>
<scm>
<connection/>
<developerConnection/>
<tag/>
```

```
<url/>
</scm>
<properties>
<java.version>21</java.version>
</properties>
<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-validation</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
<groupId>org.springdoc</groupId>
<artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
<version>2.7.0</version>
</dependency>
<dependency>
<groupId>com.mysql</groupId>
<artifactId>mysql-connector-j</artifactId>
<scope>runtime</scope>
</dependency>
<dependency>
<groupId>org.projectlombok</groupId>
<artifactId>lombok</artifactId>
<optional>true</optional>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-test</artifactId>
<scope>test</scope>
</dependency>
</dependencies>
<build>
<plugins>
```

```
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
<configuration>
<excludes>
<exclude>
<groupId>org.projectlombok</groupId>
<artifactId>lombok</artifactId>
</exclude>
</excludes>
</configuration>
</plugin>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<executions>
<execution>
<id>default-compile</id>
<phase>compile</phase>
<goals>
<goal>compile</goal>
</goals>
<configuration>
<annotationProcessorPaths>
<path>
<groupId>org.projectlombok</groupId>
<artifactId>lombok</artifactId>
</path>
</annotationProcessorPaths>
</configuration>
</execution>
<execution>
<id>default-testCompile</id>
<phase>test-compile</phase>
<goals>
<goal>testCompile</goal>
</goals>
<configuration>
<annotationProcessorPaths>
<path>
<groupId>org.projectlombok</groupId>
<artifactId>lombok</artifactId>
</path>
</annotationProcessorPaths>
</configuration>
```

```
</execution>
</executions>
</plugin>
</plugins>
</build>
</project>
```



- 2. INVENTARIO:

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.inventario**; CLASE -> **InventarioApplication**

```
package com.lartduniss.inventario;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.autoconfigure.domain.EntityScan;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;
import org.springframework.web.client.RestTemplate;

@SpringBootApplication
@ComponentScan(basePackages = {
   "com.lartduniss.inventario.controller",
   "com.lartduniss.inventario.service",
   "com.lartduniss.inventario.config",
   "exception"
```

```
})
@EntityScan(basePackages = {"com.lartduniss.inventario.model"})
@EnableJpaRepositories(basePackages = {"com.lartduniss.inventario.repository"})
public class InventarioApplication {

   public static void main(String[] args) {
       SpringApplication.run(InventarioApplication.class, args);
   }

   @Bean
   public RestTemplate restTemplate() {
       return new RestTemplate();
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.inventario.config;** CLASE -> **CustomAccessDeniedHandler**

```
package com.lartduniss.inventario.config;

import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.security.web.access.AccessDeniedHandler;
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;

public class CustomAccessDeniedHandler implements AccessDeniedHandler {

   @Override
   public void handle(HttpServletRequest request, HttpServletResponse response,
                      org.springframework.security.access.AccessDeniedException
accessDeniedException)
           throws IOException, ServletException {

       response.setStatus(HttpStatus.FORBIDDEN.value());
       response.setContentType(MediaType.APPLICATION_JSON_VALUE);

       Map<String, Object> data = new HashMap<>();
       data.put("statusCode", HttpStatus.FORBIDDEN.value());
```

```
        data.put("message", "No dispones de los permisos requeridos para modificar
las existencias del inventario.");
        data.put("timestamp", System.currentTimeMillis());

        ObjectMapper objectMapper = new ObjectMapper();
        response.getOutputStream().println(objectMapper.writeValueAsString(data));
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.inventario.config**; CLASE -> **OpenApiConfig**

```
package com.lartduniss.inventario.config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.servers.Server;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import java.util.List;

@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI inventarioOpenAPI() {
        return new OpenAPI()
                .info(new Info()
                        .title("API de Gestión de Inventario - Lartduniss")
                        .version("1.0")
                        .description("Documentación interactiva de los endpoints del
microservicio de Inventario"))
                .servers(List.of(
                        new
Server().url("http://localhost:9090").description("Servidor a través del Gateway de
la Pastelería")
                ));
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.inventario.config**; CLASE -> **RequestHeaderAuthenticationFilter**

```java
package com.lartduniss.inventario.config;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.filter.OncePerRequestFilter;
import org.springframework.lang.NonNull;
import java.io.IOException;
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class RequestHeaderAuthenticationFilter extends OncePerRequestFilter {

   @Override
   protected void doFilterInternal(@NonNull HttpServletRequest request, @NonNull
HttpServletResponse response, @NonNull FilterChain filterChain)
           throws ServletException, IOException {

       String username = request.getHeader("X-Username");
       if (username == null) {
           username = request.getHeader("X-User-Username");
       }
       String rolesHeader = request.getHeader("X-User-Roles");

       if (username != null && rolesHeader != null &&
!rolesHeader.trim().isEmpty()) {
           List<SimpleGrantedAuthority> authorities =
Arrays.stream(rolesHeader.split(","))
                   .flatMap(rol -> {
                       String normalized = rol.trim().toUpperCase();
                       return java.util.stream.Stream.of(
                               new SimpleGrantedAuthority(normalized),
                               new SimpleGrantedAuthority("ROLE_" + normalized)
                       );
                   })
                   .collect(Collectors.toList());

           UsernamePasswordAuthenticationToken auth =
```

```
                    new UsernamePasswordAuthenticationToken(username, null,
authorities);

            SecurityContextHolder.getContext().setAuthentication(auth);
        }

        filterChain.doFilter(request, response);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> com.lartduniss.inventario.config; CLASE -> SecurityConfig

```
package com.lartduniss.inventario.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.access.AccessDeniedHandler;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilte
r;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import java.util.List;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        return http
            .csrf(csrf -> csrf.disable())
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .addFilterBefore(new RequestHeaderAuthenticationFilter(),
UsernamePasswordAuthenticationFilter.class)
            .authorizeHttpRequests(auth -> auth
                // Documentación abierta
```

```
                .requestMatchers("/api/v1/inventario/v3/api-docs/**",
"/swagger-ui/**", "/swagger-ui.html").permitAll()

                // Reglas operativas para Inventario
                .requestMatchers(HttpMethod.GET,
"/api/v1/inventario/**").hasAnyRole("ADMIN", "EMPLEADO", "CLIENTE")
                .requestMatchers(HttpMethod.POST,
"/api/v1/inventario/**").hasAnyRole("ADMIN", "EMPLEADO")
                .requestMatchers(HttpMethod.PUT,
"/api/v1/inventario/**").hasAnyRole("ADMIN", "EMPLEADO")
                .requestMatchers(HttpMethod.DELETE,
"/api/v1/inventario/**").hasRole("ADMIN")

                .anyRequest().authenticated()
            )
            .exceptionHandling(exception -> exception
                .accessDeniedHandler(accessDeniedHandler())
            )
            .build();
    }

    @Bean
    public AccessDeniedHandler accessDeniedHandler() {
        return new CustomAccessDeniedHandler();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();
        configuration.setAllowedOrigins(List.of("http://localhost:8080",
"http://localhost:9090"));
        configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE",
"OPTIONS"));
        configuration.setAllowedHeaders(List.of("*"));
        configuration.setAllowCredentials(true);
        UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);
        return source;
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> com.lartduniss.inventario.config; CLASE -> SecurityConfig

```java
package com.lartduniss.inventario.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.lang.NonNull;

@Configuration
public class WebConfig implements WebMvcConfigurer {

   @Override
   public void addCorsMappings(@NonNull CorsRegistry registry) {
       registry.addMapping("/**")
               .allowedOrigins("http://localhost:9090") // Puerto de tu API Gateway
corregido
               .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
               .allowedHeaders("*")
               .allowCredentials(true);
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.inventario.controller**; CLASE -> **InventarioController**

```java
package com.lartduniss.inventario.controller;

import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;
import com.lartduniss.inventario.model.Inventario;
import com.lartduniss.inventario.service.InventarioService;

// ANOTACIONES SWAGGER <3
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.tags.Tag;

@RestController
@RequestMapping("/api/v1/inventario")
@Tag(name = "Controlador de Inventario", description = "Endpoints para la gestión,
control de stock y alertas de productos")
public class InventarioController {
```

```
    private final InventarioService inventarioService;

    public InventarioController(InventarioService inventarioService) {
        this.inventarioService = inventarioService;
    }

    @GetMapping
    @Operation(summary = "Listar todo el inventario", description = "Retorna una
lista completa de las existencias de todos los productos registrados")
    @ApiResponse(responseCode = "200", description = "Lista de inventario obtenida
con éxito")
    public List<Inventario> listar() {
        return inventarioService.listarTodo();
    }

    @GetMapping("/producto/{productoId}")
    @Operation(summary = "Obtener inventario por ID de Producto", description =
"Busca el registro de stock asociado a un producto específico")
    @ApiResponse(responseCode = "200", description = "Registro de inventario
encontrado")
    @ApiResponse(responseCode = "404", description = "Producto no registrado en el
inventario",
                 content = @io.swagger.v3.oas.annotations.media.Content(schema =
@io.swagger.v3.oas.annotations.media.Schema(implementation =
exception.ErrorResponse.class)))
    public ResponseEntity<Inventario> obtenerPorProducto(@PathVariable Long
productoId) {
        Inventario inventario = inventarioService.buscarPorProducto(productoId)
                .orElseThrow(() -> new RuntimeException("El producto con ID " +
productoId + " no se encuentra registrado en el inventario."));
        return new ResponseEntity<>(inventario, HttpStatus.OK);
    }

    @PostMapping
    @Operation(summary = "Registrar stock inicial", description = "Crea un nuevo
registro de inventario y establece alertas de stock mínimo")
    @ApiResponse(responseCode = "201", description = "Inventario registrado
exitosamente")
    @ApiResponse(responseCode = "400", description = "Datos de entrada inválidos o
negativos")
    public ResponseEntity<Inventario> registrar(@Valid @RequestBody Inventario
inventario) {
        Inventario nuevo = inventarioService.guardar(inventario);
        return ResponseEntity.status(HttpStatus.CREATED).body(nuevo);
```

```
    }

    @PutMapping("/producto/{productoId}/descontar")
    @Operation(summary = "Descontar stock disponible", description = "Disminuye la
cantidad de unidades en base a una venta o pedido")
    @ApiResponse(responseCode = "200", description = "Stock descontado
correctamente")
    @ApiResponse(responseCode = "400", description = "Stock insuficiente o producto
no registrado")
    public ResponseEntity<String> descontar(@PathVariable Long productoId,
@RequestParam Integer cantidad) {
        boolean exito = inventarioService.descontarStock(productoId, cantidad);
        if (exito) {
            return ResponseEntity.ok("Stock descontado correctamente.");
        }
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body("Error: Stock insuficiente o producto no registrado en
inventario.");
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> com.lartduniss.inventario.model; CLASE -> Inventario

```
package com.lartduniss.inventario.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "inventario")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Inventario {

    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotNull(message = "El ID del producto es obligatorio")
    private Long productoId;

    @NotNull(message = "La cantidad disponible es obligatoria")
    @Min(value = 0, message = "La cantidad en stock no puede ser negativa")
    private Integer cantidadDisponible;

    @NotNull(message = "El stock mínimo de alerta es obligatorio")
    @Min(value = 0, message = "El stock mínimo no puede ser negativo")
    private Integer stockMinimoAlerta;
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.inventario.repository**; INTERFAZ -> **InventarioRepository**

```
package com.lartduniss.inventario.repository;

import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.lartduniss.inventario.model.Inventario;

@Repository
public interface InventarioRepository extends JpaRepository<Inventario, Long> {
    Optional<Inventario> findByProductoId(Long productoId);
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.inventario.service**; CLASE -> **InventarioService**

```
package com.lartduniss.inventario.service;

import org.springframework.lang.NonNull;
import java.util.List;
import java.util.Optional;
import org.springframework.stereotype.Service;
import jakarta.transaction.Transactional;
import com.lartduniss.inventario.model.Inventario;
import com.lartduniss.inventario.repository.InventarioRepository;

@Service
```

```java
public class InventarioService {

    private final InventarioRepository inventarioRepository;

    public InventarioService(InventarioRepository inventarioRepository) {
        this.inventarioRepository = inventarioRepository;
    }

    public List<Inventario> listarTodo() {
        return inventarioRepository.findAll();
    }

    public Optional<Inventario> buscarPorProducto(Long productoId) {
        return inventarioRepository.findByProductoId(productoId);
    }

    public Inventario guardar(@NonNull Inventario nuevoInventario) {
        return inventarioRepository.save(nuevoInventario);
    }

    @Transactional
    public boolean descontarStock(Long productoId, Integer cantidadDescontar) {
        Optional<Inventario> optInventario =
inventarioRepository.findByProductoId(productoId);

        if (optInventario.isPresent()) {
            Inventario inventario = optInventario.get();

            if (inventario.getCantidadDisponible() >= cantidadDescontar) {
                int nuevoStock = inventario.getCantidadDisponible() -
cantidadDescontar;
                inventario.setCantidadDisponible(nuevoStock);

                inventarioRepository.save(inventario);

                if (nuevoStock <= inventario.getStockMinimoAlerta()) {
                    System.out.println("No hay stock disponible de: " + productoId +
" Queda unicamente; " + nuevoStock);
                }

                return true;
            }
        }
        return false;
    }
```

```
}
```

RUTA -> SRC/MAIN/RESOURCES
APPLICATION.PROPERTIES

```
spring.application.name=servicio-inventario
server.port=8099

spring.datasource.url=jdbc:mysql://localhost:3306/db_lart_inventario?createDatabase
IfNotExist=true
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Configuración Swagger para el Gateway
springdoc.api-docs.path=/api/v1/inventario/v3/api-docs
springdoc.swagger-ui.path=/swagger-ui.html
server.forward-headers-strategy=framework
```

RUTA -> SRC/TESTJAVA
PACKAGE -> **com.lartduniss.inventario**; CLASE -> **InventarioApplicationTests**

```
package com.lartduniss.inventario;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Configuration;

@SpringBootTest(classes = InventarioApplicationTests.MockConfig.class)
class InventarioApplicationTests {

    @Configuration
    static class MockConfig {
    }

    @Test
    void contextLoads() {
    }
```

```
}
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> **com.lartduniss.inventario.service**; CLASE -> **InventarioServiceTest**

```
package com.lartduniss.inventario.service;

import com.lartduniss.inventario.model.Inventario;
import com.lartduniss.inventario.repository.InventarioRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class InventarioServiceTest {

    @Mock
    private InventarioRepository inventarioRepository;

    @InjectMocks
    private InventarioService inventarioService;

    @Test
    void descontarStockExitosoTest() {
        // 1. Arrange
        Long productoId = 101L;
        Integer cantidadADescontar = 3;
        Inventario inventarioMock = new Inventario(1L, productoId, 10, 5);


when(inventarioRepository.findByProductoId(productoId)).thenReturn(Optional.of(inve
ntarioMock));

when(inventarioRepository.save(any(Inventario.class))).thenReturn(inventarioMock);

        // 2. Act
        boolean resultado = inventarioService.descontarStock(productoId,
cantidadADescontar);
```

```
        // 3. Assert
        assertTrue(resultado);
        assertEquals(7, inventarioMock.getCantidadDisponible()); // 10 - 3 = 7
        verify(inventarioRepository, times(1)).findByProductoId(productoId);
        verify(inventarioRepository, times(1)).save(inventarioMock);
    }

    @Test
    void descontarStockInsuficienteTest() {
        // 1. Arrange
        Long productoId = 101L;
        Integer cantidadADescontar = 15; // Es mayor que el stock disponible
        Inventario inventarioMock = new Inventario(1L, productoId, 10, 5);


when(inventarioRepository.findByProductoId(productoId)).thenReturn(Optional.of(inve
ntarioMock));

        // 2. Act
        boolean resultado = inventarioService.descontarStock(productoId,
cantidadADescontar);

        // 3. Assert
        assertFalse(resultado);
        assertEquals(10, inventarioMock.getCantidadDisponible()); // No se debe
haber descontado nada
        verify(inventarioRepository, times(1)).findByProductoId(productoId);
        verify(inventarioRepository, never()).save(any(Inventario.class));
    }
}
```

- 3. NOTIFICACIONES:

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **config**; CLASE -> **CustomAccessDeniedHandler**

```
package config;

import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.security.web.access.AccessDeniedHandler;
```

```
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;

public class CustomAccessDeniedHandler implements AccessDeniedHandler {

   @Override
   public void handle(HttpServletRequest request, HttpServletResponse response,
                      org.springframework.security.access.AccessDeniedException
accessDeniedException)
           throws IOException, ServletException {

       response.setStatus(HttpStatus.FORBIDDEN.value());
       response.setContentType(MediaType.APPLICATION_JSON_VALUE);

       Map<String, Object> data = new HashMap<>();
       data.put("statusCode", HttpStatus.FORBIDDEN.value());
       data.put("message", "No tienes autorización para gestionar las alertas del
sistema.");
       data.put("timestamp", System.currentTimeMillis());

       ObjectMapper objectMapper = new ObjectMapper();
       response.getOutputStream().println(objectMapper.writeValueAsString(data));
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **config**; CLASE -> **OpenApiConfig**

```
package config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenApiConfig {

   @Bean
   public OpenAPI notificacionesOpenAPI() {
       return new OpenAPI()
               .info(new Info()
                       .title("API de Gestión de Notificaciones - Lartduniss")
```

```
                        .version("1.0")
                        .description("Documentación interactiva de los endpoints del
microservicio de Notificaciones"));
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **config**; CLASE -> **RequestHeaderAuthenticationFilter**

```
package config;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.filter.OncePerRequestFilter;
import org.springframework.lang.NonNull;
import java.io.IOException;
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class RequestHeaderAuthenticationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(@NonNull HttpServletRequest request, @NonNull
HttpServletResponse response, @NonNull FilterChain filterChain)
            throws ServletException, IOException {

        String username = request.getHeader("X-Username");
        if (username == null) {
            username = request.getHeader("X-User-Username");
        }
        String rolesHeader = request.getHeader("X-User-Roles");

        if (username != null && rolesHeader != null &&
!rolesHeader.trim().isEmpty()) {
            List<SimpleGrantedAuthority> authorities =
Arrays.stream(rolesHeader.split(","))
                    .flatMap(rol -> {
                        String normalized = rol.trim().toUpperCase();
```

```
                        return java.util.stream.Stream.of(
                                new SimpleGrantedAuthority(normalized),
                                new SimpleGrantedAuthority("ROLE_" + normalized)
                        );
                    })
                    .collect(Collectors.toList());

            UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(username, null,
authorities);

            SecurityContextHolder.getContext().setAuthentication(auth);
        }

        filterChain.doFilter(request, response);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **config**; CLASE -> **SecurityConfig**

```
package config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.access.AccessDeniedHandler;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilte
r;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import java.util.List;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
```

```
   public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
       return http
           .csrf(csrf -> csrf.disable())
           .cors(cors -> cors.configurationSource(corsConfigurationSource()))
           .addFilterBefore(new RequestHeaderAuthenticationFilter(),
UsernamePasswordAuthenticationFilter.class)
           .authorizeHttpRequests(auth -> auth
               // Endpoints de Swagger expuestos de forma segura
               .requestMatchers(
                   "/api/v1/notificaciones/v3/api-docs",
                   "/api/v1/notificaciones/v3/api-docs/**",
                   "/swagger-ui/**",
                   "/swagger-ui.html",
                   "/webjars/**"
               ).permitAll()


               .requestMatchers(HttpMethod.GET,
"/api/v1/notificaciones/**").hasAnyRole("ADMIN", "EMPLEADO", "CLIENTE")
               .requestMatchers(HttpMethod.POST,
"/api/v1/notificaciones/**").hasAnyRole("ADMIN", "EMPLEADO")
               .requestMatchers(HttpMethod.PUT,
"/api/v1/notificaciones/**").hasAnyRole("ADMIN", "EMPLEADO")
               .requestMatchers(HttpMethod.DELETE,
"/api/v1/notificaciones/**").hasRole("ADMIN")

               .anyRequest().authenticated()
           )
           .exceptionHandling(exception -> exception
               .accessDeniedHandler(accessDeniedHandler())
           )
           .build();
   }

   @Bean
   public AccessDeniedHandler accessDeniedHandler() {
       return new CustomAccessDeniedHandler();
   }

   @Bean
   public CorsConfigurationSource corsConfigurationSource() {
       CorsConfiguration configuration = new CorsConfiguration();
       configuration.setAllowedOrigins(List.of("http://localhost:8080",
"http://localhost:9090"));
```

```
        configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE",
"OPTIONS"));
        configuration.setAllowedHeaders(List.of("*"));
        configuration.setAllowCredentials(true);
        UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);
        return source;
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **config**; CLASE -> **WebConfig**

```
package config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.lang.NonNull;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(@NonNull CorsRegistry registry) {
        registry.addMapping("/**")
                .allowedOrigins("http://localhost:9090") // CORREGIDO EL PUERTO :D
                .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
                .allowedHeaders("*")
                .allowCredentials(true);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **controller**; CLASE -> **NotificacionController**

```
package controller;

import java.util.List;
import java.util.Objects;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
```

```java
import jakarta.validation.Valid;
import model.Notificacion;
import service.NotificacionService;

// ANOTACIONES SWAGGER
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.tags.Tag;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;

@RestController
@RequestMapping("/api/v1/notificaciones")
@Tag(name = "Controlador de Notificaciones", description = "Endpoints para el envío, registro y auditoría de alertas del sistema")
public class NotificacionController {

   private final NotificacionService notificacionService;

   public NotificacionController(NotificacionService notificacionService) {
       this.notificacionService = notificacionService;
   }

   @GetMapping
   @Operation(summary = "Listar todas las notificaciones", description = "Retorna una lista completa de todo el historial de alertas emitidas")
   @ApiResponse(responseCode = "200", description = "Lista de notificaciones obtenida con éxito")
   public ResponseEntity<List<Notificacion>> listar() {
       List<Notificacion> lista = notificacionService.obtenerTodas();
       return new ResponseEntity<>(lista, HttpStatus.OK);
   }

   @PostMapping
   @Operation(summary = "Crear y enviar una notificación", description = "Registra una alerta en el sistema asignándole la fecha actual si no se provee")
   @ApiResponse(responseCode = "201", description = "Notificación creada con éxito")
   @ApiResponse(responseCode = "400", description = "Cuerpo de la petición inválido o incompleto")
   public ResponseEntity<Notificacion> crear(@Valid @RequestBody Notificacion notificacion) {
       Notificacion nueva = Objects.requireNonNull(notificacionService.guardarNotificacion(notificacion), "La respuesta de guardarNotificacion no puede ser null");
```

```
        return new ResponseEntity<>(nueva, HttpStatus.CREATED);
    }

    @GetMapping("/{id}")
    @Operation(summary = "Obtener notificación por ID", description = "Busca los
detalles de una notificación específica a través de su ID único")
    @ApiResponse(responseCode = "200", description = "Notificación encontrada")
    @ApiResponse(responseCode = "404", description = "Notificación no encontrada",
                 content = @Content(schema = @Schema(implementation =
exception.ErrorResponse.class)))
    public ResponseEntity<Notificacion> obtenerUna(@PathVariable Long id) {
        Notificacion notificacion = notificacionService.buscarPorId(id)
                .orElseThrow(() -> new RuntimeException("La notificación con ID " +
id + " no existe en el sistema."));
        return new ResponseEntity<>(notificacion, HttpStatus.OK);
    }

    @PutMapping("/{id}")
    @Operation(summary = "Actualizar una notificación existente", description =
"Modifica los valores de destinatario, tipo, mensaje o fecha buscando por ID")
    @ApiResponse(responseCode = "200", description = "Notificación actualizada de
forma exitosa")
    @ApiResponse(responseCode = "404", description = "La notificación con el ID
indicado no existe",
                 content = @Content(schema = @Schema(implementation =
exception.ErrorResponse.class)))
    public ResponseEntity<Notificacion> actualizar(@PathVariable Long id, @Valid
@RequestBody Notificacion notificacion) {
        Notificacion actualizada = notificacionService.actualizarNotificacion(id,
notificacion)
                .orElseThrow(() -> new RuntimeException("No se pudo actualizar. La
notificación con ID " + id + " no existe en el sistema."));
        return new ResponseEntity<>(actualizada, HttpStatus.OK);
    }

    @DeleteMapping("/{id}")
    @Operation(summary = "Eliminar una notificación", description = "Remueve de
manera permanente el registro del historial mediante su ID")
    @ApiResponse(responseCode = "200", description = "Notificación miembro eliminada
correctamente")
    @ApiResponse(responseCode = "404", description = "No se encontró el registro
para eliminar",
                 content = @Content(schema = @Schema(implementation =
exception.ErrorResponse.class)))
    public ResponseEntity<?> eliminar(@PathVariable Long id) {
```

```
        if (!notificacionService.eliminarNotificacion(id)) {
            throw new RuntimeException("No se encontró la notificación a eliminar con el ID: " + id);
        }
        return new ResponseEntity<>("Notificación eliminada correctamente", HttpStatus.OK);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **exception**; CLASE -> **ErrorResponse**

```
package exception;

import java.time.LocalDateTime;

public class ErrorResponse {
    private LocalDateTime timestamp;
    private int status;
    private String error;
    private String message;
    private String path;

    public ErrorResponse() {
    }

    public ErrorResponse(LocalDateTime timestamp, int status, String error, String message, String path) {
        this.timestamp = timestamp;
        this.status = status;
        this.error = error;
        this.message = message;
        this.path = path;
    }

    // Getters y Setters
    public LocalDateTime getTimestamp() { return timestamp; }
    public void setTimestamp(LocalDateTime timestamp) { this.timestamp = timestamp; }

    public int getStatus() { return status; }
    public void setStatus(int status) { this.status = status; }

    public String getError() { return error; }
    public void setError(String error) { this.error = error; }
```

```
    public String getMessage() { return message; }
    public void setMessage(String message) { this.message = message; }

    public String getPath() { return path; }
    public void setPath(String path) { this.path = path; }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **exception**; CLASE -> **GlobalExceptionHandler**

```
package exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;

import java.time.LocalDateTime;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RuntimeException.class)
    public ResponseEntity<ErrorResponse> handleRuntimeException(RuntimeException ex,
WebRequest request) {
        ErrorResponse error = new ErrorResponse(
                LocalDateTime.now(),
                HttpStatus.NOT_FOUND.value(),
                "Not Found",
                ex.getMessage(),
                request.getDescription(false).replace("uri=", "")
        );
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **model**; CLASE -> **Notificaciones**

```
package model;

import jakarta.persistence.Entity;
```

```java
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import java.time.LocalDateTime;
import lombok.Data;
import lombok.AllArgsConstructor;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "notificaciones")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Notificacion {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "El destinatario es obligatorio (Ej: email o teléfono)")
    private String destinatario;

    @NotBlank(message = "El tipo de notificación es obligatorio (Ej: Email,
WhatsApp)")
    private String tipo;

    @NotBlank(message = "El mensaje no puede estar vacío")
    private String mensaje;

    @NotNull(message = "La fecha de envío es obligatoria")
    private LocalDateTime fechaEnvio;
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **repository**; INTERFAZ-> **NotificacioneRepository**

```java
package repository;

import model.Notificacion;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
```

```java
@Repository
public interface NotificacionRepository extends JpaRepository<Notificacion, Long> {
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **service**; CLASE -> **NotificacionService**

```java
package service;

import model.Notificacion;
import repository.NotificacionRepository;
import org.springframework.stereotype.Service;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Objects;
import java.util.Optional;

@Service
public class NotificacionService {

    private final NotificacionRepository notificacionRepository;

    public NotificacionService(NotificacionRepository notificacionRepository) {
        this.notificacionRepository = notificacionRepository;
    }

    public List<Notificacion> obtenerTodas() {
        return notificacionRepository.findAll();
    }

    public Notificacion guardarNotificacion(Notificacion notificacion) {
        if (notificacion.getFechaEnvio() == null) {
            notificacion.setFechaEnvio(LocalDateTime.now());
        }
        return Objects.requireNonNull(notificacionRepository.save(notificacion), "La
notificación guardada no puede ser null");
    }

    public Optional<Notificacion> buscarPorId(Long id) {
        return notificacionRepository.findById(id);
    }

    public Optional<Notificacion> actualizarNotificacion(Long id, Notificacion
notificacionActualizada) {
```

```
        return notificacionRepository.findById(id).map((Notificacion
notificacionExistente) -> {

notificacionExistente.setDestinatario(notificacionActualizada.getDestinatario());
            notificacionExistente.setTipo(notificacionActualizada.getTipo());
            notificacionExistente.setMensaje(notificacionActualizada.getMensaje());

notificacionExistente.setFechaEnvio(notificacionActualizada.getFechaEnvio());
            return notificacionRepository.save(notificacionExistente);
        });
    }

    public boolean eliminarNotificacion(Long id) {
        return notificacionRepository.findById(id).map((Notificacion notificacion)
-> {
            notificacionRepository.delete(notificacion);
            return true;
        }).orElse(false);
    }
}
```

RUTA -> SRC/MAIN/RESOURCES
APPLICATION.PROPERTIES

```
spring.application.name=servicio-notificaciones
server.port=8071

# Configuración de la Base de Datos MySQL
spring.datasource.url=jdbc:mysql://localhost:3306/db_lart_notificaciones?createData
baseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# Configuración de Hibernate / JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# CONFIGURACIÓN SWAGGER
springdoc.api-docs.path=/api/v1/notificaciones/v3/api-docs
springdoc.swagger-ui.path=/swagger-ui.html

# APAGAR REPORTE INFINITO DE CONDICIONES DE ERROR DE LA CONSOLA
logging.level.org.springframework.boot.autoconfigure=ERROR
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> **proyecto.lartduniss.notificaciones**; CLASE -> **NotificacionesApplicationTests**

```
package proyecto.lartduniss.notificaciones;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Configuration;

@SpringBootTest(properties = {
  "spring.main.allow-bean-definition-overriding=true",
  "springdoc.api-docs.enabled=false",
  "springdoc.swagger-ui.enabled=false",

"spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.security.servl
et.SecurityAutoConfiguration"
}, classes = NotificacionesApplicationTests.MockConfig.class)
class NotificacionesApplicationTests {

   @Configuration
   static class MockConfig {
   }

   @Test
   void contextLoads() {
   }
}
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> **sevice** CLASE -> **NotificacionesServiceTests**

```
package service;

import model.Notificacion;
import repository.NotificacionRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.time.LocalDateTime;
```

```java
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class NotificacionServiceTest {

   @Mock
   private NotificacionRepository notificacionRepository;

   @InjectMocks
   private NotificacionService notificacionService;

   @Test
   void guardarNotificacionAsignaFechaActualTest() {
       // 1. Arrange
       Notificacion notificacionMock = new Notificacion(null, "denisse@test.com",
"Email", "Tu orden está lista", null);
       Notificacion notificacionGuardada = new Notificacion(1L, "denisse@test.com",
"Email", "Tu orden está lista", LocalDateTime.now());


when(notificacionRepository.save(any(Notificacion.class))).thenReturn(notificacionG
uardada);

       // 2. Act
       Notificacion resultado =
notificacionService.guardarNotificacion(notificacionMock);

       // 3. Assert
       assertNotNull(resultado);
       assertEquals(1L, resultado.getId());
       assertNotNull(notificacionMock.getFechaEnvio()); // Comprobamos que el
servicio le asignó la fecha actual
       verify(notificacionRepository, times(1)).save(notificacionMock);
   }

   @Test
   void buscarPorIdTest() {
       // 1. Arrange
       Long id = 5L;
       Notificacion notificacionExistente = new Notificacion(id,
"cliente@test.com", "WhatsApp", "Su pago fue recibido", LocalDateTime.now());
```

```
when(notificacionRepository.findById(id)).thenReturn(Optional.of(notificacionExiste
nte));

        // 2. Act
        Optional<Notificacion> resultado = notificacionService.buscarPorId(id);

        // 3. Assert
        assertTrue(resultado.isPresent());
        assertEquals("WhatsApp", resultado.get().getTipo());
        verify(notificacionRepository, times(1)).findById(id);
    }
}
```

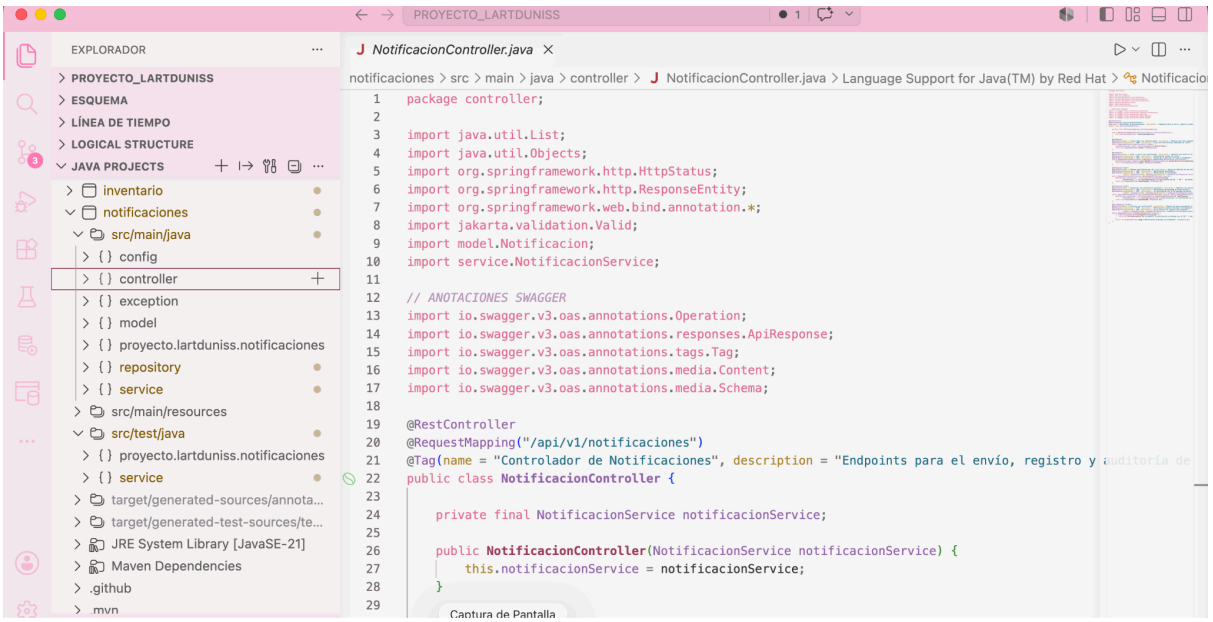
POM.XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.5.15</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>proyecto.lartduniss</groupId>
    <artifactId>notificaciones</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>notificaciones</name>
    <description/>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
```

```
</scm>
<properties>
    <java.version>21</java.version>
</properties>
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
        <version>2.8.17</version>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-hateoas</artifactId>
```

```
    </dependency>

</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <excludes>
                    <exclude>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </exclude>
                </excludes>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <executions>
                <execution>
                    <id>default-compile</id>
                    <phase>compile</phase>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok</artifactId>
                            </path>
                        </annotationProcessorPaths>
                    </configuration>
                </execution>
                <execution>
                    <id>default-testCompile</id>
                    <phase>test-compile</phase>
                    <goals>
                        <goal>testCompile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
```

```
                                <path>
                                    <groupId>org.projectlombok</groupId>
                                    <artifactId>lombok</artifactId>
                                </path>
                            </annotationProcessorPaths>
                        </configuration>
                    </execution>
                </executions>
            </plugin>
        </plugins>
    </build>

</project>
```



- 4. OPINIONES:

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.opiniones**; CLASE -> **OpinionesApplication**

```
package com.lartduniss.opiniones;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.autoconfigure.domain.EntityScan;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;
import org.springframework.web.client.RestTemplate;
```

```java
@SpringBootApplication
@EnableJpaRepositories(basePackages = "com.lartduniss.opiniones.repository") //
@EntityScan(basePackages = "com.lartduniss.opiniones.model")
@ComponentScan(basePackages = {
    "com.lartduniss.opiniones",
    "com.lartduniss.opiniones.controller",
    "com.lartduniss.opiniones.service",
    "com.lartduniss.opiniones.config",
    "exception"
})
public class OpinionesApplication {

    public static void main(String[] args) {
        SpringApplication.run(OpinionesApplication.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.opiniones.config**; CLASE -> **CustomAccessDeniedHandler**

```java
package com.lartduniss.opiniones.config;

import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.security.web.access.AccessDeniedHandler;
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;

public class CustomAccessDeniedHandler implements AccessDeniedHandler {

    @Override
    public void handle(HttpServletRequest request, HttpServletResponse response,
```

```
                    org.springframework.security.access.AccessDeniedException
accessDeniedException)
            throws IOException, ServletException {

        response.setStatus(HttpStatus.FORBIDDEN.value());
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);

        Map<String, Object> data = new HashMap<>();
        data.put("statusCode", HttpStatus.FORBIDDEN.value());
        data.put("message", "No tienes permisos para interactuar o moderar las
reseñas del sistema.");
        data.put("timestamp", System.currentTimeMillis());

        ObjectMapper objectMapper = new ObjectMapper();
        response.getOutputStream().println(objectMapper.writeValueAsString(data));
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.opiniones.config**; CLASE -> **OpenApiConfig**

```
package com.lartduniss.opiniones.config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI opinionesOpenAPI() {
        return new OpenAPI()
                .info(new Info()
                        .title("API de Gestión de Opiniones - Lartduniss")
                        .version("1.0")
                        .description("Documentación interactiva de los endpoints del
microservicio de Opiniones"));
    }
}
```

RUTA -> SRC/MAIN/JAVA

PACKAGE -> com.lartduniss.opiniones.config; CLASE -> RequestHeaderAuthenticationFilter

```
package com.lartduniss.opiniones.config;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.filter.OncePerRequestFilter;
import org.springframework.lang.NonNull;
import java.io.IOException;
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class RequestHeaderAuthenticationFilter extends OncePerRequestFilter {

   @Override
   protected void doFilterInternal(@NonNull HttpServletRequest request, @NonNull
HttpServletResponse response, @NonNull FilterChain filterChain)
           throws ServletException, IOException {

       String username = request.getHeader("X-Username");
       if (username == null) {
           username = request.getHeader("X-User-Username");
       }
       String rolesHeader = request.getHeader("X-User-Roles");

       if (username != null && rolesHeader != null &&
!rolesHeader.trim().isEmpty()) {
           List<SimpleGrantedAuthority> authorities =
Arrays.stream(rolesHeader.split(","))
                   .flatMap(rol -> {
                       String normalized = rol.trim().toUpperCase();
                       return java.util.stream.Stream.of(
                               new SimpleGrantedAuthority(normalized),
                               new SimpleGrantedAuthority("ROLE_" + normalized)
                       );
                   })
                   .collect(Collectors.toList());
```

```
            UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(username, null,
authorities);

            SecurityContextHolder.getContext().setAuthentication(auth);
        }

        filterChain.doFilter(request, response);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.opiniones.config**; CLASE -> **SecurityConfig**

```
package com.lartduniss.opiniones.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.access.AccessDeniedHandler;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilte
r;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import java.util.List;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        return http
            .csrf(csrf -> csrf.disable())
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .addFilterBefore(new RequestHeaderAuthenticationFilter(),
UsernamePasswordAuthenticationFilter.class)
            .authorizeHttpRequests(auth -> auth
```

```
                .requestMatchers("/api/v1/opiniones/v3/api-docs/**",
"/swagger-ui/**", "/swagger-ui.html").permitAll()

                .requestMatchers(HttpMethod.GET, "/api/v1/opiniones/**").permitAll()
// Reseñas públicas para que los visitantes las vean
                .requestMatchers(HttpMethod.POST,
"/api/v1/opiniones/**").hasAnyRole("CLIENTE", "ADMIN", "EMPLEADO") // Clientes
publican su feedback

                .anyRequest().authenticated()
            )
            .exceptionHandling(exception -> exception
                .accessDeniedHandler(accessDeniedHandler())
            )
            .build();
    }

    @Bean
    public AccessDeniedHandler accessDeniedHandler() {
        return new CustomAccessDeniedHandler();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();
        configuration.setAllowedOrigins(List.of("http://localhost:8080",
"http://localhost:9090"));
        configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE",
"OPTIONS"));
        configuration.setAllowedHeaders(List.of("*"));
        configuration.setAllowCredentials(true);
        UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);
        return source;
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.opiniones.config**; CLASE -> **WebConfig**

```
package com.lartduniss.opiniones.config;

import org.springframework.context.annotation.Configuration;
```

```java
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.lang.NonNull;

@Configuration
public class WebConfig implements WebMvcConfigurer {

   @Override
   public void addCorsMappings(@NonNull CorsRegistry registry) {
       registry.addMapping("/**")
               .allowedOrigins("http://localhost:9090") // puerto correctito <3
               .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
               .allowedHeaders("*")
               .allowCredentials(true);
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.opiniones.controller**; CLASE -> **OpinionesController**

```java
package com.lartduniss.opiniones.controller;

import java.util.List;
import java.util.Objects;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;

import com.lartduniss.opiniones.model.Opiniones;
import com.lartduniss.opiniones.service.OpinionesService;

// IMPORTACIONES DE SWAGGER <3
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.tags.Tag;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;

@RestController
@RequestMapping("/api/v1/opiniones")
@Tag(name = "Controlador de Opiniones", description = "Endpoints para gestionar las
reseñas y calificaciones de los productos")
public class OpinionesController {
```

```
    private final OpinionesService opinionesService;

    public OpinionesController(OpinionesService opinionesService) {
        this.opinionesService = opinionesService;
    }

    @GetMapping
    @Operation(summary = "Listar todas las opiniones", description = "Retorna el
historial global de reseñas almacenadas en la plataforma")
    @ApiResponse(responseCode = "200", description = "Lista de opiniones obtenida
con éxito")
    public List<Opiniones> listar() {
        return opinionesService.listarTodas();
    }

    @GetMapping("/producto/{productoId}")
    @Operation(summary = "Obtener opiniones por Producto", description = "Recupera
todas las reseñas asociadas a un ID de producto específico")
    @ApiResponse(responseCode = "200", description = "Reseñas del producto
localizadas exitosamente")
    @ApiResponse(responseCode = "404", description = "No se encontraron opiniones
para el producto indicado",
                 content = @Content(schema = @Schema(implementation =
exception.ErrorResponse.class)))
    public ResponseEntity<List<Opiniones>> obtenerPorProducto(@PathVariable Long
productoId) {
        List<Opiniones> opiniones = opinionesService.buscarPorProducto(productoId);
        if (opiniones.isEmpty()) {
            throw new RuntimeException("No se encontraron opiniones registradas para
el producto con ID " + productoId);
        }
        return ResponseEntity.ok(opiniones);
    }

    @GetMapping("/producto/{productoId}/promedio")
    @Operation(summary = "Calcular promedio de calificación", description = "Genera
la puntuación media en base a las estrellas asignadas al producto")
    @ApiResponse(responseCode = "200", description = "Promedio calculado
correctamente (retorna 0.0 si no registra opiniones)")
    public ResponseEntity<Double> obtenerPromedio(@PathVariable Long productoId) {
        return
ResponseEntity.ok(opinionesService.obtenerPromedioCalificacion(productoId));
    }

    @PostMapping
```

```
   @Operation(summary = "Publicar una opinión", description = "Registra una nueva
opinión asignándole automáticamente la fecha actual")
   @ApiResponse(responseCode = "201", description = "Opinión publicada
correctamente")
   @ApiResponse(responseCode = "400", description = "Cuerpo de petición no válido
(por ejemplo, calificación fuera del rango 1-5)")
   public ResponseEntity<Opiniones> crear(@Valid @RequestBody Opiniones opiniones)
{
       Opiniones nuevaOpinion =
Objects.requireNonNull(opinionesService.guardar(opiniones), "La respuesta de
guardar no puede ser null");
       return ResponseEntity.status(HttpStatus.CREATED).body(nuevaOpinion);
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.opiniones.model**; CLASE -> **Opiniones**

```
package com.lartduniss.opiniones.model;

import java.time.LocalDate;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "opiniones")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Opiniones {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;
```

```
    @NotNull(message = "El ID del producto es obligatorio")
    private Long productoId;

    @NotNull(message = "El ID del cliente es obligatorio")
    private Long clienteId;

    @NotNull(message = "La calificación es obligatoria")
    @Min(value = 1, message = "La calificación mínima es 1 estrella")
    @Max(value = 5, message = "La calificación máxima es 5 estrellas")
    private Integer calificacion;

    @NotBlank(message = "El comentario no puede estar vacío")
    @Size(max = 500, message = "El comentario no puede superar los 500 caracteres")
    private String comentario;

    private LocalDate fechaPublicacion;
}
```

RUTA -> SRC/MAIN/JAVA

PACKAGE -> **com.lartduniss.opiniones.repository**; INTERFAZ -> **OpinionesRepository**

```
package com.lartduniss.opiniones.repository;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import com.lartduniss.opiniones.model.Opiniones;


public interface OpinionesRepository extends JpaRepository<Opiniones, Long>
{
    List<Opiniones> findByProductoId(Long productoId);
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.opiniones.service**; CLASE -> **OpinionesService**

```
package com.lartduniss.opiniones.service;

import java.time.LocalDate;
import java.util.List;
import java.util.Objects;
```

```java
import org.springframework.stereotype.Service;
import com.lartduniss.opiniones.model.Opiniones;
import com.lartduniss.opiniones.repository.OpinionesRepository;
import jakarta.transaction.Transactional;

@Service
public class OpinionesService {

   private final OpinionesRepository opinionesRepository;

   public OpinionesService(OpinionesRepository opinionesRepository) {
       this.opinionesRepository = opinionesRepository;
   }

   public List<Opiniones> listarTodas() {
       return opinionesRepository.findAll();
   }

   public List<Opiniones> buscarPorProducto(Long productoId) {
       return opinionesRepository.findByProductoId(productoId);
   }

   @Transactional
   public Opiniones guardar(Opiniones opiniones) {
       opiniones.setFechaPublicacion(LocalDate.now());
       return Objects.requireNonNull(opinionesRepository.save(opiniones), "La
opinión guardada no puede ser null");
   }

   public Double obtenerPromedioCalificacion(Long productoId) {
       List<Opiniones> listaOpiniones =
opinionesRepository.findByProductoId(productoId);
       if (listaOpiniones.isEmpty()) {
           return 0.0;
       }

       double suma = listaOpiniones.stream()
                                   .mapToDouble(Opiniones::getCalificacion)
                                   .sum();

       return suma / listaOpiniones.size();
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **exception**; CLASE -> **ErrorResponse**

```
package exception;

import java.time.LocalDateTime;

public class ErrorResponse {
   private LocalDateTime timestamp;
   private int status;
   private String error;
   private String message;
   private String path;

   public ErrorResponse() {}

   public ErrorResponse(LocalDateTime timestamp, int status, String error, String
message, String path) {
        this.timestamp = timestamp;
        this.status = status;
        this.error = error;
        this.message = message;
        this.path = path;
   }

   public LocalDateTime getTimestamp() { return timestamp; }
   public void setTimestamp(LocalDateTime timestamp) { this.timestamp = timestamp;
}
   public int getStatus() { return status; }
   public void setStatus(int status) { this.status = status; }
   public String getError() { return error; }
   public void setError(String error) { this.error = error; }
   public String getMessage() { return message; }
   public void setMessage(String message) { this.message = message; }
   public String getPath() { return path; }
   public void setPath(String path) { this.path = path; }
}
```

RUTA -> SRC/MAIN/JAVA

PACKAGE -> **exception**; CLASE -> **GlobalExceptionHandler**

```
package exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;
import java.time.LocalDateTime;

@RestControllerAdvice
public class GlobalExceptionHandler {

   @ExceptionHandler(RuntimeException.class)
   public ResponseEntity<ErrorResponse> handleRuntimeException(RuntimeException ex,
WebRequest request) {
       ErrorResponse error = new ErrorResponse(
               LocalDateTime.now(),
               HttpStatus.NOT_FOUND.value(),
               "Not Found",
               ex.getMessage(),
               request.getDescription(false).replace("uri=", "")
       );
       return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
   }
}
```

RUTA -> SRC/MAIN/RESOURCES
APPLICATION.PROPERTIES

```
spring.application.name=servicio-opiniones
server.port=8098

# Configuración de la Base de Datos MySQL
spring.datasource.url=jdbc:mysql://localhost:3306/db_lart_opiniones?createDatabaseI
fNotExist=true
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# Configuración de Hibernate / JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

```
# CONFIGURACIÓN SWAGGER <3
springdoc.api-docs.path=/api/v1/opiniones/v3/api-docs
springdoc.swagger-ui.path=/swagger-ui.html

# APAGAR REPORTE INFINITO DE CONDICIONES DE ERROR DE LA CONSOLA
logging.level.org.springframework.boot.autoconfigure=ERROR
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> **com.lartduniss.opiniones;** CLASE -> **OpinionesApplicationTests**

```
package com.lartduniss.opiniones;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertTrue;

class OpinionesApplicationTests {

   @Test
   void contextLoads() {
       // Test básico unitario para asegurar que el entorno de pruebas compile de
forma correcta
       assertTrue(true);
   }
}
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> **com.lartduniss.opiniones;** CLASE -> **OpinionesApplicationTests**

```
package com.lartduniss.opiniones.service;

import com.lartduniss.opiniones.model.Opiniones;
import com.lartduniss.opiniones.repository.OpinionesRepository;
import com.lartduniss.opiniones.service.OpinionesService;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.mockito.Mockito;

import java.time.LocalDate;
import java.util.Arrays;
import java.util.Collections;
```

```java
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.junit.jupiter.api.Assertions.assertEquals;

@ExtendWith(MockitoExtension.class)
class OpinionesServiceTest {

   @Mock
   private OpinionesRepository opinionesRepository;

   @InjectMocks
   private OpinionesService opinionesService;

   @Test
   void obtenerPromedioCalificacionTest() {
       // 1. Arrange <3
       Long productoId = 200L;
       Opiniones op1 = new Opiniones(1L, productoId, 1L, 5, "Excelente producto",
LocalDate.now());
       Opiniones op2 = new Opiniones(2L, productoId, 2L, 3, "Regular",
LocalDate.now());
       List<Opiniones> listaMock = Arrays.asList(op1, op2);


Mockito.when(opinionesRepository.findByProductoId(productoId)).thenReturn(listaMock
);

       // 2. Act <3
       Double promedio = opinionesService.obtenerPromedioCalificacion(productoId);

       // 3. Assert <3
       assertNotNull(promedio);
       assertEquals(4.0, promedio); // (5 + 3) / 2 = 4.0
       Mockito.verify(opinionesRepository,
Mockito.times(1)).findByProductoId(productoId);
   }

   @Test
   void obtenerPromedioSinOpinionesTest() {
       // 1. Arrange
       Long productoId = 200L;

Mockito.when(opinionesRepository.findByProductoId(productoId)).thenReturn(Collectio
ns.emptyList());
```

```
        // 2. Act
        Double promedio = opinionesService.obtenerPromedioCalificacion(productoId);

        // 3. Assert
        assertEquals(0.0, promedio);
        Mockito.verify(opinionesRepository,
Mockito.times(1)).findByProductoId(productoId);
    }
}
```

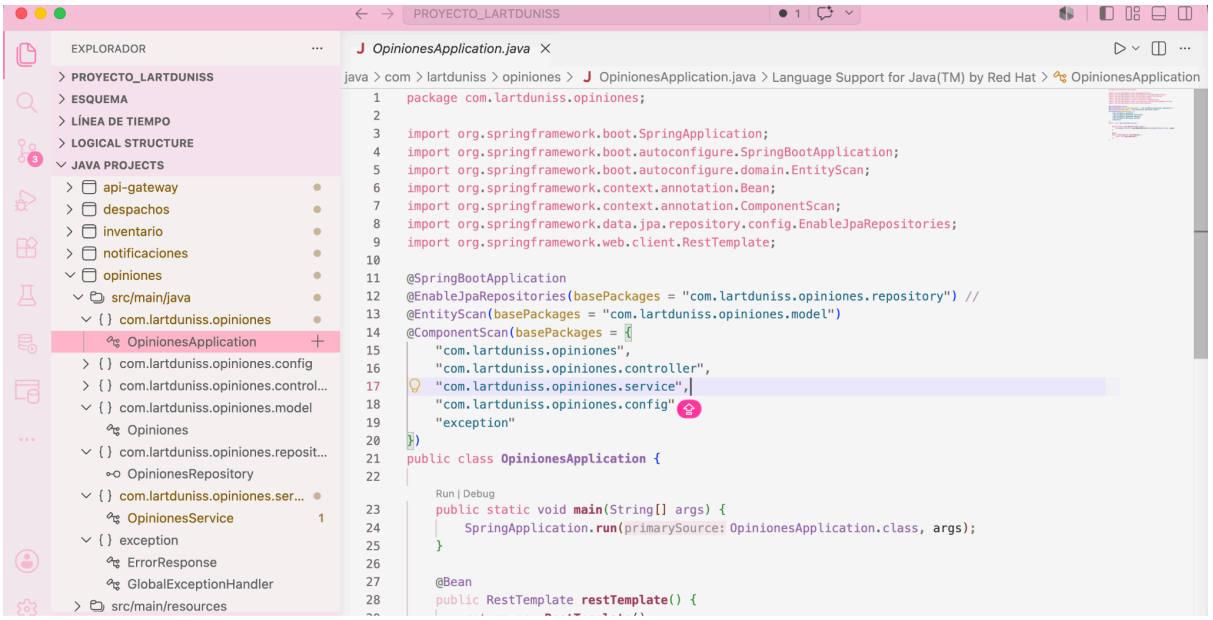
POM.XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>
   <parent>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-parent</artifactId>
       <version>3.5.15</version>
       <relativePath/> <!-- lookup parent from repository -->
   </parent>
   <groupId>com.lartduniss</groupId>
   <artifactId>opiniones</artifactId>
   <version>0.0.1-SNAPSHOT</version>
   <name>opiniones</name>
   <description/>
   <url/>
   <licenses>
       <license/>
   </licenses>
   <developers>
       <developer/>
   </developers>
   <scm>
       <connection/>
       <developerConnection/>
       <tag/>
       <url/>
   </scm>
   <properties>
       <java.version>21</java.version>
```

```
</properties>
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
        <version>2.8.17</version>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-hateoas</artifactId>
    </dependency>

</dependencies>
```

```xml
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <excludes>
                    <exclude>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </exclude>
                </excludes>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <executions>
                <execution>
                    <id>default-compile</id>
                    <phase>compile</phase>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok</artifactId>
                            </path>
                        </annotationProcessorPaths>
                    </configuration>
                </execution>
                <execution>
                    <id>default-testCompile</id>
                    <phase>test-compile</phase>
                    <goals>
                        <goal>testCompile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok</artifactId>
```

```
                        </path>
                    </annotationProcessorPaths>
                </configuration>
            </execution>
        </executions>
    </plugin>
  </plugins>
 </build>

</project>
```



- 5. PAGOS:

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pagos;** CLASE -> **PagosApplication**

```
package com.lartduniss.pagos;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.autoconfigure.domain.EntityScan;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;
import org.springframework.web.client.RestTemplate;

@SpringBootApplication
@ComponentScan(basePackages = {
```

```
    "com.lartduniss.pagos",
    "com.lartduniss.pagos.config",
    "com.lartduniss.pagos.controller",
    "com.lartduniss.pagos.service",
    "com.lartduniss.pagos.exception"
})
@EntityScan(basePackages = {"com.lartduniss.pagos.model"})
@EnableJpaRepositories(basePackages = {"com.lartduniss.pagos.repository"})
public class PagosApplication {

    public static void main(String[] args) {
        SpringApplication.run(PagosApplication.class, args);
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pagos.config**; CLASE -> **CustomAccessDeniedHandler**

```
package com.lartduniss.pagos.config;

import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.HttpStatus;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.web.access.AccessDeniedHandler;
import java.io.IOException;
import java.time.LocalDateTime;

public class CustomAccessDeniedHandler implements AccessDeniedHandler {

    @Override
    public void handle(HttpServletRequest request, HttpServletResponse response,
                       AccessDeniedException accessDeniedException) throws
IOException, ServletException {

        response.setContentType("application/json;charset=UTF-8");
        response.setStatus(HttpStatus.FORBIDDEN.value());

        String jsonResponse = String.format(
```

```
                "{\"timestamp\": \"%s\", \"status\": 403, \"error\": \"Forbidden\",
" +
                "\"mensaje\": \"Acceso denegado: Tu cuenta no posee los privilegios
administrativos requeridos para modificar registros de pagos.\"}",
                LocalDateTime.now()
        );

        response.getWriter().write(jsonResponse);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pagos.config**; CLASE -> **ErrorResponse**

```
package com.lartduniss.pagos.config;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class ErrorResponse {
    private int status;
    private String message;
    private long timestamp;
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pagos.config**; CLASE -> **OpenApiConfig**

```
package com.lartduniss.pagos.config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.servers.Server;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import java.util.List;

@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI customOpenAPI() {
```

```
        return new OpenAPI()
                .info(new Info()
                        .title("API Lartduniss - Sistema de Pago")
                        .version("1.0")
                        .description("Información de los pagos"))
                .servers(List.of(
                        new
Server().url("http://localhost:9090").description("Puerto Unificado del API
Gateway")
                ));
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pagos.config**; CLASE -> **RequestHeaderAuthenticationFilter**

```
package com.lartduniss.pagos.config;

import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.filter.OncePerRequestFilter;
import org.springframework.lang.NonNull;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class RequestHeaderAuthenticationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(@NonNull HttpServletRequest request, @NonNull
HttpServletResponse response, @NonNull FilterChain filterChain)
            throws ServletException, IOException {

        String username = request.getHeader("X-User-Username");
        if (username == null) {
            username = request.getHeader("X-Username");
        }
```

```
        String rolesStr = request.getHeader("X-User-Roles");

        if (username != null && rolesStr != null && !rolesStr.trim().isEmpty()) {
            List<SimpleGrantedAuthority> authorities =
Arrays.stream(rolesStr.split(","))
                    .flatMap(rol -> {
                        String normalized = rol.trim().toUpperCase();
                        return java.util.stream.Stream.of(
                                new SimpleGrantedAuthority(normalized),
                                new SimpleGrantedAuthority("ROLE_" + normalized)
                        );
                    })
                    .collect(Collectors.toList());

            UsernamePasswordAuthenticationToken authentication =
                    new UsernamePasswordAuthenticationToken(username, null,
authorities);

            SecurityContextHolder.getContext().setAuthentication(authentication);
        }

        filterChain.doFilter(request, response);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pagos.config**; CLASE -> **SecurityConfig**

```
package com.lartduniss.pagos.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilte
r;

@Configuration
@EnableWebSecurity
public class SecurityConfig {
```

```
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                // Acceso público para Swagger de Pagos
                .requestMatchers("/api/v1/pagos/v3/api-docs/**", "/swagger-ui/**",
"/swagger-ui.html").permitAll()

                // Endpoints protegidos
                .requestMatchers(HttpMethod.GET,
"/api/v1/pagos/**").hasAnyAuthority("ADMINISTRADOR", "CONTADOR")
                .requestMatchers(HttpMethod.POST,
"/api/v1/pagos/**").hasAnyAuthority("CLIENTE", "ADMINISTRADOR")
                .requestMatchers(HttpMethod.PUT,
"/api/v1/pagos/**").hasAnyAuthority("ADMINISTRADOR")
                .requestMatchers(HttpMethod.DELETE,
"/api/v1/pagos/**").hasAnyAuthority("ADMINISTRADOR")
                .anyRequest().authenticated()
            )
            .exceptionHandling(exception -> exception
                .accessDeniedHandler(new CustomAccessDeniedHandler())
                .authenticationEntryPoint((request, response, authException) -> {
                    response.setContentType("application/json;charset=UTF-8");
                    response.setStatus(403);
                    response.getWriter().write("{\"status\": 403, \"error\":
\"Forbidden\", \"mensaje\": \"Acceso denegado: No posees los privilegios requeridos
para realizar acciones de pago.\"}");
                })
            )
            .addFilterBefore(new RequestHeaderAuthenticationFilter(),
UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pagos.config**; CLASE -> **WebConfig**

```
package com.lartduniss.pagos.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
```

```java
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.lang.NonNull;

@Configuration
public class WebConfig implements WebMvcConfigurer {

   @Override
   public void addCorsMappings(@NonNull CorsRegistry registry) {
       registry.addMapping("/**")
               .allowedOrigins("http://localhost:9090")
               .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
               .allowedHeaders("*")
               .allowCredentials(true);
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pagos.controller**; CLASE -> **PagoController**

```java
package com.lartduniss.pagos.controller;

import java.util.List;
import java.util.stream.Collectors;
import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.linkTo;
import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.methodOn;

import org.springframework.hateoas.CollectionModel;
import org.springframework.hateoas.EntityModel;
import org.springframework.http.ResponseEntity;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import com.lartduniss.pagos.model.Pago;
import com.lartduniss.pagos.service.PagoService;
import org.springframework.lang.NonNull;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;

@RestController
@RequestMapping("/api/v1/pagos")
@Tag(name = "Pagos", description = "Controlador para la gestión de pagos de
pedidos")
```

```java
@SuppressWarnings("null")
public class PagoController {

    private final PagoService pagoService;

    public PagoController(PagoService pagoService) {
        this.pagoService = pagoService;
    }

    @Operation(summary = "Obtener todos los pagos", description = "Retorna una
colección de pagos con sus respectivos enlaces hipermedia")
    @GetMapping
    public CollectionModel<EntityModel<Pago>> listar() {
        List<EntityModel<Pago>> pagos = pagoService.listarTodos().stream()
                .map(pago -> EntityModel.of(pago,

linkTo(methodOn(PagoController.class).obtener(pago.getId())).withSelfRel(),

linkTo(methodOn(PagoController.class).listar()).withRel("pagos")))
                .collect(Collectors.toList());
        return CollectionModel.of(pagos,
                linkTo(methodOn(PagoController.class).listar()).withSelfRel());
    }

    @Operation(summary = "Obtener pago por ID", description = "Retorna un pago
individual con enlaces a acciones relacionadas")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Pago encontrado"),
        @ApiResponse(responseCode = "404", description = "El registro de pago no
existe")
    })
    @GetMapping("/{id}")
    public EntityModel<Pago> obtener(@PathVariable @NonNull Long id) {
        Pago pago = pagoService.buscarPorId(id)
                .orElseThrow(() -> new RuntimeException("Pago no encontrado"));
        return EntityModel.of(pago,
                linkTo(methodOn(PagoController.class).obtener(id)).withSelfRel(),

linkTo(methodOn(PagoController.class).listar()).withRel("todos-los-pagos"),

linkTo(methodOn(PagoController.class).eliminar(id)).withRel("eliminar"));
    }

    @Operation(summary = "Crear un nuevo registro de pago", description = "Procesa
el monto y notifica al microservicio de Pedidos")
```

```
    @PostMapping
    public ResponseEntity<EntityModel<Pago>> crear(@Valid @RequestBody Pago pago) {
        Pago nuevoPago = pagoService.guardar(pago);
        EntityModel<Pago> recurso = EntityModel.of(nuevoPago,

linkTo(methodOn(PagoController.class).obtener(nuevoPago.getId())).withSelfRel(),

linkTo(methodOn(PagoController.class).listar()).withRel("todos-los-pagos"));
        return ResponseEntity.status(HttpStatus.CREATED).body(recurso);
    }

    @Operation(summary = "Eliminar un registro de pago")
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminar(@PathVariable @NonNull Long id) {
        pagoService.eliminar(id);
        return ResponseEntity.noContent().build();
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pagos.exception**; CLASE -> **GlobalExceptionHandler**

```
package com.lartduniss.pagos.exception;

import java.util.HashMap;
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import com.lartduniss.pagos.config.ErrorResponse;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(AccessDeniedException.class)
    public ResponseEntity<ErrorResponse>
handleAccessDeniedException(AccessDeniedException ex) {
        ErrorResponse error = new ErrorResponse(
                HttpStatus.FORBIDDEN.value(),
                "Acceso denegado: Esta función está reservada únicamente para los
roles de CONTADOR y ADMINISTRADOR.",
                System.currentTimeMillis()
```

```
        );
        return new ResponseEntity<>(error, HttpStatus.FORBIDDEN);
    }

    @ExceptionHandler(RuntimeException.class)
    public ResponseEntity<ErrorResponse> handleRuntimeException(RuntimeException ex)
{
        ErrorResponse error = new ErrorResponse(
                HttpStatus.NOT_FOUND.value(),
                ex.getMessage(),
                System.currentTimeMillis()
        );
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>>
handleValidationExceptions(MethodArgumentNotValidException ex) {
        Map<String, String> errores = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error -> {
            errores.put(error.getField(), error.getDefaultMessage());
        });
        return new ResponseEntity<>(errores, HttpStatus.BAD_REQUEST);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pagos.model**; CLASE -> **Pago**

```
package com.lartduniss.pagos.model;

import java.time.LocalDate;
import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
```

```java
@Data
@Schema(description = "Representa el pago de un pedido")
@AllArgsConstructor
@NoArgsConstructor
public class Pago {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   @Schema(description = "ID único autoincremental del registro de pago", example =
"1")
   private Long id;

   @NotNull(message = "El id del pedido es obligatorio")
   @Positive(message = "El id del pedido debe ser un valor positivo")
   @Schema(description = "ID del pedido proveniente del microservicio de Pedidos",
example = "10000", requiredMode = Schema.RequiredMode.REQUIRED)
   private Long pedidoId;

   @NotNull(message = "El monto total es obligatorio")
   @Positive(message = "El monto total debe ser un valor positivo")
   @Schema(description = "Monto total a pagar por el pedido", example = "15500.00")
   private Double montoTotal;

   @NotNull(message = "El monto pagado no puede ser nulo")
   @Positive(message = "El monto pagado debe ser un valor positivo")
   @Schema(description = "Monto que el cliente ya ha cancelado", example =
"15500.00")
   private Double montoPagado;

   @NotBlank(message = "El método de pago es obligatorio")
   @Schema(description = "Método utilizado para el pago", example = "Webpay",
allowableValues = {"Webpay", "Transferencia", "Efectivo"})
   private String metodoPago;

   @NotNull(message = "La fecha de pago es obligatoria")
   @Schema(description = "Fecha en la que se procesó el pago", example =
"2026-06-16")
   private LocalDate fechaPago;

   @NotBlank(message = "El estado del pago es obligatorio")
   @Schema(description = "Estado actual de la transacción", example = "APROBADO")
   private String estadoPago;
}
```

RUTA -> SRC/MAIN/JAVA

PACKAGE -> com.lartduniss.pagos.repository; INTERFAZ -> PagoRepository

```java
package com.lartduniss.pagos.repository;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.lartduniss.pagos.model.Pago;

@Repository
public interface PagoRepository extends JpaRepository<Pago, Long> {
   List<Pago> findByPedidoId(Long pedidoId);
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> com.lartduniss.pagos.service; INTERFAZ -> PagoService

```java
package com.lartduniss.pagos.service;

import java.time.LocalDate;
import java.util.List;
import java.util.Optional;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;
import com.lartduniss.pagos.model.Pago;
import com.lartduniss.pagos.repository.PagoRepository;
import org.springframework.lang.NonNull;
import jakarta.transaction.Transactional;

@Service
public class PagoService {

   private final PagoRepository pagoRepository;
   private final RestTemplate restTemplate;

   public PagoService(PagoRepository pagoRepository, RestTemplate restTemplate) {
       this.pagoRepository = pagoRepository;
       this.restTemplate = restTemplate;
   }

   public List<Pago> listarTodos() {
       return pagoRepository.findAll();
   }

   public Optional<Pago> buscarPorId(@NonNull Long id) {
```

```
        return pagoRepository.findById(id);
    }

    @Transactional
    public Pago guardar(@NonNull Pago pago) {
        double abonoMinimo = pago.getMontoTotal() * 0.5;

        if (pago.getMontoPagado() >= pago.getMontoTotal()) {
            pago.setEstadoPago("PAGADO_TOTAL");
        } else if (pago.getMontoPagado() >= abonoMinimo) {
            pago.setEstadoPago("ABONO_CONFIRMADO");
        } else {
            pago.setEstadoPago("MONTO_INSUFICIENTE");
        }

        pago.setFechaPago(LocalDate.now());
        Pago pagoGuardado = pagoRepository.save(pago);

        if (!pagoGuardado.getEstadoPago().equals("MONTO_INSUFICIENTE")) {
            try {
                String urlPedido = "http://localhost:9090/pedidos/" +
pagoGuardado.getPedidoId() + "/estado";
                String nuevoEstadoPedido = pagoGuardado.getEstadoPago();
                restTemplate.put(urlPedido, nuevoEstadoPedido);
            } catch (Exception e) {
                System.err.println("Error al notificar al microservicio de Pedidos:
" + e.getMessage());
            }
        }

        return pagoGuardado;
    }

    @Transactional
    public void eliminar(@NonNull Long id) {
        pagoRepository.deleteById(id);
    }
}
```

RUTA -> SRC/MAIN/RESOURCES
APPLICATION.PROPERTIES

```
spring.application.name=service-pago
server.port=8093
```

```
spring.datasource.url=jdbc:mysql://localhost:3306/db_lart_pagos?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# Indicar a Spring que respete los encabezados X-Forwarded Host, Port, Proto!!!!
server.forward-headers-strategy=framework

# Configuración personalizada para que la documentación de OpenAPI funcione con el Gateway
springdoc.api-docs.path=/api/v1/pagos/v3/api-docs

# Configuración de Tomcat para interceptar las cabeceras del Gateway correctamente
server.tomcat.remote-ip-header=x-forwarded-for
server.tomcat.protocol-header=x-forwarded-proto
server.tomcat.port-header=x-forwarded-port
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> **com.lartduniss.pagos**; CLASE -> **PagoApplicationTests**

```
package com.lartduniss.pagos;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.autoconfigure.domain.EntityScan;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;

@SpringBootTest(properties = {
  "spring.main.allow-bean-definition-overriding=true",
  "springdoc.api-docs.enabled=false",
  "springdoc.swagger-ui.enabled=false",

"spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.security.servlet.SecurityAutoConfiguration",
  "spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;MODE=MySQL",
  "spring.datasource.driver-class-name=org.h2.Driver"
})
@ComponentScan(basePackages = {
  "com.lartduniss.pagos",
  "com.lartduniss.pagos.config",
  "com.lartduniss.pagos.controller",
  "com.lartduniss.pagos.service",
```

```
  "com.lartduniss.pagos.exception"
})
@EntityScan(basePackages = {"com.lartduniss.pagos.model"})
@EnableJpaRepositories(basePackages = {"com.lartduniss.pagos.repository"})
class PagosApplicationTests {

  @Test
  void contextLoads() {
  }
}
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> **com.lartduniss.pagos.service**; CLASE -> **PagoServiceTest**

```
package com.lartduniss.pagos.service;

import com.lartduniss.pagos.model.Pago;
import com.lartduniss.pagos.repository.PagoRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.Mockito;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.web.client.RestTemplate;

import java.time.LocalDate;

import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.anyString;

@ExtendWith(MockitoExtension.class)
class PagoServiceTest {

  @Mock
  private PagoRepository pagoRepository;

  @Mock
  private RestTemplate restTemplate;

  @InjectMocks
  private PagoService pagoService;
```

```
  @Test
  void guardarPagoTotalExitosoTest() {
      // 1. Arrange
      Pago mockPagoInput = new Pago(null, 100L, 20000.0, 20000.0, "Webpay", null,
null);
      Pago mockPagoGuardado = new Pago(1L, 100L, 20000.0, 20000.0, "Webpay",
LocalDate.now(), "PAGADO_TOTAL");


Mockito.when(pagoRepository.save(any(Pago.class))).thenReturn(mockPagoGuardado);

      Mockito.doNothing().when(restTemplate).put(anyString(), any());

      // 2. Act
      Pago resultado = pagoService.guardar(mockPagoInput);

      // 3. Assert
      assertNotNull(resultado);
      assertEquals("PAGADO_TOTAL", resultado.getEstadoPago());
      Mockito.verify(pagoRepository, Mockito.times(1)).save(mockPagoInput);
  }

  @Test
  void guardarPagoMontoInsuficienteTest() {
      // 1. Arrange
      Pago mockPagoInput = new Pago(null, 101L, 20000.0, 5000.0, "Efectivo", null,
null);
      Pago mockPagoGuardado = new Pago(2L, 101L, 20000.0, 5000.0, "Efectivo",
LocalDate.now(), "MONTO_INSUFICIENTE");


Mockito.when(pagoRepository.save(any(Pago.class))).thenReturn(mockPagoGuardado);

      // 2. Act
      Pago resultado = pagoService.guardar(mockPagoInput);

      // 3. Assert
      assertNotNull(resultado);
      assertEquals("MONTO_INSUFICIENTE", resultado.getEstadoPago());
      Mockito.verify(pagoRepository, Mockito.times(1)).save(mockPagoInput);
      Mockito.verify(restTemplate, Mockito.never()).put(anyString(), any());
  }
}
```

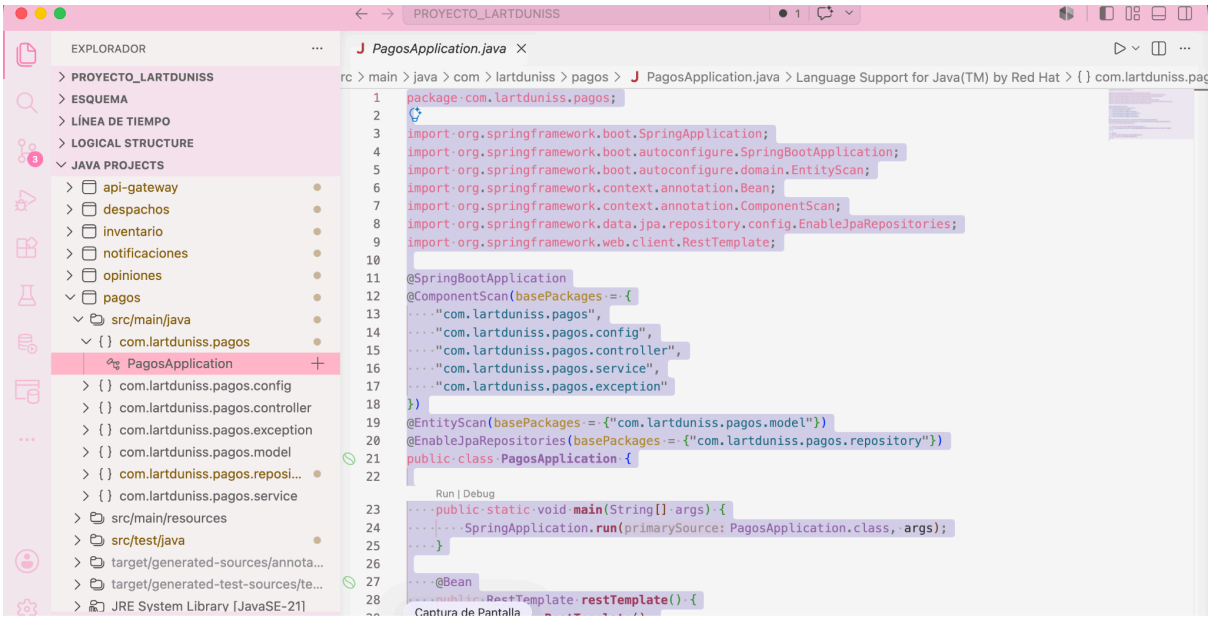
POM.XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>
   <parent>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-parent</artifactId>
       <version>3.5.15</version>
       <relativePath/> <!-- lookup parent from repository -->
   </parent>
   <groupId>com.lartduniss</groupId>
   <artifactId>pagos</artifactId>
   <version>0.0.1-SNAPSHOT</version>
   <name>pagos</name>
   <description/>
   <url/>
   <licenses>
       <license/>
   </licenses>
   <developers>
       <developer/>
   </developers>
   <scm>
       <connection/>
       <developerConnection/>
       <tag/>
       <url/>
   </scm>
   <properties>
       <java.version>21</java.version>
   </properties>
   <dependencies>
   <dependency>
   <groupId>com.h2database</groupId>
   <artifactId>h2</artifactId>
   <scope>test</scope>
</dependency>
       <dependency>
           <groupId>org.springframework.boot</groupId>
           <artifactId>spring-boot-starter-data-jpa</artifactId>
       </dependency>
       <dependency>
```

```
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
                    <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>

        <dependency>
            <groupId>org.springdoc</groupId>
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
            <version>2.8.17</version>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-hateoas</artifactId>
        </dependency>

    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
```

```
                <excludes>
                    <exclude>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </exclude>
                </excludes>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <executions>
                <execution>
                    <id>default-compile</id>
                    <phase>compile</phase>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok</artifactId>
                            </path>
                        </annotationProcessorPaths>
                    </configuration>
                </execution>
                <execution>
                    <id>default-testCompile</id>
                    <phase>test-compile</phase>
                    <goals>
                        <goal>testCompile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok</artifactId>
                            </path>
                        </annotationProcessorPaths>
                    </configuration>
                </execution>
            </executions>
        </plugin>
    </plugins>
```

```
    </build>

</project>
```



- 6. PEDIDO:

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pedido;** CLASE -> **PedidoApplication**

```
package com.lartduniss.pedido;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.autoconfigure.domain.EntityScan;
import org.springframework.context.annotation.Bean;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;
import org.springframework.web.client.RestTemplate;

@SpringBootApplication
@EntityScan(basePackages = {"com.lartduniss.pedido.model"})
@EnableJpaRepositories(basePackages = {"com.lartduniss.pedido.repository"})
public class PedidoApplication {

   public static void main(String[] args) {
       SpringApplication.run(PedidoApplication.class, args);
```

```
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }

}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pedido.config;** CLASE -> **CustomAccessDeniedHandler**

```
package com.lartduniss.pedido.config;

import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.HttpStatus;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.web.access.AccessDeniedHandler;
import java.io.IOException;
import java.time.LocalDateTime;

public class CustomAccessDeniedHandler implements AccessDeniedHandler {

    @Override
    public void handle(HttpServletRequest request, HttpServletResponse response,
                       AccessDeniedException accessDeniedException) throws
IOException, ServletException {

        response.setContentType("application/json;charset=UTF-8");
        response.setStatus(HttpStatus.FORBIDDEN.value());

        String jsonResponse = String.format(
                "{\"timestamp\": \"%s\", \"status\": 403, \"error\": \"Forbidden\",
" +
                "\"mensaje\": \"Lo sentimos, tu cuenta de CLIENTE no tiene permisos
para modificar o gestionar estados de órdenes de compra. Esta acción es exclusiva
de ADMINISTRADORES.\"}",
                LocalDateTime.now()
        );

        response.getWriter().write(jsonResponse);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pedido.config;** CLASE -> **ErrorResponse**

```
package com.lartduniss.pedido.config;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class ErrorResponse {
   private int status;
   private String message;
   private long timestamp;
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pedido.config;** CLASE -> **OpenApiConfig**

```
package com.lartduniss.pedido.config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.servers.Server;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import java.util.List;

@Configuration
public class OpenApiConfig {

   @Bean
   public OpenAPI customOpenAPI() {
       return new OpenAPI()
               .info(new Info()
                       .title("L'art du niss - Microservicio de Pedidos")
                       .version("1.0")
                       .description("Documentación centralizada del Sistema de
Pedidos y Ventas"))
               .servers(List.of(
```

```
                        new
Server().url("http://localhost:8094").description("Puerto Local del Microservicio")
                ));
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pedido.config;** CLASE -> **RequestHeaderAuthenticationFilter**

```
package com.lartduniss.pedido.config;

import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.filter.OncePerRequestFilter;
import org.springframework.lang.NonNull;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class RequestHeaderAuthenticationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(@NonNull HttpServletRequest request, @NonNull
HttpServletResponse response, @NonNull FilterChain filterChain)
            throws ServletException, IOException {

        String username = request.getHeader("X-User-Username");
        if (username == null) {
            username = request.getHeader("X-Username");
        }
        String rolesStr = request.getHeader("X-User-Roles");

        if (username != null && rolesStr != null && !rolesStr.trim().isEmpty()) {
            List<SimpleGrantedAuthority> authorities =
Arrays.stream(rolesStr.split(","))
                    .flatMap(rol -> {
                        String normalized = rol.trim().toUpperCase();
```

```
                        return java.util.stream.Stream.of(
                                new SimpleGrantedAuthority(normalized),
                                new SimpleGrantedAuthority("ROLE_" + normalized)
                        );
                    })
                    .collect(Collectors.toList());

            UsernamePasswordAuthenticationToken authentication =
                    new UsernamePasswordAuthenticationToken(username, null,
authorities);

            SecurityContextHolder.getContext().setAuthentication(authentication);
        }

        filterChain.doFilter(request, response);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pedido.config;** CLASE -> **SecurityConfig**

```
package com.lartduniss.pedido.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilte
r;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
```

```
                .requestMatchers("/api/v1/pedidos/v3/api-docs/**", "/swagger-ui/**",
"/swagger-ui.html").permitAll()

                .requestMatchers(HttpMethod.GET,
"/api/v1/pedidos/**").hasAnyAuthority("CLIENTE", "ADMINISTRADOR")
                .requestMatchers(HttpMethod.POST,
"/api/v1/pedidos/**").hasAnyAuthority("CLIENTE", "ADMINISTRADOR")
                .requestMatchers(HttpMethod.PUT,
"/api/v1/pedidos/**").hasAnyAuthority("ADMINISTRADOR")
                .requestMatchers(HttpMethod.DELETE,
"/api/v1/pedidos/**").hasAnyAuthority("ADMINISTRADOR")
                .anyRequest().authenticated()
            )
            .exceptionHandling(exception -> exception
                .accessDeniedHandler(new CustomAccessDeniedHandler())
                .authenticationEntryPoint((request, response, authException) -> {
                    response.setContentType("application/json;charset=UTF-8");
                    response.setStatus(403);
                    response.getWriter().write("{\"status\": 403, \"error\":
\"Forbidden\", \"mensaje\": \"Acceso denegado: No se han proporcionado cabeceras de
autenticación válidas para gestionar el pedido.\"}");
                })
            )
            .addFilterBefore(new RequestHeaderAuthenticationFilter(),
UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pedido.config;** CLASE -> **WebConfig**

```
package com.lartduniss.pedido.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.lang.NonNull;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(@NonNull CorsRegistry registry) {
```

```
        registry.addMapping("/**")
                .allowedOrigins("http://localhost:9090")
                .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
                .allowedHeaders("*")
                .allowCredentials(true);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pedido.controller;** CLASE -> **PedidoController**

```
package com.lartduniss.pedido.controller;

import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.linkTo;
import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.methodOn;

import java.util.List;
import java.util.stream.Collectors;
import org.springframework.hateoas.CollectionModel;
import org.springframework.hateoas.EntityModel;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.lang.NonNull;
import org.springframework.web.bind.annotation.*;

import com.lartduniss.pedido.model.Pedido;
import com.lartduniss.pedido.service.PedidoService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;

@RestController
@RequestMapping("/api/v1/pedidos")
@Tag(name = "Pedidos", description = "Operaciones relacionadas con la gestión de
pedidos con soporte HATEOAS")
@SuppressWarnings("null")
public class PedidoController {

    private final PedidoService pedidoService;

    public PedidoController(PedidoService pedidoService) {
        this.pedidoService = pedidoService;
    }
```

```
    @Operation(summary = "Obtener todos los pedidos", description = "Retorna una
colección de pedidos con sus respectivos enlaces hipermedia")
    @GetMapping
    public CollectionModel<EntityModel<Pedido>> listar() {
        List<EntityModel<Pedido>> pedidos = pedidoService.listarTodos().stream()
                .map(pedido -> EntityModel.of(pedido,

linkTo(methodOn(PedidoController.class).obtenerPorId(pedido.getId())).withSelfRel()
,

linkTo(methodOn(PedidoController.class).listar()).withRel("pedidos")))
                .collect(Collectors.toList());
        return CollectionModel.of(pedidos,
                linkTo(methodOn(PedidoController.class).listar()).withSelfRel());
    }

    @Operation(summary = "Crear un nuevo pedido", description = "Registra el pedido
e inicializa su estado como PENDIENTE_PAGO de forma automática")
    @PostMapping
    public ResponseEntity<EntityModel<Pedido>> crear(@Valid @RequestBody Pedido
pedido) {
        Pedido nuevoPedido = pedidoService.crear(pedido);
        EntityModel<Pedido> recurso = EntityModel.of(nuevoPedido,

linkTo(methodOn(PedidoController.class).obtenerPorId(nuevoPedido.getId())).withSelf
Rel(),

linkTo(methodOn(PedidoController.class).listar()).withRel("todos-los-pedidos"));
        return ResponseEntity.status(HttpStatus.CREATED).body(recurso);
    }

    @Operation(summary = "Obtener pedido por ID", description = "Retorna un pedido
individual con enlaces a acciones relacionadas")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Pedido encontrado"),
        @ApiResponse(responseCode = "404", description = "El pedido no existe")
    })
    @GetMapping("/{id}")
    public EntityModel<Pedido> obtenerPorId(@PathVariable @NonNull Long id) {
        Pedido pedido = pedidoService.buscarPorId(id)
                .orElseThrow(() -> new RuntimeException("Pedido no encontrado"));
        return EntityModel.of(pedido,

linkTo(methodOn(PedidoController.class).obtenerPorId(id)).withSelfRel(),
```

```
linkTo(methodOn(PedidoController.class).listar()).withRel("todos-los-pedidos"),
                linkTo(methodOn(PedidoController.class).actualizarEstado(id,
null)).withRel("actualizar-estado"));
    }

    @Operation(summary = "Actualizar el estado de un pedido", description =
"Endpoint utilizado principalmente por el microservicio de Pagos para cambiar el
estado de la transacción")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Estado del pedido
actualizado correctamente"),
        @ApiResponse(responseCode = "404", description = "El pedido a actualizar no
existe")
    })
    @PutMapping("/{id}/estado")
    public ResponseEntity<EntityModel<Pedido>> actualizarEstado(@PathVariable Long
id, @RequestBody String nuevoEstado) {
        Pedido actualizado = pedidoService.actualizarEstado(id, nuevoEstado);
        EntityModel<Pedido> recurso = EntityModel.of(actualizado,

linkTo(methodOn(PedidoController.class).obtenerPorId(id)).withSelfRel(),

linkTo(methodOn(PedidoController.class).listar()).withRel("todos-los-pedidos"));
        return ResponseEntity.ok(recurso);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pedido.exception;** CLASE -> **GlobalExceptionHandler**

```
package com.lartduniss.pedido.exception;

import java.util.HashMap;
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import com.lartduniss.pedido.config.ErrorResponse;

@RestControllerAdvice
public class GlobalExceptionHandler {
```

```
    @ExceptionHandler(AccessDeniedException.class)
    public ResponseEntity<ErrorResponse>
handleAccessDeniedException(AccessDeniedException ex) {
        ErrorResponse error = new ErrorResponse(
                HttpStatus.FORBIDDEN.value(),
                "Lo sentimos, tu cuenta de CLIENTE no tiene permisos para modificar
o gestionar estados de órdenes de compra. Esta acción es exclusiva de
ADMINISTRADORES.",
                System.currentTimeMillis()
        );
        return new ResponseEntity<>(error, HttpStatus.FORBIDDEN);
    }

    @ExceptionHandler(RuntimeException.class)
    public ResponseEntity<ErrorResponse> handleRuntimeException(RuntimeException ex)
{
        ErrorResponse error = new ErrorResponse(
                HttpStatus.NOT_FOUND.value(),
                ex.getMessage(),
                System.currentTimeMillis()
        );
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>>
handleValidationExceptions(MethodArgumentNotValidException ex) {
        Map<String, String> errores = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error -> {
            errores.put(error.getField(), error.getDefaultMessage());
        });
        return new ResponseEntity<>(errores, HttpStatus.BAD_REQUEST);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pedido.model;** CLASE -> **Pedido**

```
package com.lartduniss.pedido.model;

import java.time.LocalDate;
import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
```

```java
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "pedidos")
@Data
@Schema(description = "Modelo que representa un pedido dentro del sistema de ventas")
@AllArgsConstructor
@NoArgsConstructor
public class Pedido {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   @Schema(description = "ID único autoincremental del pedido", example = "10000")
   private Long id;

   @NotNull(message = "El id del cliente no puede estar vacío")
   @Positive(message = "El id del cliente debe ser un valor positivo")
   @Schema(description = "ID del cliente proveniente del microservicio de Clientes", example = "55", requiredMode = Schema.RequiredMode.REQUIRED)
   private Long clienteId;

   @Schema(description = "Fecha en la que se registró el pedido en el sistema", example = "2026-06-16")
   private LocalDate fechaCreacion;

   @NotNull(message = "El monto total es obligatorio")
   @Positive(message = "El monto total debe ser un valor positivo")
   @Schema(description = "Monto total calculado para el pedido", example = "15500.00", requiredMode = Schema.RequiredMode.REQUIRED)
   private Double montoTotal;

   @NotBlank(message = "El estado del pedido es obligatorio")
   @Schema(description = "Estado actual del ciclo del pedido", example = "PENDIENTE_PAGO", allowableValues = {"PENDIENTE_PAGO", "ABONO_CONFIRMADO", "PAGADO_TOTAL", "CANCELADO"})
   private String estadoPedido;
```

```
   @Schema(description = "Notas adicionales o instrucciones especiales para el
pedido", example = "Entregar después de las 18:00 hrs")
   private String observaciones;
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pedido.repository;** INTERFAZ -> **PedidoRepository**

```
package com.lartduniss.pedido.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.lartduniss.pedido.model.Pedido;

@Repository
public interface PedidoRepository extends JpaRepository<Pedido, Long> {
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.pedido.service;** CLASE -> **PedidoService**

```
package com.lartduniss.pedido.service;

import java.time.LocalDate;
import java.util.List;
import java.util.Optional;
import org.springframework.stereotype.Service;
import org.springframework.lang.NonNull;
import jakarta.transaction.Transactional;
import com.lartduniss.pedido.model.Pedido;
import com.lartduniss.pedido.repository.PedidoRepository;

@Service
public class PedidoService {

   private final PedidoRepository pedidoRepository;
```

```
    public PedidoService(PedidoRepository pedidoRepository) {
        this.pedidoRepository = pedidoRepository;
    }
    public List<Pedido> listarTodos() {
        return pedidoRepository.findAll();
    }

    public Optional<Pedido> buscarPorId(@NonNull Long id) {
        return pedidoRepository.findById(id);
    }

    @Transactional
    public Pedido crear(@NonNull Pedido pedido) {
        pedido.setFechaCreacion(LocalDate.now());
        pedido.setEstadoPedido("PENDIENTE_PAGO");
        return pedidoRepository.save(pedido);
    }

    @Transactional
    public Pedido actualizarEstado(@NonNull Long id, String nuevoEstado) {
        Pedido pedido = pedidoRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Pedido no encontrado con el
ID: " + id));

        pedido.setEstadoPedido(nuevoEstado.replace("\"", "").trim()); // Limpia
posibles comillas residuales del string body
        return pedidoRepository.save(pedido);
    }
}
```

RUTA -> SRC/MAIN/RESOURCES
APPLICATION.PROPERTIES

```
spring.application.name=service-pedidos
server.port=8094

spring.datasource.url=jdbc:mysql://localhost:3306/db_lart_pedidos?createDatabaseIfN
otExist=true
spring.datasource.username=root
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# Indicar a Spring que respete los encabezados X-Forwarded Host, Port, Proto
```

```
server.forward-headers-strategy=framework

# Configuracion unificada para mapear la documentacion a traves del Gateway
springdoc.api-docs.path=/api/v1/pedidos/v3/api-docs

# Configuracion de Tomcat para interceptar las cabeceras del Gateway
server.tomcat.remote-ip-header=x-forwarded-for
server.tomcat.protocol-header=x-forwarded-proto
server.tomcat.port-header=x-forwarded-port
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> **com.lartduniss.pedido;** CLASE -> **PedidoApplicationTests**

```
package com.lartduniss.pedido;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.autoconfigure.domain.EntityScan;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;

@SpringBootTest(properties = {
  "spring.main.allow-bean-definition-overriding=true",
  "springdoc.api-docs.enabled=false",
  "springdoc.swagger-ui.enabled=false",

"spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.security.servl
et.SecurityAutoConfiguration",
  "spring.datasource.url=jdbc:h2:mem:testdb_pedido;DB_CLOSE_DELAY=-1;MODE=MySQL",
  "spring.datasource.driver-class-name=org.h2.Driver"
})
@ComponentScan(basePackages = {
  "com.lartduniss.pedido",
  "com.lartduniss.pedido.config",
  "com.lartduniss.pedido.controller",
  "com.lartduniss.pedido.service",
  "com.lartduniss.pedido.exception"
})
@EntityScan(basePackages = {"com.lartduniss.pedido.model"})
@EnableJpaRepositories(basePackages = {"com.lartduniss.pedido.repository"})
class PedidoApplicationTests {

  @Test
  void contextLoads() {
  }
```

```
}
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> **com.lartduniss.pedido.service;** CLASE -> **PedidoServiceTest**

```
package com.lartduniss.pedido.service;

import com.lartduniss.pedido.model.Pedido;
import com.lartduniss.pedido.repository.PedidoRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.Mockito;
import org.mockito.junit.jupiter.MockitoExtension;

import java.time.LocalDate;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.ArgumentMatchers.any;

@ExtendWith(MockitoExtension.class)
class PedidoServiceTest {

    @Mock
    private PedidoRepository pedidoRepository;

    @InjectMocks
    private PedidoService pedidoService;

    @Test
    void crearPedidoInicializaEstadoTest() {
        // 1. Arrange
        Pedido pedidoInput = new Pedido(null, 55L, null, 15500.0, "PENDIENTE",
"Notas");
        Pedido pedidoGuardado = new Pedido(10000L, 55L, LocalDate.now(), 15500.0,
"PENDIENTE_PAGO", "Notas");


Mockito.when(pedidoRepository.save(any(Pedido.class))).thenReturn(pedidoGuardado);

        // 2. Act
        Pedido resultado = pedidoService.crear(pedidoInput);
```

```
        // 3. Assert
        assertNotNull(resultado);
        assertEquals("PENDIENTE_PAGO", resultado.getEstadoPedido());
        Mockito.verify(pedidoRepository, Mockito.times(1)).save(pedidoInput);
    }

    @Test
    void actualizarEstadoPedidoExitosoTest() {
        // 1. Arrange
        Long pedidoId = 10000L;
        Pedido pedidoExistente = new Pedido(pedidoId, 55L, LocalDate.now(), 15500.0,
"PENDIENTE_PAGO", "Notas");
        Pedido pedidoActualizado = new Pedido(pedidoId, 55L, LocalDate.now(),
15500.0, "PAGADO_TOTAL", "Notas");


Mockito.when(pedidoRepository.findById(pedidoId)).thenReturn(Optional.of(pedidoExis
tente));

Mockito.when(pedidoRepository.save(any(Pedido.class))).thenReturn(pedidoActualizado
);

        // 2. Act
        Pedido resultado = pedidoService.actualizarEstado(pedidoId, "PAGADO_TOTAL");

        // 3. Assert
        assertNotNull(resultado);
        assertEquals("PAGADO_TOTAL", resultado.getEstadoPedido());
        Mockito.verify(pedidoRepository, Mockito.times(1)).findById(pedidoId);
        Mockito.verify(pedidoRepository, Mockito.times(1)).save(pedidoExistente);
    }
}
```

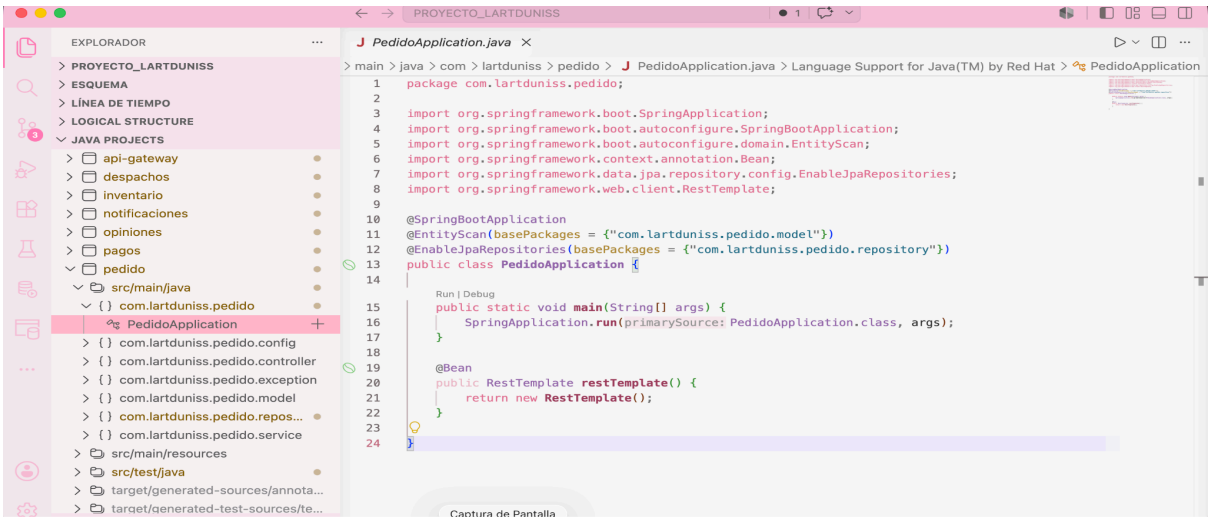
POM.XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
```

```
        <version>3.5.15</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.lartduniss</groupId>
    <artifactId>pedido</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>pedido</name>
    <description/>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>21</java.version>
    </properties>
    <dependencies>
    <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>test</scope>
</dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
                    <artifactId>spring-boot-starter-security</artifactId>
```

```
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>

        <dependency>
            <groupId>org.springdoc</groupId>
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
            <version>2.8.17</version>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-hateoas</artifactId>
        </dependency>

    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <excludes>
                        <exclude>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </exclude>
                    </excludes>
                </configuration>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
```

```
            <artifactId>maven-compiler-plugin</artifactId>
            <executions>
                <execution>
                    <id>default-compile</id>
                    <phase>compile</phase>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok</artifactId>
                            </path>
                        </annotationProcessorPaths>
                    </configuration>
                </execution>
                <execution>
                    <id>default-testCompile</id>
                    <phase>test-compile</phase>
                    <goals>
                        <goal>testCompile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok</artifactId>
                            </path>
                        </annotationProcessorPaths>
                    </configuration>
                </execution>
            </executions>
        </plugin>
    </plugins>
  </build>

</project>
```



- 7. PRODUCTO:

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.productos;** CLASE -> **ProductosApplication**

```
package com.lartduniss.productos;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.autoconfigure.domain.EntityScan;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;

@SpringBootApplication
@ComponentScan(basePackages = {"com.lartduniss.productos", "controller",
"service"})
@EntityScan(basePackages = {"com.lartduniss.productos.model", "model"})
@EnableJpaRepositories(basePackages = {"com.lartduniss.productos.repository",
"repository"})
public class ProductosApplication {
   public static void main(String[] args) {
        SpringApplication.run(ProductosApplication.class, args);
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.productos.config;** CLASE -> **CustomAccessDeniedHandler**

```
package com.lartduniss.productos.config;
```

```
import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.security.web.access.AccessDeniedHandler;
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;

public class CustomAccessDeniedHandler implements AccessDeniedHandler {

   @Override
   public void handle(HttpServletRequest request, HttpServletResponse response,
                      org.springframework.security.access.AccessDeniedException
accessDeniedException)
           throws IOException, ServletException {

       response.setStatus(HttpStatus.FORBIDDEN.value());
       response.setContentType(MediaType.APPLICATION_JSON_VALUE);

       Map<String, Object> data = new HashMap<>();
       data.put("statusCode", HttpStatus.FORBIDDEN.value());
       data.put("message", "Acceso denegado: No posees los privilegios de
administrador para alterar el catálogo.");
       data.put("timestamp", System.currentTimeMillis());

       ObjectMapper objectMapper = new ObjectMapper();
       response.getOutputStream().println(objectMapper.writeValueAsString(data));
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.productos.config;** CLASE -> **OpenConfig**

```
package com.lartduniss.productos.config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.servers.Server;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import java.util.List;
```

```
@Configuration
public class OpenApiConfig {

   @Bean
   public OpenAPI customOpenAPI() {
       return new OpenAPI()
               .info(new Info()
                       .title("API Lartduniss - Catálogo de Productos")
                       .version("1.0")
                       .description("Servicio encargado de la gestión de
existencias y precios de catálogo"))
               .servers(List.of(
                       new Server().url("http://localhost:9090").description("API
Gateway (Entrada Unificada)")
               ));
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.productos.config;** CLASE -> **RequestHeaderAuthenticationFilter**

```
package com.lartduniss.productos.config;

import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.filter.OncePerRequestFilter;
import org.springframework.lang.NonNull;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class RequestHeaderAuthenticationFilter extends OncePerRequestFilter {

   @Override
   protected void doFilterInternal(@NonNull HttpServletRequest request, @NonNull
HttpServletResponse response, @NonNull FilterChain filterChain)
```

```
            throws ServletException, IOException {

        String username = request.getHeader("X-Username");
        if (username == null) {
            username = request.getHeader("X-User-Username");
        }
        String rolesStr = request.getHeader("X-User-Roles");

        if (username != null && rolesStr != null && !rolesStr.trim().isEmpty()) {
            List<SimpleGrantedAuthority> authorities =
Arrays.stream(rolesStr.split(","))
                    .flatMap(rol -> {
                        String normalized = rol.trim().toUpperCase();
                        return java.util.stream.Stream.of(
                                new SimpleGrantedAuthority(normalized),
                                new SimpleGrantedAuthority("ROLE_" + normalized)
                        );
                    })
                    .collect(Collectors.toList());

            UsernamePasswordAuthenticationToken authentication =
                    new UsernamePasswordAuthenticationToken(username, null,
authorities);

            SecurityContextHolder.getContext().setAuthentication(authentication);
        }
        filterChain.doFilter(request, response);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.productos.config;** CLASE -> **SecurityConfig**

```
package com.lartduniss.productos.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.access.AccessDeniedHandler;
```

```
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilte
r;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

   @Bean
   public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
       http
           .csrf(csrf -> csrf.disable())
           .authorizeHttpRequests(auth -> auth
               .requestMatchers(HttpMethod.GET, "/api/v1/productos/**").permitAll()
               .requestMatchers(HttpMethod.POST,
"/api/v1/productos/**").hasAnyRole("ADMINISTRADOR", "ADMIN")
               .requestMatchers(HttpMethod.PUT,
"/api/v1/productos/**").hasAnyRole("ADMINISTRADOR", "ADMIN")
               .requestMatchers("/api/v1/productos/v3/api-docs/**",
"/swagger-ui/**", "/swagger-ui.html").permitAll()
               .anyRequest().authenticated()
           )
           .exceptionHandling(exception -> exception
               .accessDeniedHandler(accessDeniedHandler())
           )
           .addFilterBefore(new RequestHeaderAuthenticationFilter(),
UsernamePasswordAuthenticationFilter.class);

       return http.build();
   }

   @Bean
   public AccessDeniedHandler accessDeniedHandler() {
       return new CustomAccessDeniedHandler();
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.productos.config;** CLASE -> **WebConfig**

```
package com.lartduniss.productos.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
```

```java
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.lang.NonNull;

@Configuration
public class WebConfig implements WebMvcConfigurer {

   @Override
   public void addCorsMappings(@NonNull CorsRegistry registry) {
       registry.addMapping("/**")
               .allowedOrigins("http://localhost:8080", "http://localhost:9090")
               .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
               .allowedHeaders("*")
               .allowCredentials(true);
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.productos.dto;** CLASE -> **ErrorResponse**

```java
package com.lartduniss.productos.dto;

import java.time.LocalDateTime;

public class ErrorResponse {
   private int status;
   private String error;
   private String mensaje;
   private LocalDateTime timestamp;

   public ErrorResponse(int status, String error, String mensaje) {
       this.status = status;
       this.error = error;
       this.mensaje = mensaje;
       this.timestamp = LocalDateTime.now();
   }

   // Getters y Setters
   public int getStatus() { return status; }
   public void setStatus(int status) { this.status = status; }

   public String getError() { return error; }
   public void setError(String error) { this.error = error; }

   public String getMensaje() { return mensaje; }
   public void setMensaje(String mensaje) { this.mensaje = mensaje; }
```

```
    public LocalDateTime getTimestamp() { return timestamp; }
    public void setTimestamp(LocalDateTime timestamp) { this.timestamp = timestamp;
}
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **controller;** CLASE -> **ProductoController**

```
package controller;

import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.linkTo;
import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.methodOn;

import java.util.List;
import java.util.stream.Collectors;
import org.springframework.hateoas.CollectionModel;
import org.springframework.hateoas.EntityModel;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.lang.NonNull;
import org.springframework.web.bind.annotation.*;

import model.Producto;
import service.ProductoService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;

@RestController
@RequestMapping("/api/v1/productos")
@Tag(name = "Productos", description = "Controlador para la gestión del catálogo de
productos de Lartduniss")
@SuppressWarnings("null")
public class ProductoController {

    private final ProductoService productoService;

    public ProductoController(ProductoService productoService) {
        this.productoService = productoService;
```

```
    }

    @Operation(summary = "Obtener todos los productos", description = "Retorna el
catálogo completo con enlaces hipermedia HATEOAS")
    @GetMapping
    public CollectionModel<EntityModel<Producto>> listar() {
        List<EntityModel<Producto>> productos =
productoService.obtenerTodos().stream()
                .map(producto -> EntityModel.of(producto,

linkTo(methodOn(ProductoController.class).obtenerUno(producto.getId())).withSelfRel
(),

linkTo(methodOn(ProductoController.class).listar()).withRel("productos")))
                .collect(Collectors.toList());

        return CollectionModel.of(productos,
                linkTo(methodOn(ProductoController.class).listar()).withSelfRel());
    }

    @Operation(summary = "Guardar un nuevo producto", description = "Registra un
producto en el catálogo verificando reglas de precio y stock. Acción exclusiva de
ADMINISTRADORES.")
    @PostMapping
    public ResponseEntity<EntityModel<Producto>> crear(@NonNull @Valid @RequestBody
Producto producto) {
        Producto nuevo = productoService.guardar(producto);
        EntityModel<Producto> recurso = EntityModel.of(nuevo,

linkTo(methodOn(ProductoController.class).obtenerUno(nuevo.getId())).withSelfRel(),

linkTo(methodOn(ProductoController.class).listar()).withRel("todos-los-productos"))
;
        return ResponseEntity.status(HttpStatus.CREATED).body(recurso);
    }

    @Operation(summary = "Buscar un producto por ID", description = "Retorna un
producto específico con sus enlaces de navegación")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Producto encontrado con
éxito"),
        @ApiResponse(responseCode = "404", description = "El ID del producto
solicitado no existe")
    })
    @GetMapping("/{id}")
```

```
    public EntityModel<Producto> obtenerUno(@PathVariable @NonNull Long id) {
        Producto producto = productoService.buscarPorId(id)
                .orElseThrow(() -> new RuntimeException("Producto no encontrado con
el ID: " + id));

        return EntityModel.of(producto,

linkTo(methodOn(ProductoController.class).obtenerUno(id)).withSelfRel(),

linkTo(methodOn(ProductoController.class).listar()).withRel("todos-los-productos"),
                linkTo(methodOn(ProductoController.class).actualizar(id, new
Producto())).withRel("actualizar-producto"));
    }

    @Operation(summary = "Actualizar un producto existente", description = "Modifica
los datos de un producto en el catálogo. Acción exclusiva de ADMINISTRADORES.")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Producto actualizado
correctamente"),
        @ApiResponse(responseCode = "404", description = "El producto a actualizar
no existe")
    })
    @PutMapping("/{id}")
    public ResponseEntity<EntityModel<Producto>> actualizar(@PathVariable Long id,
@Valid @RequestBody Producto productoDatos) {
        Producto productoActualizado = productoService.buscarPorId(id)
                .map(productoExistente -> {
                    productoExistente.setNombre(productoDatos.getNombre());
                    productoExistente.setPrecio(productoDatos.getPrecio());
                    productoExistente.setStock(productoDatos.getStock());
                    return productoService.guardar(productoExistente);
                })
                .orElseThrow(() -> new RuntimeException("Producto no encontrado con
el ID: " + id));

        EntityModel<Producto> recurso = EntityModel.of(productoActualizado,

linkTo(methodOn(ProductoController.class).obtenerUno(id)).withSelfRel(),

linkTo(methodOn(ProductoController.class).listar()).withRel("todos-los-productos"))
;

        return ResponseEntity.ok(recurso);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **exception;** CLASE -> **GlobalExceptionHandler**

```
package exception;

import com.lartduniss.productos.dto.ErrorResponse;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import java.util.stream.Collectors;

@RestControllerAdvice
public class GlobalExceptionHandler {

   @ExceptionHandler(RuntimeException.class)
   public ResponseEntity<ErrorResponse> manejarRuntimeException(RuntimeException
ex) {
       ErrorResponse error = new ErrorResponse(
               HttpStatus.NOT_FOUND.value(),
               "Recurso No Encontrado",
               ex.getMessage()
       );
       return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
   }

   @ExceptionHandler(MethodArgumentNotValidException.class)
   public ResponseEntity<ErrorResponse>
manejarErroresValidacion(MethodArgumentNotValidException ex) {
       String detalles = ex.getBindingResult().getFieldErrors().stream()
               .map(FieldError::getDefaultMessage)
               .collect(Collectors.joining(", "));

       ErrorResponse error = new ErrorResponse(
               HttpStatus.BAD_REQUEST.value(),
```

```
                "Error de Validación",
                detalles
        );
        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> manejarErroresGlobales(Exception ex) {
        ErrorResponse error = new ErrorResponse(
                HttpStatus.INTERNAL_SERVER_ERROR.value(),
                "Error Interno del Servidor",
                "Ocurrió un error inesperado en el sistema: " + ex.getMessage()
        );
        return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **model;** CLASE -> **Producto**

```
package model;

import jakarta.persistence.Entity;
import jakarta.persistence.Table;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.Data;

@Entity
@Table(name = "productos")
@Data
public class Producto {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "El nombre es obligatorio")
    private String nombre;

    @NotNull(message = "El precio no puede ser nulo")
    @Min(value = 0, message = "El precio debe ser mayor o igual a 0")
```

```
    private Double precio;

    @NotNull(message = "El stock no puede ser nulo")
    @Min(value = 0, message = "El stock no puede ser negativo")
    private Integer stock;
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **repository;** INTERFAZ -> **ProductoRepository**

```
package repository;

import model.Producto;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ProductoRepository extends JpaRepository<Producto, Long> {
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **service;** CLASE -> **ProductoService**

```
package service;

import org.springframework.lang.NonNull;
import java.util.List;
import java.util.Optional;
import org.springframework.stereotype.Service;
import model.Producto;
import repository.ProductoRepository;

@Service
public class ProductoService {

    private final ProductoRepository productoRepository;

    public ProductoService(ProductoRepository productoRepository) {
        this.productoRepository = productoRepository;
    }

    public List<Producto> obtenerTodos() {
        return productoRepository.findAll();
    }
```

```
    public Producto guardar(@NonNull Producto producto) {
        return productoRepository.save(producto);
    }

    public Optional<Producto> buscarPorId(@NonNull Long id) {
        return productoRepository.findById(id);
    }
}
```

RUTA -> SRC/MAIN/RESOURCES
APPLICATION.PROPERTIES

```
spring.application.name=service-productos
server.port=8092

spring.datasource.url=jdbc:mysql://localhost:3306/db_lart_productos?createDatabaseI
fNotExist=true
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# Estrategia para Gateway
server.forward-headers-strategy=framework
server.tomcat.remote-ip-header=x-forwarded-for
server.tomcat.protocol-header=x-forwarded-proto
server.tomcat.port-header=x-forwarded-port

# Mapeo unificado para Swagger OpenAPI
springdoc.api-docs.path=/api/v1/productos/v3/api-docs
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> **com.lartduniss.productos;**
CLASE -> **ProductosApplicationTests**

```
package com.lartduniss.productos;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertTrue;

class ProductosApplicationTests {
```

```
    @Test
    void contextLoads() {

        assertTrue(true);
    }
}
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> com.lartduniss.productos.service;
CLASE -> ProductosServiceTest

```
package service;

import model.Producto;
import repository.ProductoRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.Mockito;
import org.mockito.junit.jupiter.MockitoExtension;

import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.ArgumentMatchers.any;

@ExtendWith(MockitoExtension.class)
class ProductoServiceTest {

    @Mock
    private ProductoRepository productoRepository;

    @InjectMocks
    private ProductoService productoService;

    @Test
    void guardarProductoExitosoTest() {
        // 1. Arrange
        Producto productoInput = new Producto();
        productoInput.setNombre("Pastel de Chocolate");
        productoInput.setPrecio(25000.0);
        productoInput.setStock(10);

        Producto productoGuardado = new Producto();
```

```
        productoGuardado.setId(1L);
        productoGuardado.setNombre("Pastel de Chocolate");
        productoGuardado.setPrecio(25000.0);
        productoGuardado.setStock(10);


Mockito.when(productoRepository.save(any(Producto.class))).thenReturn(productoGuard
ado);

        // 2. Act
        Producto resultado = productoService.guardar(productoInput);

        // 3. Assert (Patrón AAA)
        assertNotNull(resultado);
        assertEquals(1L, resultado.getId());
        assertEquals("Pastel de Chocolate", resultado.getNombre());
        Mockito.verify(productoRepository, Mockito.times(1)).save(productoInput);
    }
}
```

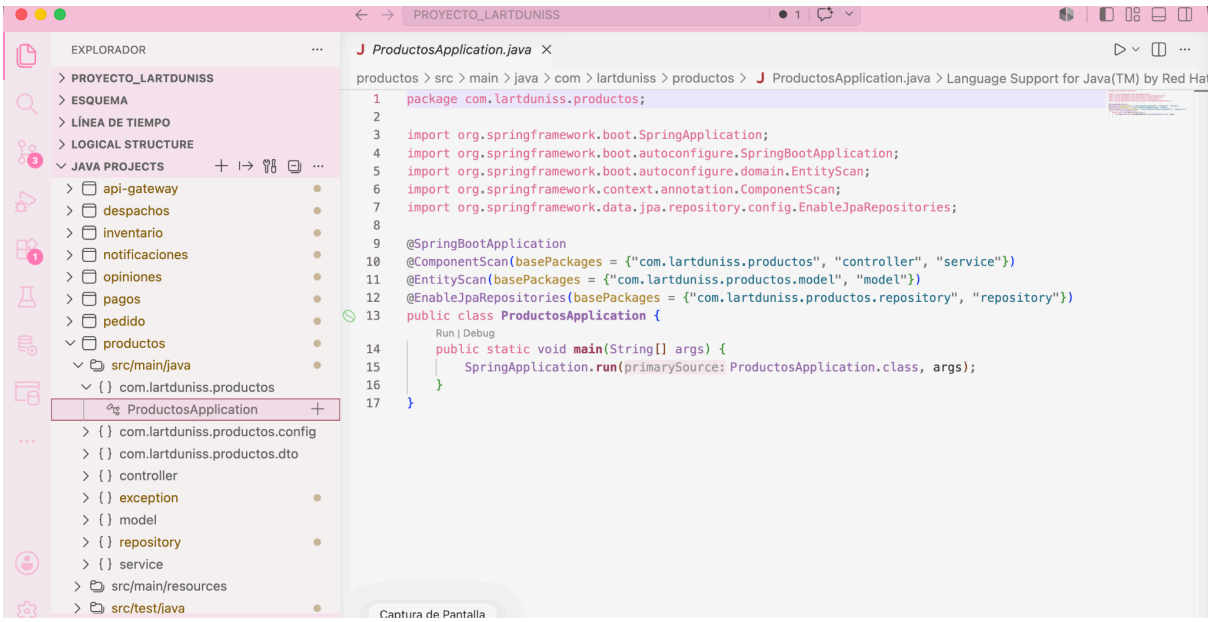
POM.XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>
   <parent>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-parent</artifactId>
       <version>3.5.15</version>
       <relativePath/> </parent>
   <groupId>com.lartduniss</groupId>
   <artifactId>productos</artifactId>
   <version>0.0.1-SNAPSHOT</version>
   <name>productos</name>
   <description/>
   <url/>
   <licenses>
       <license/>
   </licenses>
   <developers>
       <developer/>
   </developers>
```

```
<scm>
    <connection/>
    <developerConnection/>
    <tag/>
    <url/>
</scm>
<properties>
    <java.version>21</java.version>
</properties>
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
```

```
            <version>2.8.17</version>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-hateoas</artifactId>
        </dependency>

    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <excludes>
                        <exclude>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </exclude>
                    </excludes>
                </configuration>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <executions>
                    <execution>
                        <id>default-compile</id>
                        <phase>compile</phase>
                        <goals>
                            <goal>compile</goal>
                        </goals>
                        <configuration>
                            <annotationProcessorPaths>
                                <path>
                                    <groupId>org.projectlombok</groupId>
                                    <artifactId>lombok</artifactId>
                                </path>
                            </annotationProcessorPaths>
                        </configuration>
                    </execution>
                    <execution>
                        <id>default-testCompile</id>
                        <phase>test-compile</phase>
```

```
                        <goals>
                            <goal>testCompile</goal>
                        </goals>
                        <configuration>
                            <annotationProcessorPaths>
                                <path>
                                    <groupId>org.projectlombok</groupId>
                                    <artifactId>lombok</artifactId>
                                </path>
                            </annotationProcessorPaths>
                        </configuration>
                    </execution>
                </executions>
            </plugin>
        </plugins>
    </build>

</project>
```



- 8. PROYECTO (CLIENTE):

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **controller;** CLASE -> **ClienteController**

```
package controller;

import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.linkTo;
import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.methodOn;

import java.util.List;
import java.util.stream.Collectors;

import org.springframework.hateoas.CollectionModel;
import org.springframework.hateoas.EntityModel;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.lang.NonNull;
import org.springframework.web.bind.annotation.*;

import model.Cliente;
import service.ClienteService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;

@RestController
@RequestMapping("/api/v1/clientes")
@Tag(name = "Clientes", description = "Controlador para la gestión y registro de
perfiles de clientes")
@SuppressWarnings("null")
public class ClienteController {

   private final ClienteService clienteService;

   public ClienteController(ClienteService clienteService) {
       this.clienteService = clienteService;
   }

   @Operation(summary = "Listar todos los clientes", description = "Retorna el
listado de clientes registrados con soporte hipermedia HATEOAS")
   @GetMapping
   public CollectionModel<EntityModel<Cliente>> listar() {
       List<EntityModel<Cliente>> clientes = clienteService.obtenerTodos().stream()
```

```
                .map(cliente -> EntityModel.of(cliente,

linkTo(methodOn(ClienteController.class).obtenerUno(cliente.getId())).withSelfRel()
,

linkTo(methodOn(ClienteController.class).listar()).withRel("clientes")))
                .collect(Collectors.toList());

        return CollectionModel.of(clientes,
                linkTo(methodOn(ClienteController.class).listar()).withSelfRel());
    }

    @Operation(summary = "Registrar un nuevo cliente", description = "Crea un
registro de cliente validando que el email sea único y posea formato correcto")
    @PostMapping
    public ResponseEntity<EntityModel<Cliente>> crear(@Valid @RequestBody Cliente
cliente) {
        Cliente nuevo = clienteService.guardarCliente(cliente);
        EntityModel<Cliente> recurso = EntityModel.of(nuevo,

linkTo(methodOn(ClienteController.class).obtenerUno(nuevo.getId())).withSelfRel(),

linkTo(methodOn(ClienteController.class).listar()).withRel("todos-los-clientes"));
        return ResponseEntity.status(HttpStatus.CREATED).body(recurso);
    }

    @Operation(summary = "Obtener un cliente por ID", description = "Busca un
cliente específico en el sistema según su identificador único")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Cliente localizado con
éxito"),
        @ApiResponse(responseCode = "404", description = "El ID del cliente no
existe en los registros")
    })
    @GetMapping("/{id}")
    public EntityModel<Cliente> obtenerUno(@PathVariable @NonNull Long id) {
        Cliente cliente = clienteService.buscarPorId(id)
                .orElseThrow(() -> new RuntimeException("Cliente no encontrado con
el ID: " + id));

        return EntityModel.of(cliente,

linkTo(methodOn(ClienteController.class).obtenerUno(id)).withSelfRel(),

linkTo(methodOn(ClienteController.class).listar()).withRel("todos-los-clientes"),
```

```
                linkTo(methodOn(ClienteController.class).actualizar(id, new
Cliente())).withRel("actualizar-cliente"));
    }

    @Operation(summary = "Actualizar datos de un cliente", description = "Permite
modificar el nombre, email o teléfono de un cliente existente")
    @PutMapping("/{id}")
    public ResponseEntity<EntityModel<Cliente>> actualizar(@PathVariable Long id,
@Valid @RequestBody Cliente clienteDatos) {
        Cliente clienteActualizado = clienteService.buscarPorId(id)
                .map(clienteExistente -> {
                    clienteExistente.setNombre(clienteDatos.getNombre());
                    clienteExistente.setEmail(clienteDatos.getEmail());
                    clienteExistente.setTelefono(clienteDatos.getTelefono());
                    return clienteService.guardarCliente(clienteExistente);
                })
                .orElseThrow(() -> new RuntimeException("Cliente no encontrado con
el ID: " + id));

        EntityModel<Cliente> recurso = EntityModel.of(clienteActualizado,

linkTo(methodOn(ClienteController.class).obtenerUno(id)).withSelfRel(),

linkTo(methodOn(ClienteController.class).listar()).withRel("todos-los-clientes"));

        return ResponseEntity.ok(recurso);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **lartduniss.proyecto**; CLASE -> **ProyectoApplication**

```
package lartduniss.proyecto;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.autoconfigure.domain.EntityScan;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;

@SpringBootApplication
@ComponentScan(basePackages = {"lartduniss.proyecto", "controller", "service"})
@EntityScan(basePackages = {"model"})
@EnableJpaRepositories(basePackages = {"repository"})
public class ProyectoApplication {
```

```
    public static void main(String[] args) {
        SpringApplication.run(ProyectoApplication.class, args);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **lartduniss.proyecto.config**; CLASE -> **CustomAccessDeniedHandler**

```
package lartduniss.proyecto.config;

import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.security.web.access.AccessDeniedHandler;
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;

public class CustomAccessDeniedHandler implements AccessDeniedHandler {

    @Override
    public void handle(HttpServletRequest request, HttpServletResponse response,
                       org.springframework.security.access.AccessDeniedException
accessDeniedException)
            throws IOException, ServletException {

        response.setStatus(HttpStatus.FORBIDDEN.value());
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);

        Map<String, Object> data = new HashMap<>();
        data.put("statusCode", HttpStatus.FORBIDDEN.value());
        data.put("message", "Acceso denegado: No posees los privilegios necesarios
para realizar esta acción.");
        data.put("timestamp", System.currentTimeMillis());

        ObjectMapper objectMapper = new ObjectMapper();
        response.getOutputStream().println(objectMapper.writeValueAsString(data));
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **lartduniss.proyecto.config**; CLASE -> **OpenApiConfig**

```
package lartduniss.proyecto.config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.servers.Server;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import java.util.List;

@Configuration
public class OpenApiConfig {

   @Bean
   public OpenAPI customOpenAPI() {
       return new OpenAPI()
               .info(new Info()
                       .title("API Lartduniss - Gestión de Clientes")
                       .version("1.0")
                       .description("Servicio encargado de la administración de
usuarios y perfiles de compradores"))
               .servers(List.of(
                       new Server().url("http://localhost:9090").description("API
Gateway (Entrada Unificada)")
               ));
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **lartduniss.proyecto.config**; CLASE -> **RequestHeaderAuthenticationFilter**

```
package lartduniss.proyecto.config;

import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.filter.OncePerRequestFilter;
import org.springframework.lang.NonNull;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
```

```java
import java.io.IOException;
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class RequestHeaderAuthenticationFilter extends OncePerRequestFilter {

   @Override
   protected void doFilterInternal(@NonNull HttpServletRequest request, @NonNull
HttpServletResponse response, @NonNull FilterChain filterChain)
           throws ServletException, IOException {

       String username = request.getHeader("X-Username");
       if (username == null) {
           username = request.getHeader("X-User-Username");
       }
       String rolesStr = request.getHeader("X-User-Roles");

       if (username != null && rolesStr != null && !rolesStr.trim().isEmpty()) {
           List<SimpleGrantedAuthority> authorities =
Arrays.stream(rolesStr.split(","))
                   .flatMap(rol -> {
                       String normalized = rol.trim().toUpperCase();
                       return java.util.stream.Stream.of(
                               new SimpleGrantedAuthority(normalized),
                               new SimpleGrantedAuthority("ROLE_" + normalized)
                       );
                   })
                   .collect(Collectors.toList());

           UsernamePasswordAuthenticationToken authentication =
                   new UsernamePasswordAuthenticationToken(username, null,
authorities);

           SecurityContextHolder.getContext().setAuthentication(authentication);
       }
       filterChain.doFilter(request, response);
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **lartduniss.proyecto.config**; CLASE -> **SecurityConfig**

```java
package lartduniss.proyecto.config;
```

```java
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.access.AccessDeniedHandler;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilte
r;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

   @Bean
   public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
       http
           .csrf(csrf -> csrf.disable())
           .authorizeHttpRequests(auth -> auth
               .requestMatchers("/api/v1/clientes/v3/api-docs/**",
"/swagger-ui/**", "/swagger-ui.html", "/webjars/**", "/error").permitAll()
               .requestMatchers(HttpMethod.POST, "/api/v1/clientes/**").permitAll()
// Registro libre
               .requestMatchers(HttpMethod.GET,
"/api/v1/clientes/**").hasAnyRole("CLIENTE", "ADMINISTRADOR", "ADMIN")
               .requestMatchers(HttpMethod.PUT,
"/api/v1/clientes/**").hasAnyRole("CLIENTE", "ADMINISTRADOR", "ADMIN")
               .anyRequest().authenticated()
           )
           .exceptionHandling(exception -> exception
               .accessDeniedHandler(accessDeniedHandler())
           )
           .addFilterBefore(new RequestHeaderAuthenticationFilter(),
UsernamePasswordAuthenticationFilter.class);

       return http.build();
   }

   @Bean
   public AccessDeniedHandler accessDeniedHandler() {
       return new CustomAccessDeniedHandler();
   }
```

```
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **lartduniss.proyecto.config**; CLASE -> **WebConfig**

```
package lartduniss.proyecto.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.lang.NonNull;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(@NonNull CorsRegistry registry) {
        registry.addMapping("/**")
                .allowedOrigins("http://localhost:8080", "http://localhost:9090")
                .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
                .allowedHeaders("*")
                .allowCredentials(true);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **lartduniss.proyecto.dto**; CLASE -> **ErrorResponse**

```
package lartduniss.proyecto.dto;

import java.time.LocalDateTime;

public class ErrorResponse {
    private int status;
    private String error;
    private String mensaje;
    private LocalDateTime timestamp;

    public ErrorResponse(int status, String error, String mensaje) {
        this.status = status;
        this.error = error;
        this.mensaje = mensaje;
```

```
        this.timestamp = LocalDateTime.now();
    }

    // Getters y Setters
    public int getStatus() { return status; }
    public void setStatus(int status) { this.status = status; }

    public String getError() { return error; }
    public void setError(String error) { this.error = error; }

    public String getMensaje() { return mensaje; }
    public void setMensaje(String mensaje) { this.mensaje = mensaje; }

    public LocalDateTime getTimestamp() { return timestamp; }
    public void setTimestamp(LocalDateTime timestamp) { this.timestamp = timestamp;
}
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **lartduniss.proyecto.exception**; CLASE -> **GlobalExceptionHandler**

```
package lartduniss.proyecto.exception;

import lartduniss.proyecto.dto.ErrorResponse;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import java.util.stream.Collectors;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RuntimeException.class)
    public ResponseEntity<ErrorResponse> manejarRuntimeException(RuntimeException
ex) {
        ErrorResponse error = new ErrorResponse(
                HttpStatus.NOT_FOUND.value(),
                "Cliente No Encontrado",
                ex.getMessage()
        );
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }
```

```
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse>
manejarErroresValidacion(MethodArgumentNotValidException ex) {
        String detalles = ex.getBindingResult().getFieldErrors().stream()
                .map(FieldError::getDefaultMessage)
                .collect(Collectors.joining(", "));

        ErrorResponse error = new ErrorResponse(
                HttpStatus.BAD_REQUEST.value(),
                "Error de Validación",
                detalles
        );
        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> manejarErroresGlobales(Exception ex) {
        ErrorResponse error = new ErrorResponse(
                HttpStatus.INTERNAL_SERVER_ERROR.value(),
                "Error Interno",
                "Ocurrió un error inesperado en el sistema: " + ex.getMessage()
        );
        return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **model**; CLASE -> **Cliente**

```
package model;

import jakarta.persistence.Entity;
import jakarta.persistence.Table;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "clientes")
```

```java
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Cliente {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @NotBlank(message = "El nombre es obligatorio")
   private String nombre;

   @NotBlank(message = "El email es obligatorio")
   @Email(message = "El formato del email no es válido")
   private String email;

   private String telefono;
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **repository**; INTERFAZ -> **ClienteRepository**

```java
package repository;

import model.Cliente;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ClienteRepository extends JpaRepository<Cliente, Long> {
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **service**; CLASE -> **ClienteService**

```java
package service;

import org.springframework.lang.NonNull;
import model.Cliente;
import repository.ClienteRepository;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;
```

```java
@Service
public class ClienteService {

    private final ClienteRepository clienteRepository;

    public ClienteService(ClienteRepository clienteRepository) {
        this.clienteRepository = clienteRepository;
    }

    public List<Cliente> obtenerTodos() {
        return clienteRepository.findAll();
    }

    public Cliente guardarCliente(@NonNull Cliente cliente) {
        return clienteRepository.save(cliente);
    }

    public Optional<Cliente> buscarPorId(@NonNull Long id) {
        return clienteRepository.findById(id);
    }
}
```

RUTA -> SRC/MAIN/RESOURCES
APPLICATION.PROPERTIES

```properties
spring.application.name=proyecto
server.port=8091
spring.profiles.active=local

# Base de datos local de desarrollo (H2 en memoria)
spring.datasource.url=jdbc:h2:mem:db_lart_clientes;DB_CLOSE_DELAY=-1;MODE=MySQL
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Estrategia para Gateway <3
server.forward-headers-strategy=framework
server.tomcat.remote-ip-header=x-forwarded-for
server.tomcat.protocol-header=x-forwarded-proto
server.tomcat.port-header=x-forwarded-port
```

```
# Mapeo unificado para Swagger OpenAPI <3
springdoc.api-docs.path=/api/v1/clientes/v3/api-docs
springdoc.swagger-ui.path=/swagger-ui.html
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> **lartduniss.proyecto**; CLASE ->**ProyectoApplicationTests**

```
package lartduniss.proyecto;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Configuration;

@SpringBootTest(classes = ProyectoApplicationTests.TestConfig.class)
class ProyectoApplicationTests {

    @Configuration
    static class TestConfig {

    }

    @Test
    void contextLoads() {

    }
}
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> **service**; CLASE -> **ClienteServiceTest**

```
package service;

import model.Cliente;
import repository.ClienteRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.Mockito;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;

import static org.junit.jupiter.api.Assertions.assertNotNull;
```

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.ArgumentMatchers.any;

@ExtendWith(MockitoExtension.class)
class ClienteServiceTest {

   @Mock
   private ClienteRepository clienteRepository;

   @InjectMocks
   private ClienteService clienteService;

   @Test
   void guardarClienteExitosoTest() {
       // 1. Arrange
       Cliente clienteInput = new Cliente(null, "Denisse Lazcano",
"denisse@lartduniss.com", "+56912345678");
       Cliente clienteGuardado = new Cliente(1L, "Denisse Lazcano",
"denisse@lartduniss.com", "+56912345678");


Mockito.when(clienteRepository.save(any(Cliente.class))).thenReturn(clienteGuardado
);

       // 2. Act
       Cliente resultado = clienteService.guardarCliente(clienteInput);

       // 3. Assert
       assertNotNull(resultado);
       assertEquals(1L, resultado.getId());
       assertEquals("denisse@lartduniss.com", resultado.getEmail());
       Mockito.verify(clienteRepository, Mockito.times(1)).save(clienteInput);
   }

   @Test
   void buscarClientePorIdTest() {
       // 1. Arrange
       Long clienteId = 1L;
       Cliente clienteMock = new Cliente(clienteId, "Denisse Lazcano",
"denisse@lartduniss.com", "+56912345678");

Mockito.when(clienteRepository.findById(clienteId)).thenReturn(Optional.of(clienteM
ock));

       // 2. Act
```

```
        Optional<Cliente> resultado = clienteService.buscarPorId(clienteId);

        // 3. Assert
        assertEquals(true, resultado.isPresent());
        assertEquals("Denisse Lazcano", resultado.get().getNombre());
        Mockito.verify(clienteRepository, Mockito.times(1)).findById(clienteId);
    }
}
```

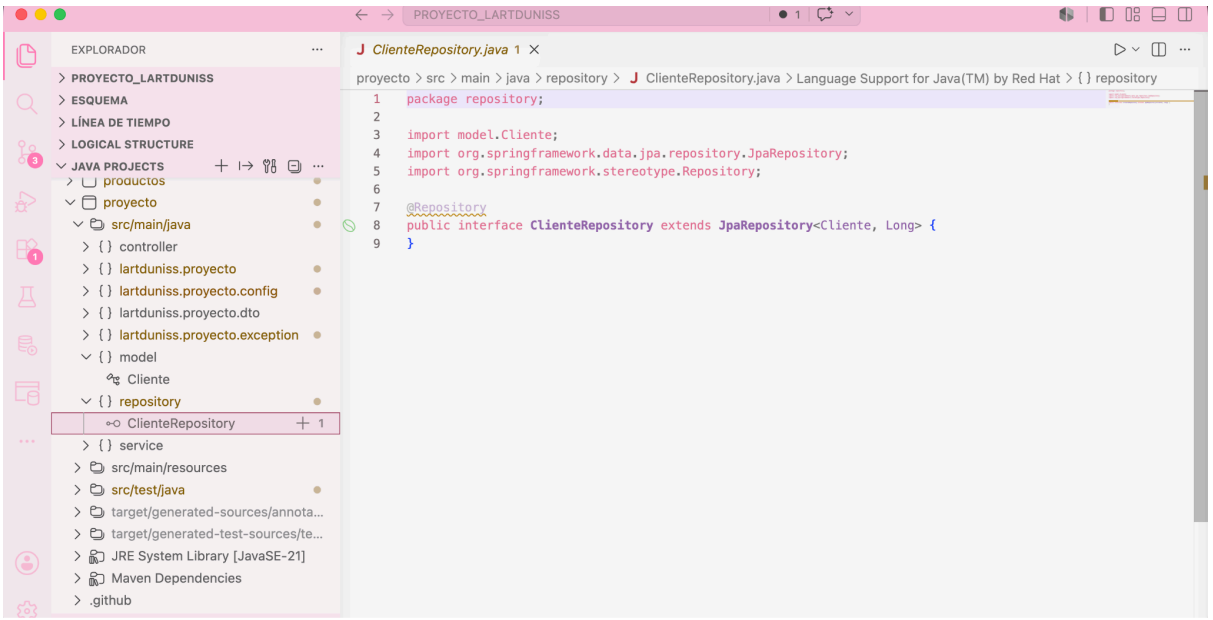
POM.XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>
   <parent>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-parent</artifactId>
       <version>3.5.15</version>
       <relativePath/> </parent>
   <groupId>lartduniss</groupId>
   <artifactId>proyecto</artifactId>
   <version>0.0.1-SNAPSHOT</version>
   <name>proyecto</name>
   <description>Proyecto L'art du niss</description>
   <url/>
   <licenses>
       <license/>
   </licenses>
   <developers>
       <developer/>
   </developers>
   <scm>
       <connection/>
       <developerConnection/>
       <tag/>
       <url/>
   </scm>
   <properties>
       <java.version>21</java.version>
   </properties>
   <dependencies>
       <dependency>
```

```
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
                    <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <scope>provided</scope>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>

        <dependency>
            <groupId>org.springdoc</groupId>
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
            <version>2.8.17</version>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-hateoas</artifactId>
```

```
    </dependency>

</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <excludes>
                    <exclude>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </exclude>
                </excludes>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <executions>
                <execution>
                    <id>default-compile</id>
                    <phase>compile</phase>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok</artifactId>
                            </path>
                        </annotationProcessorPaths>
                    </configuration>
                </execution>
                <execution>
                    <id>default-testCompile</id>
                    <phase>test-compile</phase>
                    <goals>
                        <goal>testCompile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
```

```
                                <path>
                                    <groupId>org.projectlombok</groupId>
                                    <artifactId>lombok</artifactId>
                                </path>
                            </annotationProcessorPaths>
                        </configuration>
                    </execution>
                </executions>
            </plugin>
        </plugins>
    </build>

</project>
```



- 9. REPORTES:

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **controller;** CLASE -> **ReportesController**

```
package controller;

import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.linkTo;
import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.methodOn;

import java.util.List;
import java.util.stream.Collectors;
```

```java
import org.springframework.hateoas.CollectionModel;
import org.springframework.hateoas.EntityModel;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestBody;

import model.Reportes;
import service.ReportesService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;

@RestController
@RequestMapping("/api/v1/reportes")
@Tag(name = "Reportes", description = "Controlador para la generación y auditoría de reportes financieros")
@SuppressWarnings("null")
public class ReportesController {

   private final ReportesService reportesService;

   public ReportesController(ReportesService reportesService) {
       this.reportesService = reportesService;
   }

   @Operation(summary = "Obtener todos los reportes", description = "Retorna el histórico completo de balances consolidados. Exclusivo de ADMINISTRADORES.")
   @GetMapping
   public CollectionModel<EntityModel<Reportes>> listar() {
       List<EntityModel<Reportes>> reportes = reportesService.obtenerTodos().stream()
               .map(reporte -> EntityModel.of(reporte,

linkTo(methodOn(ReportesController.class).obtenerUno(reporte.getId())).withSelfRel(),
```

```
linkTo(methodOn(ReportesController.class).listar()).withRel("reportes")))
                .collect(Collectors.toList());

        return CollectionModel.of(reportes,
                linkTo(methodOn(ReportesController.class).listar()).withSelfRel());
    }

    @Operation(summary = "Crear un nuevo reporte", description = "Registra un nuevo
balance financiero en el sistema")
    @PostMapping
    public ResponseEntity<EntityModel<Reportes>> crear(@Valid @RequestBody Reportes
reporte) {
        Reportes nuevo = reportesService.guardarReporte(reporte);
        EntityModel<Reportes> recurso = EntityModel.of(nuevo,

linkTo(methodOn(ReportesController.class).obtenerUno(nuevo.getId())).withSelfRel(),

linkTo(methodOn(ReportesController.class).listar()).withRel("todos-los-reportes"));
        return ResponseEntity.status(HttpStatus.CREATED).body(recurso);
    }

    @Operation(summary = "Buscar reporte por ID", description = "Obtiene los
detalles de auditoría de un reporte específico")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Reporte localizado
correctamente"),
        @ApiResponse(responseCode = "404", description = "El ID del reporte no
existe")
    })
    @GetMapping("/{id}")
    public EntityModel<Reportes> obtenerUno(@PathVariable Long id) {
        Reportes reporte = reportesService.buscarPorId(id)
                .orElseThrow(() -> new RuntimeException("Reporte no encontrado con
el ID: " + id));

        return EntityModel.of(reporte,

linkTo(methodOn(ReportesController.class).obtenerUno(id)).withSelfRel(),

linkTo(methodOn(ReportesController.class).listar()).withRel("todos-los-reportes"));
    }

    @Operation(summary = "Modificar un reporte existente", description = "Permite
corregir títulos, tipos o montos calculados de un reporte")
```

```
    @PutMapping("/{id}")
    public ResponseEntity<EntityModel<Reportes>> actualizar(@PathVariable Long id,
@Valid @RequestBody Reportes reporte) {
        Reportes actualizado = reportesService.actualizarReporte(id, reporte)
                .orElseThrow(() -> new RuntimeException("No se pudo actualizar.
Reporte no encontrado con el ID: " + id));

        EntityModel<Reportes> recurso = EntityModel.of(actualizado,

linkTo(methodOn(ReportesController.class).obtenerUno(id)).withSelfRel(),

linkTo(methodOn(ReportesController.class).listar()).withRel("todos-los-reportes"));

        return ResponseEntity.ok(recurso);
    }

    @Operation(summary = "Eliminar un reporte del sistema", description = "Borra
físicamente un reporte de la base de datos")
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminar(@PathVariable Long id) {
        if (!reportesService.eliminarReporte(id)) {
            throw new RuntimeException("No se encontró el reporte a eliminar con el
ID: " + id);
        }
        return ResponseEntity.noContent().build();
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **lartduniss.proyecto.reportes**; CLASE -> **ReportesApplication**

```
package lartduniss.proyecto.reportes;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.autoconfigure.domain.EntityScan;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;

@SpringBootApplication
@ComponentScan(basePackages = {
  "lartduniss.proyecto.reportes",
  "lartduniss.proyecto.reportes.config",
  "controller",
```

```
  "service"
})
@EntityScan(basePackages = {"model"})
@EnableJpaRepositories(basePackages = {"repository"})
public class ReportesApplication {
  public static void main(String[] args) {
      SpringApplication.run(ReportesApplication.class, args);
  }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **lartduniss.proyecto.reportes.config;**
CLASE -> **CustomAccessDeniedHandler**

```
package lartduniss.proyecto.reportes.config;

import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.security.web.access.AccessDeniedHandler;
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;

public class CustomAccessDeniedHandler implements AccessDeniedHandler {

   @Override
   public void handle(HttpServletRequest request, HttpServletResponse response,
                      org.springframework.security.access.AccessDeniedException
accessDeniedException)
           throws IOException, ServletException {

       response.setStatus(HttpStatus.FORBIDDEN.value());
       response.setContentType(MediaType.APPLICATION_JSON_VALUE);

       Map<String, Object> data = new HashMap<>();
       data.put("status", HttpStatus.FORBIDDEN.value());
       data.put("error", "Forbidden");
       data.put("mensaje", "Acceso denegado: El módulo de Reportes Financieros es
confidencial y de uso exclusivo para ADMINISTRADORES.");

       ObjectMapper objectMapper = new ObjectMapper();
```

```
        response.getOutputStream().println(objectMapper.writeValueAsString(data));
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **lartduniss.proyecto.reportes.config**; CLASE -> **OpenApiConfig**

```
package lartduniss.proyecto.reportes.config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.servers.Server;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import java.util.List;

@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
                .info(new Info()
                        .title("API Lartduniss - Auditoría y Reportes")
                        .version("1.0")
                        .description("Servicio encargado del procesamiento de
balances generales del ecosistema"))
                .servers(List.of(
                        new
Server().url("http://localhost:8096").description("Puerto de Reportes")
                ));
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **lartduniss.proyecto.reportes.config**;
CLASE -> **RequestHeaderAuthenticationFilter**

```
package lartduniss.proyecto.reportes.config;

import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
```

```java
import org.springframework.web.filter.OncePerRequestFilter;
import org.springframework.lang.NonNull;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class RequestHeaderAuthenticationFilter extends OncePerRequestFilter {

   @Override
   protected void doFilterInternal(@NonNull HttpServletRequest request, @NonNull
HttpServletResponse response, @NonNull FilterChain filterChain)
           throws ServletException, IOException {

       String username = request.getHeader("X-Username");
       if (username == null) {
           username = request.getHeader("X-User-Username");
       }
       String rolesStr = request.getHeader("X-User-Roles");

       if (username != null && rolesStr != null && !rolesStr.trim().isEmpty()) {
           List<SimpleGrantedAuthority> authorities =
Arrays.stream(rolesStr.split(","))
                   .flatMap(rol -> {
                       String normalized = rol.trim().toUpperCase();
                       return java.util.stream.Stream.of(
                               new SimpleGrantedAuthority(normalized),
                               new SimpleGrantedAuthority("ROLE_" + normalized)
                       );
                   })
                   .collect(Collectors.toList());

           UsernamePasswordAuthenticationToken authentication =
                   new UsernamePasswordAuthenticationToken(username, null,
authorities);

           SecurityContextHolder.getContext().setAuthentication(authentication);
       }
       filterChain.doFilter(request, response);
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **lartduniss.proyecto.reportes.config**; CLASE -> **SecurityConfig**

```
package lartduniss.proyecto.reportes.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.access.AccessDeniedHandler;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilte
r;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

   @Bean
   public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
       http
           .csrf(csrf -> csrf.disable())
           .authorizeHttpRequests(auth -> auth
               .requestMatchers("/api/v1/reportes/v3/api-docs/**",
"/swagger-ui/**", "/swagger-ui.html", "/webjars/**", "/error").permitAll()
               .requestMatchers("/api/v1/reportes/**").hasAnyRole("ADMINISTRADOR",
"ADMIN")
               .anyRequest().authenticated()
           )
           .exceptionHandling(exception -> exception
               .accessDeniedHandler(accessDeniedHandler())
           )
           .addFilterBefore(new RequestHeaderAuthenticationFilter(),
UsernamePasswordAuthenticationFilter.class);

       return http.build();
   }

   @Bean
   public AccessDeniedHandler accessDeniedHandler() {
       return new CustomAccessDeniedHandler();
```

```
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **lartduniss.proyecto.reportes.config**; CLASE -> **WebConfig**

```
package lartduniss.proyecto.reportes.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.lang.NonNull;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(@NonNull CorsRegistry registry) {
        registry.addMapping("/**")
                .allowedOrigins("http://localhost:8080")
                .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
                .allowedHeaders("*")
                .allowCredentials(true);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **lartduniss.proyecto.reportes.dto**; CLASE -> **ErrorResponse**

```
package lartduniss.proyecto.reportes.dto;

import java.time.LocalDateTime;

public class ErrorResponse {
    private int status;
    private String error;
    private String mensaje;
    private LocalDateTime timestamp;

    public ErrorResponse(int status, String error, String mensaje) {
        this.status = status;
        this.error = error;
        this.mensaje = mensaje;
        this.timestamp = LocalDateTime.now();
```

```
    }

    // Getters y Setters
    public int getStatus() { return status; }
    public void setStatus(int status) { this.status = status; }

    public String getError() { return error; }
    public void setError(String error) { this.error = error; }

    public String getMensaje() { return mensaje; }
    public void setMensaje(String mensaje) { this.mensaje = mensaje; }

    public LocalDateTime getTimestamp() { return timestamp; }
    public void setTimestamp(LocalDateTime timestamp) { this.timestamp = timestamp;
}
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **lartduniss.proyecto.reportes.exception;**
CLASE -> **GlobalExceptionHandler**

```
package lartduniss.proyecto.reportes.exception;

import lartduniss.proyecto.reportes.dto.ErrorResponse;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import java.util.stream.Collectors;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RuntimeException.class)
    public ResponseEntity<ErrorResponse> manejarRuntimeException(RuntimeException
ex) {
        ErrorResponse error = new ErrorResponse(
                HttpStatus.NOT_FOUND.value(),
                "Recurso No Encontrado",
                ex.getMessage()
        );
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }
```

```
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse>
manejarErroresValidacion(MethodArgumentNotValidException ex) {
        String detalles = ex.getBindingResult().getFieldErrors().stream()
                .map(FieldError::getDefaultMessage)
                .collect(Collectors.joining(", "));

        ErrorResponse error = new ErrorResponse(
                HttpStatus.BAD_REQUEST.value(),
                "Error de Validación",
                detalles
        );
        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> manejarErroresGlobales(Exception ex) {
        ErrorResponse error = new ErrorResponse(
                HttpStatus.INTERNAL_SERVER_ERROR.value(),
                "Error Interno",
                "Fallo inesperado en el sistema de reportes: " + ex.getMessage()
        );
        return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **model**; CLASE -> **Reportes**

```
package model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import java.time.LocalDate;
import lombok.Data;
import lombok.AllArgsConstructor;
import lombok.NoArgsConstructor;

@Entity
```

```java
@Table(name = "reportes")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Reportes {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @NotBlank(message = "El titulo del reporte es obligatorio")
   private String titulo;

   @NotBlank(message = "El tipo de reporte no puede estar vacio")
   private String tipo;

   @NotNull(message = "La fecha de generacion del reporte es obligatoria")
   private LocalDate fechaGeneracion;

   @NotNull(message = "El monto total calculado es obligatorio")
   private Double montoTotalCalculado;
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **repository**; INTERFAZ -> **ReportesRepository**

```java
package repository;

import model.Reportes;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ReportesRepository extends JpaRepository<Reportes, Long> {
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **service**; CLASE -> **ReportesService**

```java
package service;

import org.springframework.lang.NonNull;
import model.Reportes;
import repository.ReportesRepository;
```

```java
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;

@Service
public class ReportesService {

    private final ReportesRepository reportesRepository;

    public ReportesService(ReportesRepository reportesRepository) {
        this.reportesRepository = reportesRepository;
    }

    public List<Reportes> obtenerTodos() {
        return reportesRepository.findAll();
    }

    public Reportes guardarReporte(@NonNull Reportes reporte) {
        return reportesRepository.save(reporte);
    }

    public Optional<Reportes> buscarPorId(@NonNull Long id) {
        return reportesRepository.findById(id);
    }

    public Optional<Reportes> actualizarReporte(@NonNull Long id, Reportes
reporteActualizado) {
        return reportesRepository.findById(id).map((@NonNull Reportes
reporteExistente) -> {
            reporteExistente.setTitulo(reporteActualizado.getTitulo());
            reporteExistente.setTipo(reporteActualizado.getTipo());

reporteExistente.setFechaGeneracion(reporteActualizado.getFechaGeneracion());

reporteExistente.setMontoTotalCalculado(reporteActualizado.getMontoTotalCalculado()
);
            return reportesRepository.save(reporteExistente);
        });
    }

    public boolean eliminarReporte(@NonNull Long id) {
        return reportesRepository.findById(id).map((@NonNull Reportes reporte) -> {
            reportesRepository.delete(reporte);
            return true;
        }).orElse(false);
```

```
    }
}
```

RUTA -> SRC/MAIN/RESOURCES
APPLICATION.PROPERTIES

```
spring.application.name=servicio-reportes
server.port=8096
spring.profiles.active=local

# Base de datos local de desarrollo (H2 en memoria)
spring.datasource.url=jdbc:h2:mem:db_lart_reportes;DB_CLOSE_DELAY=-1;MODE=MySQL
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Ajuste Swagger para Reportes
springdoc.api-docs.path=/api/v1/reportes/v3/api-docs
springdoc.swagger-ui.path=/swagger-ui.html
server.forward-headers-strategy=framework
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> **lartduniss.proyecto.reportes**; CLASE -> **ReportesApplicationTests**

```
package lartduniss.proyecto.reportes;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Configuration;

@SpringBootTest(classes = ReportesApplicationTests.MockConfig.class)
class ReportesApplicationTests {

    @Configuration
    static class MockConfig {

    }

    @Test
    void contextLoads() {
```

```
    }
}
```

RUTA -> SRC/TEST/JAVA
PACKAGE -> **service**; CLASE -> **ReportesServiceTests**

```
package service;

import model.Reportes;
import repository.ReportesRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.Mockito;
import org.mockito.junit.jupiter.MockitoExtension;

import java.time.LocalDate;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.assertTrue;
import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.times;

@ExtendWith(MockitoExtension.class)
class ReportesServiceTest {

    @Mock
    private ReportesRepository reportesRepository;

    @InjectMocks
    private ReportesService reportesService;

    @Test
    void eliminarReporteExitosoTest() {
        // 1. Arrange <3
        Long reporteId = 1L;
        Reportes reporteMock = new Reportes(reporteId, "Ventas Mayo", "MENSUAL",
LocalDate.now(), 540000.0);

Mockito.when(reportesRepository.findById(reporteId)).thenReturn(Optional.of(reporte
Mock));

        // 2. Act <3
```

```
        boolean resultado = reportesService.eliminarReporte(reporteId);

        // 3. Assert <3
        assertTrue(resultado);
        verify(reportesRepository, times(1)).findById(reporteId);
        verify(reportesRepository, times(1)).delete(reporteMock);
    }

    @Test
    void eliminarReporteInexistenteTest() {
        // 1. Arrange
        Long reporteId = 99L;

Mockito.when(reportesRepository.findById(reporteId)).thenReturn(Optional.empty());

        // 2. Act
        boolean resultado = reportesService.eliminarReporte(reporteId);

        // 3. Assert
        assertFalse(resultado);
        verify(reportesRepository, times(1)).findById(reporteId);
    }
}
```

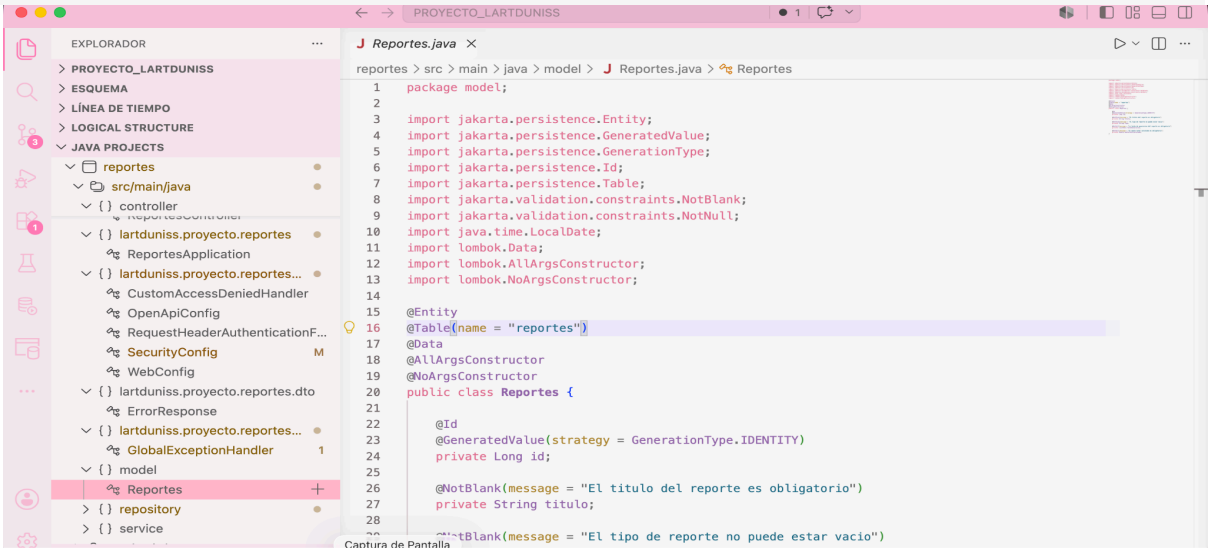
POM.XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>
   <parent>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-parent</artifactId>
       <version>3.5.15</version>
       <relativePath/> <!-- lookup parent from repository -->
   </parent>
   <groupId>lartduniss.proyecto</groupId>
   <artifactId>reportes</artifactId>
   <version>0.0.1-SNAPSHOT</version>
   <name>reportes</name>
   <description/>
   <url/>
   <licenses>
```

```
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>21</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
                    <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <scope>provided</scope>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
```

```
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>

        <dependency>
            <groupId>org.springdoc</groupId>
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
            <version>2.8.17</version>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-hateoas</artifactId>
        </dependency>

    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <excludes>
                        <exclude>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </exclude>
                    </excludes>
                </configuration>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <executions>
                    <execution>
                        <id>default-compile</id>
                        <phase>compile</phase>
                        <goals>
                            <goal>compile</goal>
                        </goals>
                        <configuration>
```

```
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok</artifactId>
                            </path>
                        </annotationProcessorPaths>
                    </configuration>
                </execution>
                <execution>
                    <id>default-testCompile</id>
                    <phase>test-compile</phase>
                    <goals>
                        <goal>testCompile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok</artifactId>
                            </path>
                        </annotationProcessorPaths>
                    </configuration>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>

</project>
```



- 10. USUARIO:

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.usuario;** CLASE -> **UsuarioApplication**

```
package com.lartduniss.usuario;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.autoconfigure.domain.EntityScan;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;
import org.springframework.web.client.RestTemplate;

@SpringBootApplication
@ComponentScan(basePackages = {"com.lartduniss.usuario"})
@EntityScan(basePackages = {"com.lartduniss.usuario.model"})
@EnableJpaRepositories(basePackages = {"com.lartduniss.usuario.repository"})
public class UsuarioApplication {

   public static void main(String[] args) {
       SpringApplication.run(UsuarioApplication.class, args);
   }

   @Bean
   public RestTemplate restTemplate() {
       return new RestTemplate();
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.usuario.config;** CLASE -> **SecurityConfig**

```
package com.lartduniss.usuario.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.web.cors.CorsConfiguration;
```

```java
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import java.util.List;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

   @Bean
   public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
       return http
           .csrf(csrf -> csrf.disable())
           .cors(cors -> cors.configurationSource(corsConfigurationSource()))
           .authorizeHttpRequests(auth -> auth
               .requestMatchers("/error").permitAll()
               .requestMatchers("/api/v1/usuarios/**").permitAll()
               .requestMatchers("/api/v1/usuarios/v3/api-docs/**",
"/api/v1/usuarios/v3/api-docs", "/swagger-ui/**", "/swagger-ui.html").permitAll()
               .anyRequest().authenticated()
           )
           .build();
   }

   @Bean
   public CorsConfigurationSource corsConfigurationSource() {
       CorsConfiguration configuration = new CorsConfiguration();
       configuration.setAllowedOrigins(List.of("http://localhost:8080",
"http://localhost:9090"));
       configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE",
"OPTIONS"));
       configuration.setAllowedHeaders(List.of("*"));
       configuration.setAllowCredentials(true);
       UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
       source.registerCorsConfiguration("/**", configuration);
       return source;
   }

   @Bean
   public PasswordEncoder passwordEncoder() {
       return new BCryptPasswordEncoder();
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.usuario.controller;** CLASE -> **UsuarioController**

```java
package com.lartduniss.usuario.controller;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.lartduniss.usuario.dto.UsuarioRequest;
import com.lartduniss.usuario.service.UsuarioService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;

@RestController
@RequestMapping("/api/v1/usuarios")
@Tag(name = "Authentication", description = "Endpoints para registro e inicio de sesión de usuarios")
public class UsuarioController {

    private final UsuarioService usuarioService;

    public UsuarioController(UsuarioService usuarioService) {
        this.usuarioService = usuarioService;
    }

    @Operation(summary = "Registrar un nuevo usuario", description = "Guarda el usuario asignándole por defecto el rol de CLIENTE si no se especifican roles.")
    @PostMapping("/registrar")
    public ResponseEntity<String> registrar(@RequestBody UsuarioRequest request) {
        try {
            return ResponseEntity.ok(usuarioService.registrar(request));
        } catch (RuntimeException e) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(e.getMessage());
        }
    }

    @Operation(summary = "Iniciar sesión", description = "Retorna el Token JWT correspondiente si las credenciales coinciden")
    @PostMapping("/login")
    public ResponseEntity<String> login(@RequestBody UsuarioRequest request) {
        try {
```

```
            String token = usuarioService.login(request.getNombreUsuario(),
request.getClave());
            return ResponseEntity.ok(token);
        } catch (RuntimeException e) {
            return
ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(e.getMessage());
        }
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.usuario.dto;** CLASE -> **UsuarioRequest**

```
package com.lartduniss.usuario.dto;

import java.util.Set;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class UsuarioRequest
{
    private String nombreUsuario;
    private String clave;
    private String correo;

    private Set<String> roles;

}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.usuario.model;** CLASE -> **Rol**

```
package com.lartduniss.usuario.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
```

```java
import jakarta.persistence.Table;
import lombok.AllArgsConstructor;
import lombok.EqualsAndHashCode;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import lombok.ToString;

@Entity
@Table(name = "roles")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@ToString
@EqualsAndHashCode(onlyExplicitlyIncluded = true)
public class Rol
{
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   @EqualsAndHashCode.Include
   private Long id;
   @Column(name = "nombre_rol", unique = true, nullable = false)
   private String nombreRol;

}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.usuario.model;** CLASE -> **Usuario**

```java
package com.lartduniss.usuario.model;

import java.util.HashSet;
import java.util.Set;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.JoinTable;
import jakarta.persistence.ManyToMany;
```

```java
import jakarta.persistence.Table;
import lombok.AllArgsConstructor;
import lombok.EqualsAndHashCode;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import lombok.ToString;


@Entity
@Table(name = "usuarios")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@ToString(exclude = "roles")
@EqualsAndHashCode(onlyExplicitlyIncluded = true) //Solo usa los campos
seleccionados para comparar
public class Usuario
{
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   @EqualsAndHashCode.Include
   private Long id;
   @Column(name = "nombre_usuario", unique = true, nullable = false)
   private String nombreUsuario;
   @Column(name = "clave", nullable = false)
   private String clave;
   @Column(unique = true, nullable = false)
   private String correo;
   @ManyToMany(fetch = FetchType.EAGER)
   @JoinTable(
       name = "usuario_roles",
       joinColumns = @JoinColumn(name = "usuario_id"),
       inverseJoinColumns = @JoinColumn(name = "rol_id")
   )
   private Set<Rol> roles = new HashSet<>();
   public void agregarRol(Rol rol)
   {
   if (this.roles == null)
   {
       this.roles = new HashSet<>();
   }
   this.roles.add(rol);
   }
```

```
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.usuario.repository;** INTERFAZ -> **RolRepository**

```
package com.lartduniss.usuario.repository;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.lartduniss.usuario.model.Rol;

@Repository
public interface RolRepository extends JpaRepository<Rol, Long>
{
   Optional<Rol> findByNombreRol(String nombreRol);
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.usuario.repository;** INTERFAZ -> **UsuarioRepository**

```
package com.lartduniss.usuario.repository;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.lartduniss.usuario.model.Usuario;

@Repository
public interface UsuarioRepository extends JpaRepository <Usuario, Long>
{
   Optional<Usuario>findByNombreUsuario(String nombreUsuario);

}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.usuario.service;** CLASE -> **JwtService**

```
package com.lartduniss.usuario.service;

import java.util.Date;
import java.util.List;
import org.springframework.stereotype.Service;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
@Service
public class JwtService
{
   @Value("${jwt.secret}")
   private String secreto;
   public String generarToken(String username, List<String> roles)
   {
        long dosHorasEnMilisegundos = 1000L * 60 * 60 * 2;
        return Jwts.builder()
             .setSubject(username)
             .claim("roles", roles)
             .setIssuedAt(new Date())
             .setExpiration(new Date(System.currentTimeMillis()+
dosHorasEnMilisegundos))

.signWith(Keys.hmacShaKeyFor(secreto.getBytes()),SignatureAlgorithm.HS256)
             .compact();
   }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.usuario.service;** CLASE -> **UsuarioService**

```
package com.lartduniss.usuario.service;

import org.springframework.lang.NonNull;
import java.nio.charset.StandardCharsets;
import java.util.List;
import java.util.stream.Collectors;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
```

```java
import com.lartduniss.usuario.dto.UsuarioRequest;
import com.lartduniss.usuario.model.Rol;
import com.lartduniss.usuario.model.Usuario;
import com.lartduniss.usuario.repository.RolRepository;
import com.lartduniss.usuario.repository.UsuarioRepository;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;

@Service
public class UsuarioService {

   @Autowired
   private UsuarioRepository usuarioRepository;

   @Autowired
   private RolRepository rolRepository;

   @org.springframework.beans.factory.annotation.Value("${jwt.secret}")
   private String secreto;

   @Autowired
   private PasswordEncoder passwordEncoder;

   public String registrar(UsuarioRequest request) {
       if
(usuarioRepository.findByNombreUsuario(request.getNombreUsuario()).isPresent()) {
           throw new RuntimeException("El nombre de usuario ya existe.");
       }

       Usuario nuevoUsuario = new Usuario();
       nuevoUsuario.setNombreUsuario(request.getNombreUsuario());
       nuevoUsuario.setCorreo(request.getCorreo());
       nuevoUsuario.setClave(passwordEncoder.encode(request.getClave()));

       if (request.getRoles() == null || request.getRoles().isEmpty()) {
           Rol rolPorDefecto = rolRepository.findByNombreRol("CLIENTE")
               .orElseGet(() -> rolRepository.save(new Rol(null, "CLIENTE")));
           nuevoUsuario.agregarRol(rolPorDefecto);
       } else {
           for (String nombreRol : request.getRoles()) {
               Rol rolEncontrado =
rolRepository.findByNombreRol(nombreRol.toUpperCase())
                   .orElseThrow(() -> new RuntimeException("Error: El rol " +
nombreRol + " no existe en la DB."));
```

```
                nuevoUsuario.agregarRol(rolEncontrado);
            }
        }

        usuarioRepository.save(nuevoUsuario);
        return "Usuario Registrado";
    }

    @Transactional(readOnly = true)
    public String login(String nombreUsuario, String clave) {
        Usuario usuario = usuarioRepository.findByNombreUsuario(nombreUsuario)
                .orElseThrow(() -> new RuntimeException("Credenciales inválidas"));

        if (!passwordEncoder.matches(clave, usuario.getClave())) {
            throw new RuntimeException("Credenciales inválidas");
        }

        List<String> rolesList = usuario.getRoles().stream()
                .map(Rol::getNombreRol)
                .collect(Collectors.toList());

        java.util.Date ahora = new java.util.Date();
        java.util.Date expiracion = new java.util.Date(ahora.getTime() + 86400000);
// 24 Horas

        return Jwts.builder()
                .setSubject(usuario.getNombreUsuario())
                .claim("roles", rolesList)
                .setIssuedAt(ahora)
                .setExpiration(expiracion)

.signWith(Keys.hmacShaKeyFor(secreto.getBytes(StandardCharsets.UTF_8)),
SignatureAlgorithm.HS256)
                .compact();
    }
}
```

RUTA -> SRC/MAIN/SOURCES
APPLICATION.PROPERTIES

```
spring.application.name=service-auth
server.port=8095

spring.datasource.url=jdbc:mysql://localhost:3306/db_usuario?createDatabaseIfNotExi
st=true
```

```
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Llave secreta para firmar el Token (Mínimo 32 caracteres)
jwt.secret=493322258340235739058340598340598340598340598340598340598

# Ajustes de documentación Swagger
springdoc.api-docs.path=/api/v1/usuarios/v3/api-docs
springdoc.swagger-ui.path=/swagger-ui.html
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.usuario;** CLASE -> **UsuarioApplicationTests**

```
package com.lartduniss.usuario;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Configuration;

@SpringBootTest(classes = UsuarioApplicationTests.MockConfig.class)
class UsuarioApplicationTests {

    @Configuration
    static class MockConfig {
    }

    @Test
    void contextLoads() {
    }
}
```

RUTA -> SRC/MAIN/JAVA
PACKAGE -> **com.lartduniss.usuario.service;** CLASE -> **UsuarioServiceTests**

```
package com.lartduniss.usuario.service;

import com.lartduniss.usuario.model.Usuario;
import com.lartduniss.usuario.repository.UsuarioRepository;
```

```java
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.Mockito;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.security.crypto.password.PasswordEncoder;

import java.util.Optional;

import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.junit.jupiter.api.Assertions.assertEquals;

@ExtendWith(MockitoExtension.class)
class UsuarioServiceTest {

    @Mock
    private UsuarioRepository usuarioRepository;

    @Mock
    private PasswordEncoder passwordEncoder;

    @InjectMocks
    private UsuarioService usuarioService;

    @Test
    void loginCredencialesInvalidasTest() {
        // 1. Arrange
        String username = "denisse";
        String claveFalsa = "123456";

Mockito.when(usuarioRepository.findByNombreUsuario(username)).thenReturn(Optional.e
mpty());

        // 2. Act & 3. Assert (Patrón AAA)
        RuntimeException excepcion = assertThrows(RuntimeException.class, () -> {
            usuarioService.login(username, claveFalsa);
        });

        assertEquals("Credenciales inválidas", excepcion.getMessage());
    }
}
```

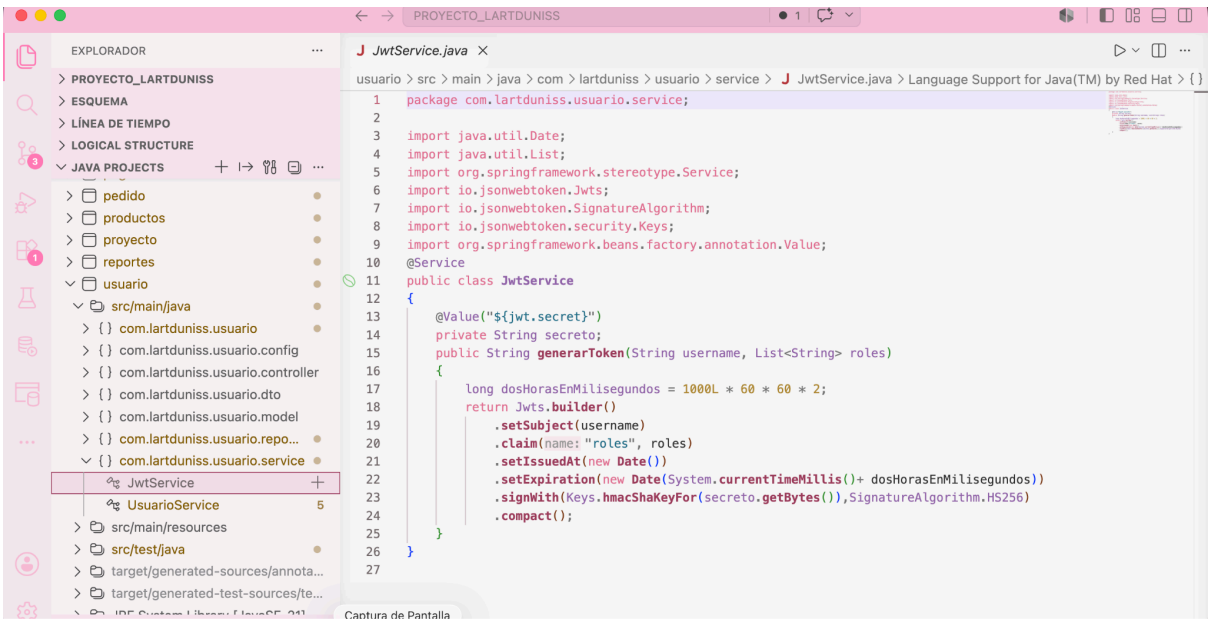
POM.XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>
   <parent>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-parent</artifactId>
       <version>3.5.15</version>
       <relativePath/> <!-- lookup parent from repository -->
   </parent>
   <groupId>com.lartduniss</groupId>
   <artifactId>usuario</artifactId>
   <version>0.0.1-SNAPSHOT</version>
   <name>usuario</name>
   <description/>
   <url/>
   <licenses>
       <license/>
   </licenses>
   <developers>
       <developer/>
   </developers>
   <scm>
       <connection/>
       <developerConnection/>
       <tag/>
       <url/>
   </scm>
   <properties>
       <java.version>21</java.version>
   </properties>
   <dependencies>
       <dependency>
           <groupId>org.springframework.boot</groupId>
           <artifactId>spring-boot-starter-data-jpa</artifactId>
       </dependency>
       <dependency>
           <groupId>org.springframework.boot</groupId>
           <artifactId>spring-boot-starter-security</artifactId>
       </dependency>
       <dependency>
           <groupId>org.springframework.boot</groupId>
```

```
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-api</artifactId>
            <version>0.11.5</version>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-impl</artifactId>
            <version>0.11.5</version>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-jackson</artifactId>
            <version>0.11.5</version>
        </dependency>
        <dependency>
            <groupId>org.springdoc</groupId>
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
            <version>2.8.17</version>
        </dependency>
        <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.11.5</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.11.5</version>
</dependency>

        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
```

```
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.security</groupId>
            <artifactId>spring-security-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <excludes>
                        <exclude>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </exclude>
                    </excludes>
                </configuration>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <executions>
                    <execution>
                        <id>default-compile</id>
                        <phase>compile</phase>
                        <goals>
                            <goal>compile</goal>
                        </goals>
                        <configuration>
                            <annotationProcessorPaths>
```

```
                                <path>
                                    <groupId>org.projectlombok</groupId>
                                    <artifactId>lombok</artifactId>
                                </path>
                            </annotationProcessorPaths>
                        </configuration>
                    </execution>
                    <execution>
                        <id>default-testCompile</id>
                        <phase>test-compile</phase>
                        <goals>
                            <goal>testCompile</goal>
                        </goals>
                        <configuration>
                            <annotationProcessorPaths>
                                <path>
                                    <groupId>org.projectlombok</groupId>
                                    <artifactId>lombok</artifactId>
                                </path>
                            </annotationProcessorPaths>
                        </configuration>
                    </execution>
                </executions>
            </plugin>
        </plugins>
    </build>

</project>
```



Instrucciones de Ejecución detalladas (Indicador IE 3.3.2)

Para garantizar el correcto funcionamiento del ecosistema de microservicios y evitar errores de conectividad o resolución de rutas, el despliegue debe realizarse de manera secuencial siguiendo los pasos descritos a continuación:

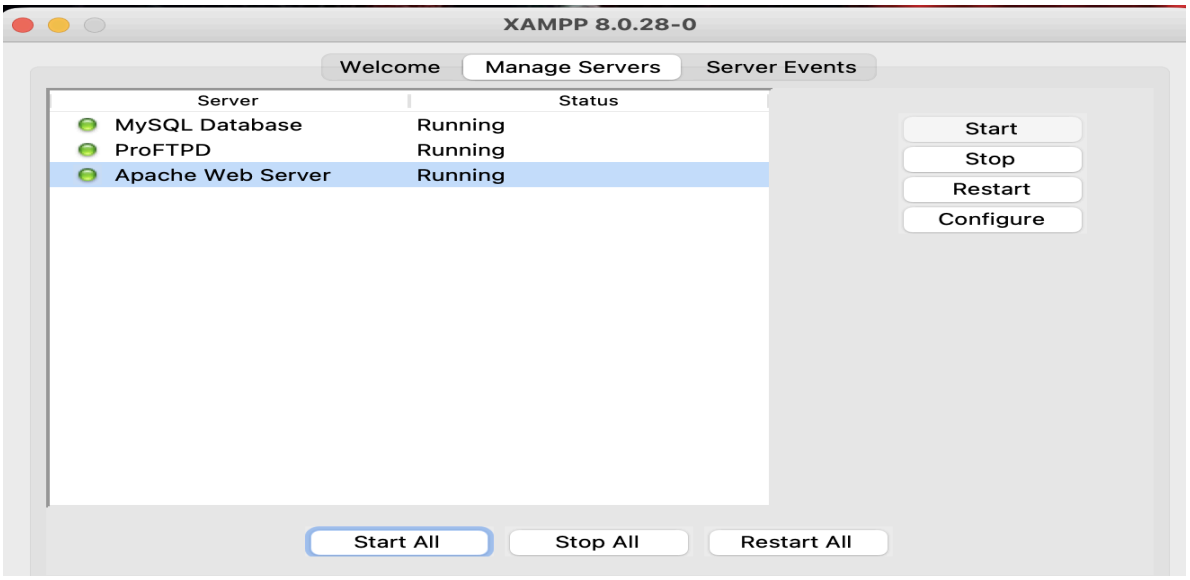
5.1. Requisitos Previos del Sistema

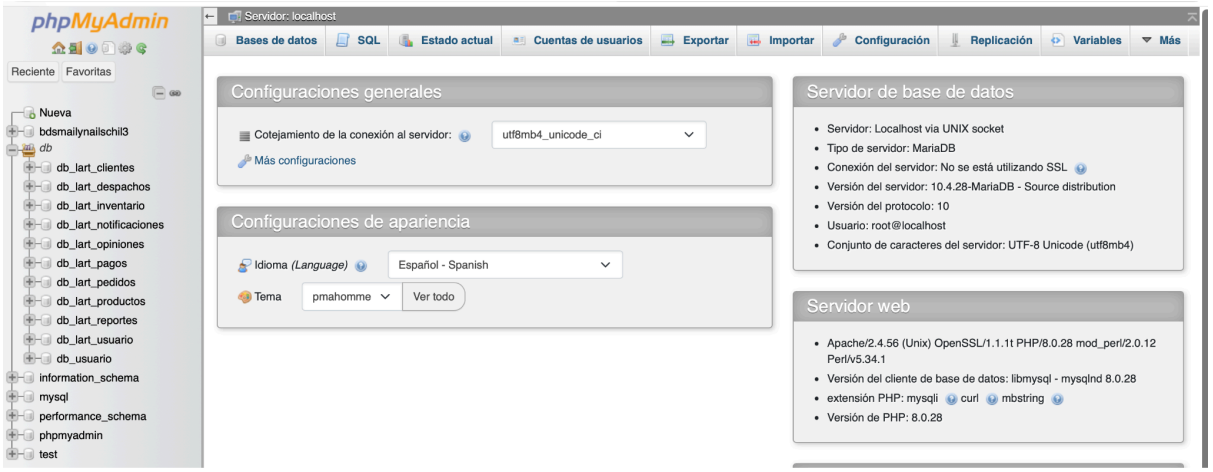
Antes de iniciar los servicios, asegurarse de contar con las siguientes herramientas instaladas y configuradas:

- **Java Development Kit (JDK):** Versión 17 o superior.
- **Apache Maven:** Versión 3.8+ instalada y configurada en las variables de entorno (PATH).
- **Motor de Base de Datos:** Servidor MySQL (XAMPP / phpMyAdmin.
- **Herramienta de Pruebas:** Postman o acceso a un navegador web para validar los endpoints de Swagger.

5.2. Paso 1: Preparación de la Base de Datos

1. Inicie los servicios de Apache y MySQL desde el panel de control de XAMPP.
2. Acceda a http://localhost/phpmyadmin/.
3. Cree las bases de datos relacionales correspondientes a los microservicios que requieren persistencia (por ejemplo, db_usuario, db_producto, etc.) de acuerdo con los nombres definidos en los archivos application.properties / application.yml de cada proyecto.

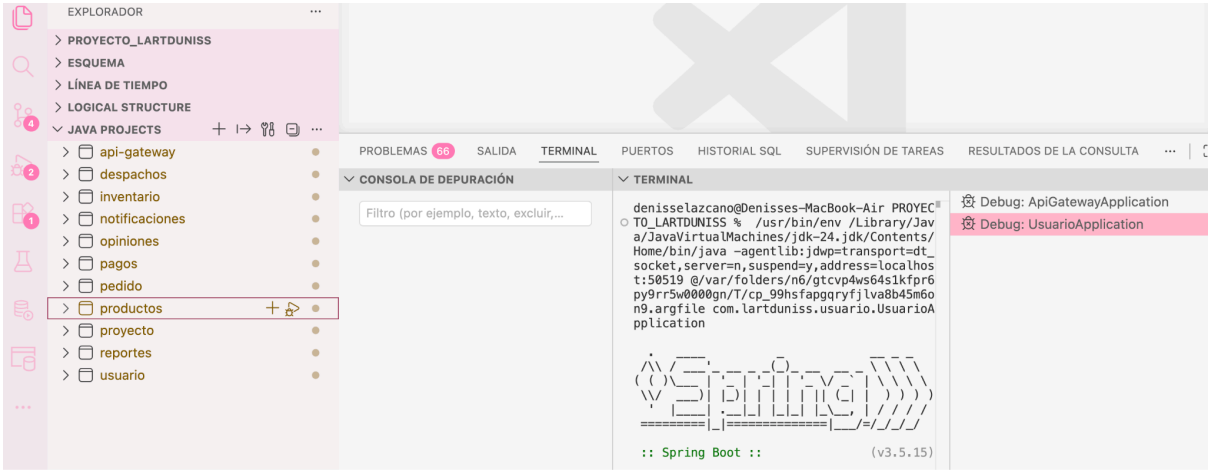


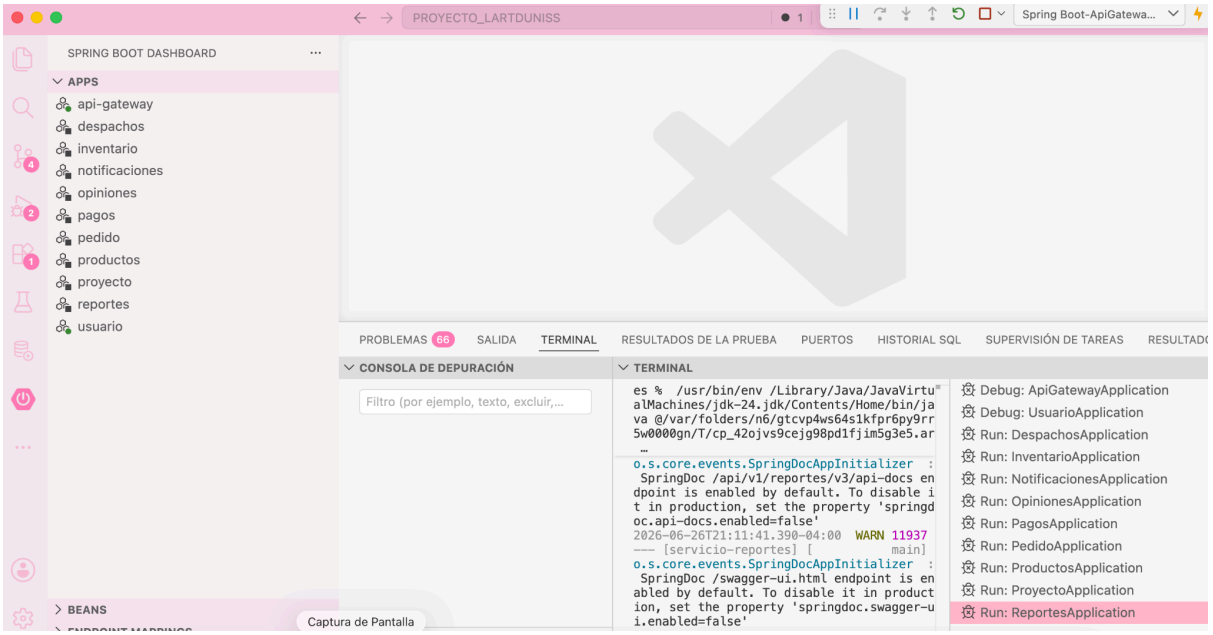


5.3. Paso 2: Orden Estricto de Encendido de los Microservicios

Para evitar excepciones de enrutamiento, los servicios deben ser levantados de forma independiente en terminales separadas de Visual Studio Code o la consola de comandos, respetando estrictamente el siguiente orden jerárquico:

1. **Servidor de Enrutamiento (api-gateway):** Debe ser el primer componente en encenderse (Puerto 9090). Actúa como punto único de entrada, interceptor de seguridad JWT y balanceador de carga hacia las distintas APIs.
2. **Microservicio Núcleo de Autenticación (usuario o auth-service):** Esencial para proveer la generación de tokens y validar las credenciales de los endpoints protegidos.
3. **Microservicios de Negocio y Operaciones:** Una vez que el Gateway y el servicio de Usuarios se encuentren estables, proceda a levantar los 8 microservicios restantes en cualquier orden (Clientes, Productos, Despachos, Opiniones, Carrito, Ventas, etc.)





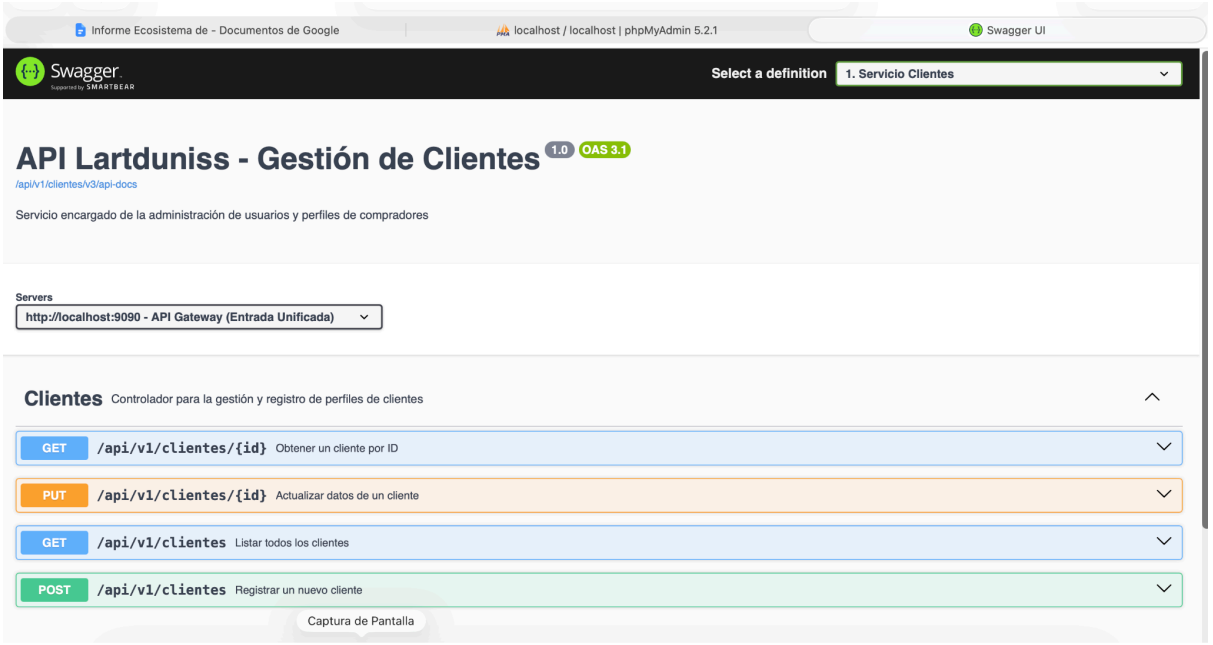
5.4. Paso 3: Verificación del Ecosistema

Una vez que las 10 terminales se encuentren en estado de escucha (*Started... in X seconds*), se puede comprobar la salud del ecosistema mediante las siguientes vías:

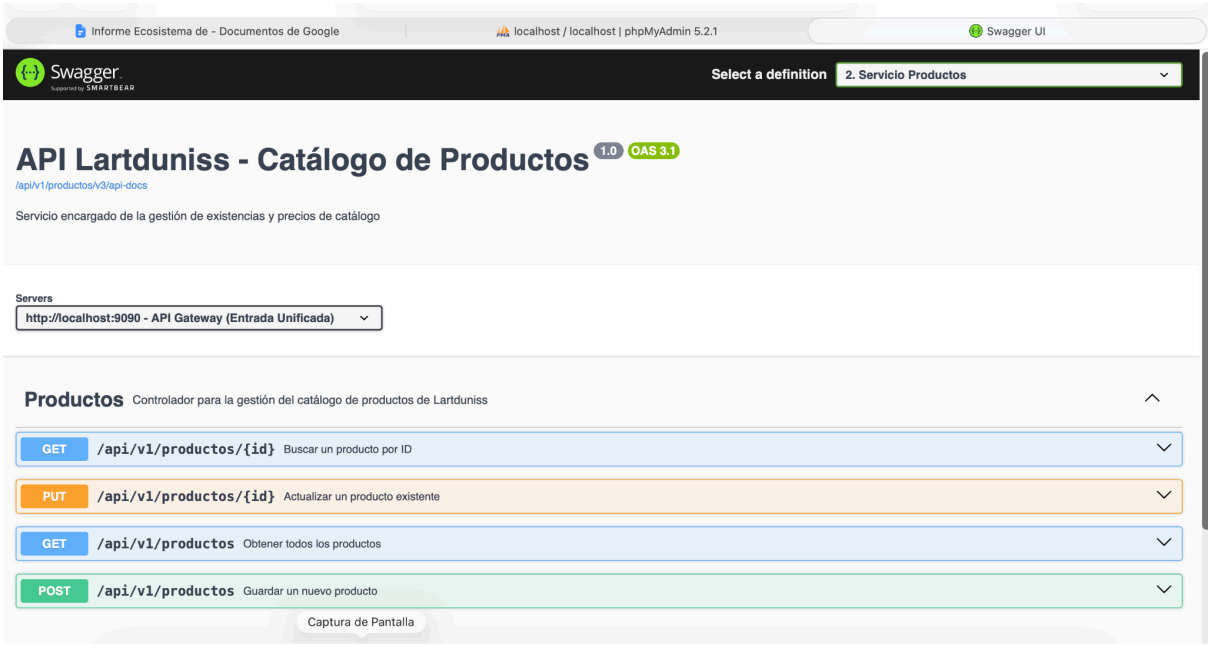
- **Acceso a la Documentación (Swagger UI):** Ingrese a la URL expuesta por el Gateway para interactuar con las APIs de forma centralizada: http://localhost:9090/webjars/swagger-ui/index.html?urls.primaryName=1.+Servicio+Clientes

- **Pruebas de Consumo Externo:** Importe la colección de endpoints en Postman, apunte las peticiones hacia el puerto del Gateway (9090) usando los prefijos configurados (ej: /api/v1/usuarios/login) y verifique la recepción correcta de las cabeceras Authorization: Bearer <token_jwt>.

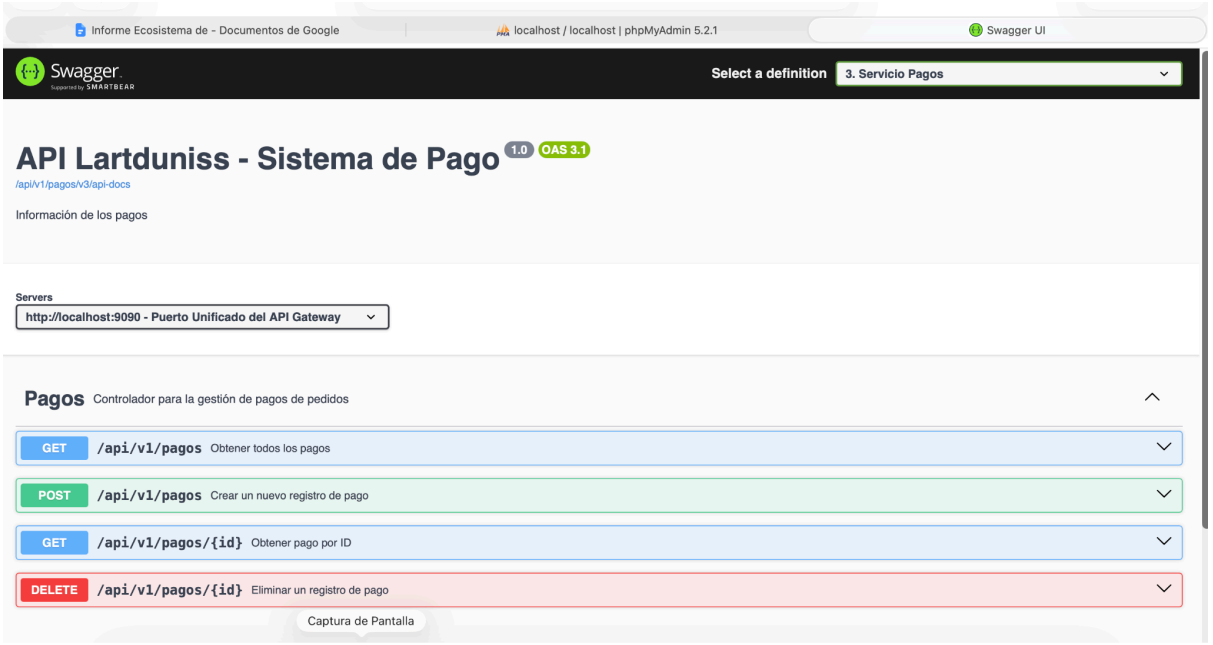
URL CLIENTES:
http://localhost:9090/webjars/swagger-ui/index.html?urls.primaryName=1.+Servicio+Clientes



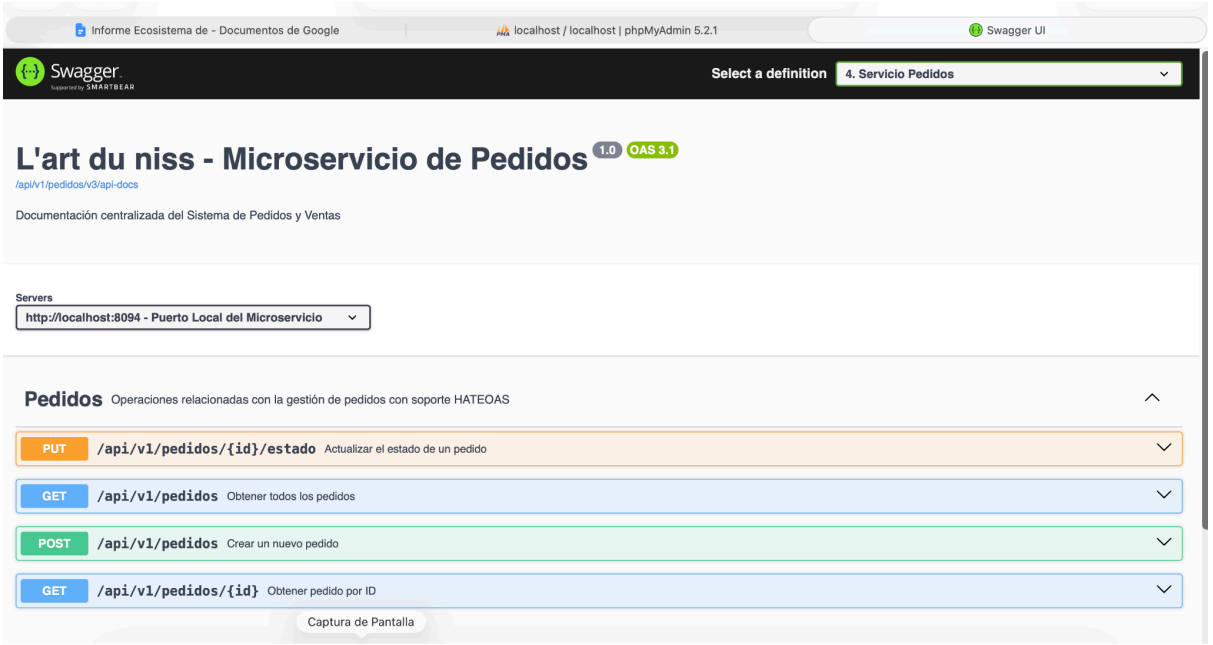
URL PRODUCTOS
http://localhost:9090/webjars/swagger-ui/index.html?urls.primaryName=2.+Servicio+Productos:



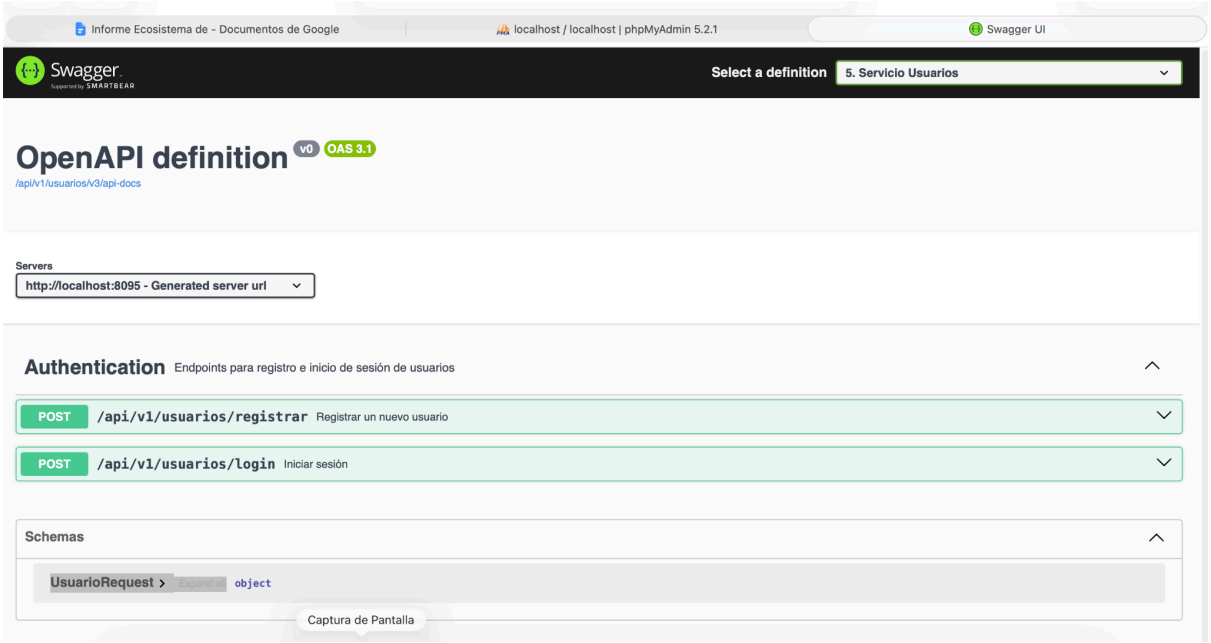
URL PAGOS:
http://localhost:9090/webjars/swagger-ui/index.html?urls.primaryName=3.+Servicio+Pagos



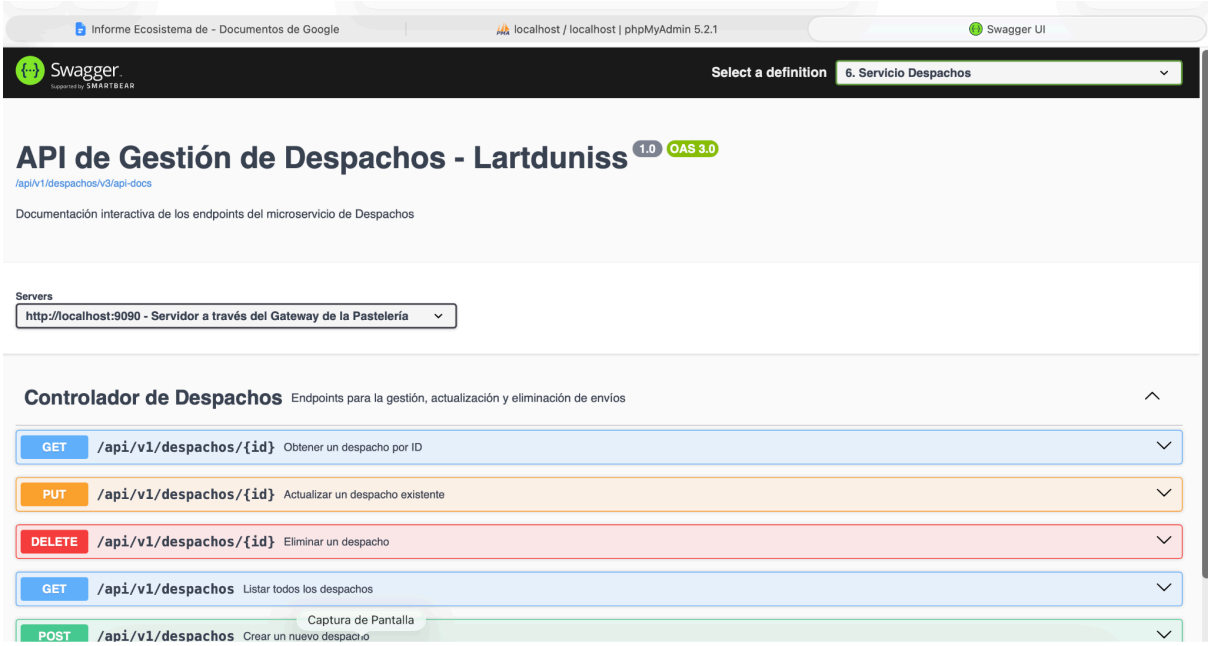
URL PEDIDOS:
http://localhost:9090/webjars/swagger-ui/index.html?urls.primaryName=4.+Servicio+Pedidos



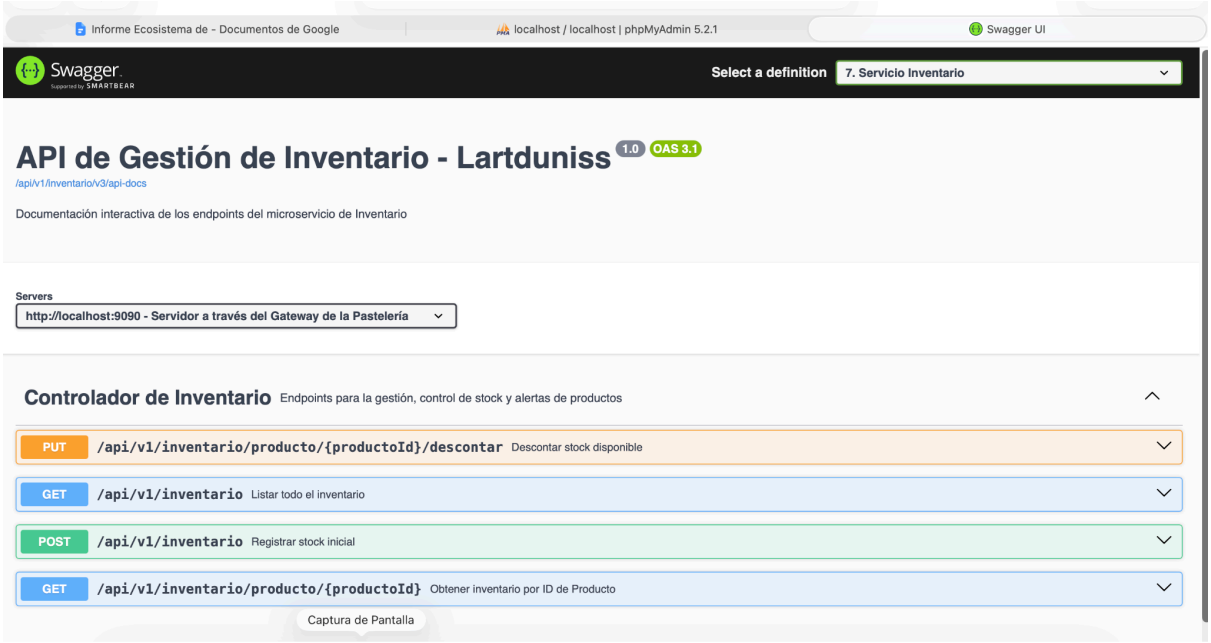
URL USUARIOS:
http://localhost:9090/webjars/swagger-ui/index.html?urls.primaryName=5.+Servicio+Usuarios



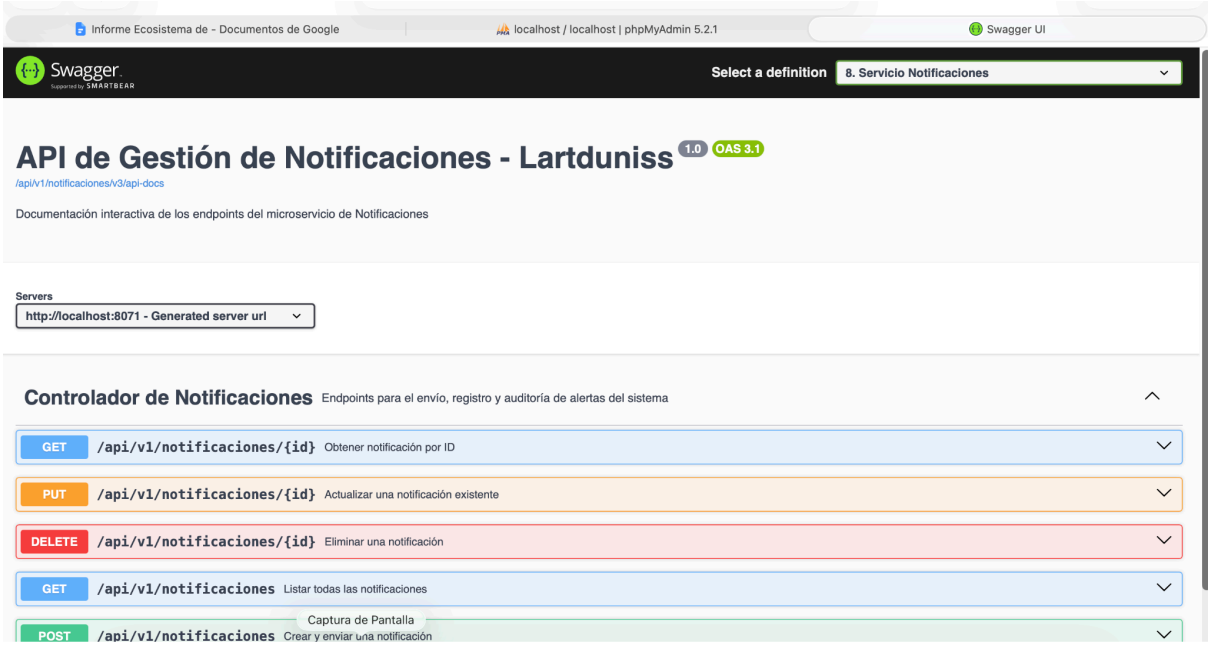
URL DESPACHOS:
http://localhost:9090/webjars/swagger-ui/index.html?urls.primaryName=6.+Servicio+Despachos



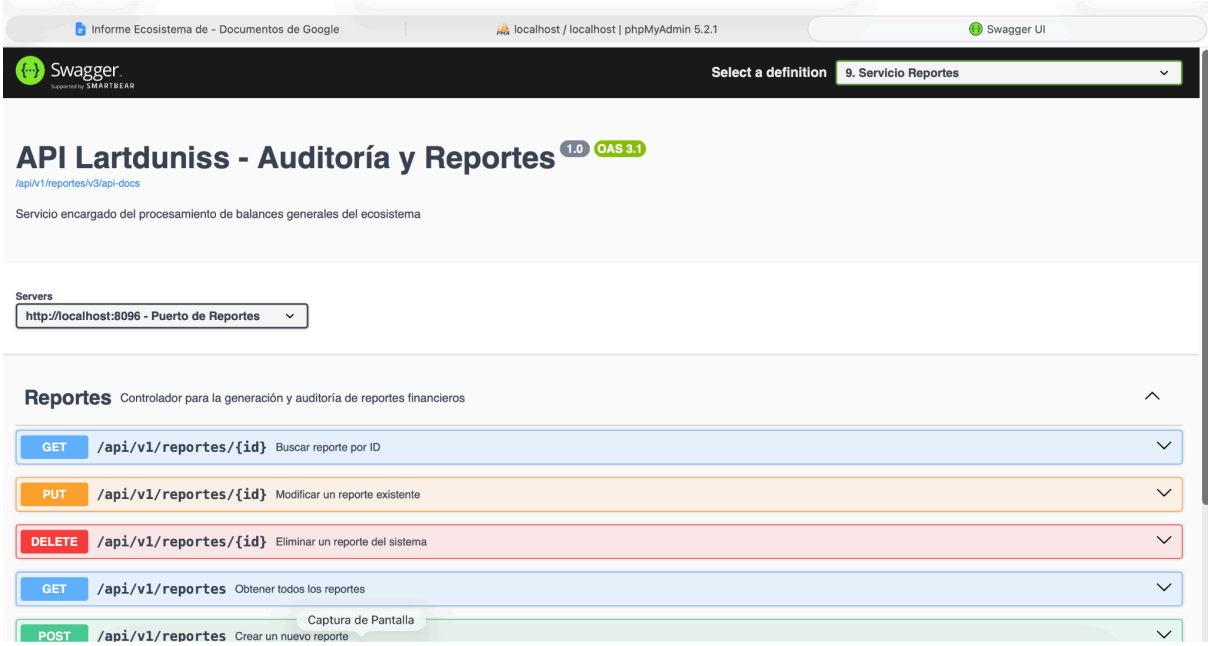
URL INVENTARIO:
http://localhost:9090/webjars/swagger-ui/index.html?urls.primaryName=7.+Servicio+Inventario



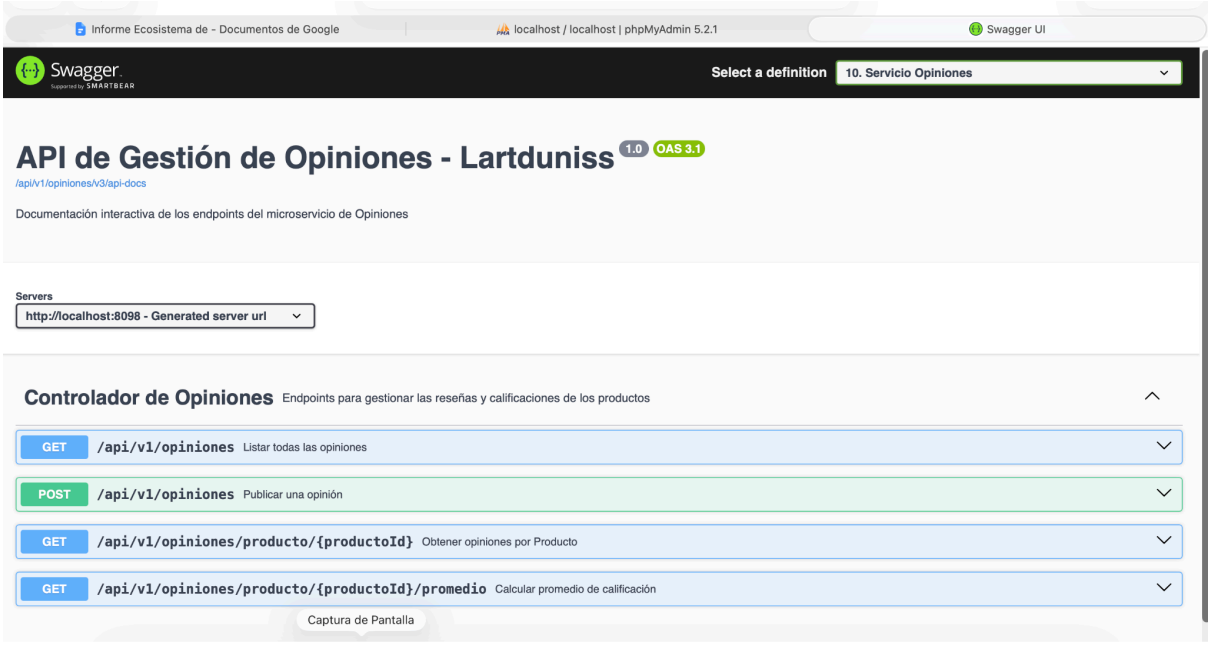
URL INVENTARIO:
http://localhost:9090/webjars/swagger-ui/index.html?urls.primaryName=8.+Servicio+Notificaciones



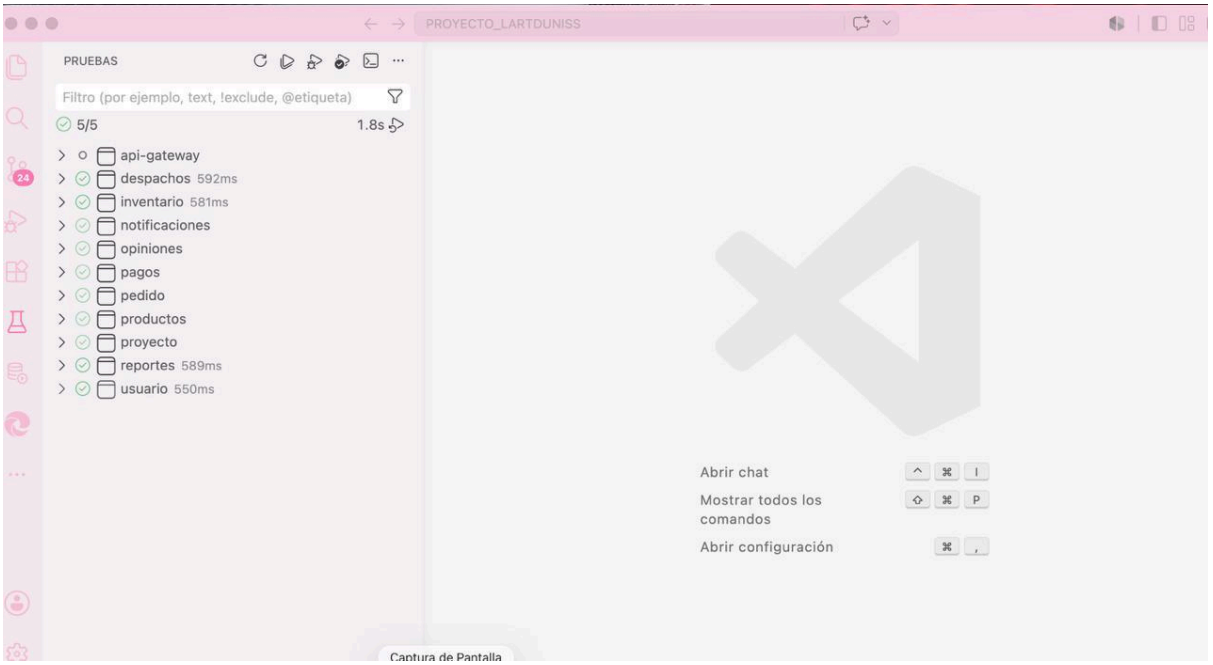
URL REPORTES:
http://localhost:9090/webjars/swagger-ui/index.html?urls.primaryName=9.+Servicio+Reportes



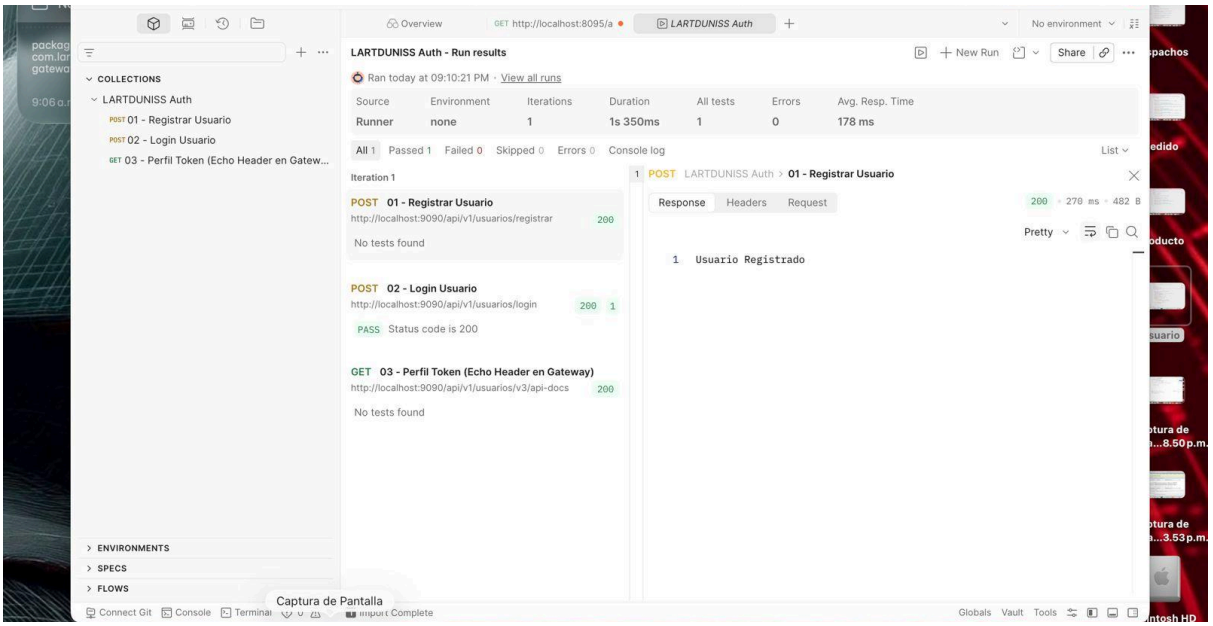
URL OPINIONES:
http://localhost:9090/webjars/swagger-ui/index.html?urls.primaryName=9.+Servicio+Reportes

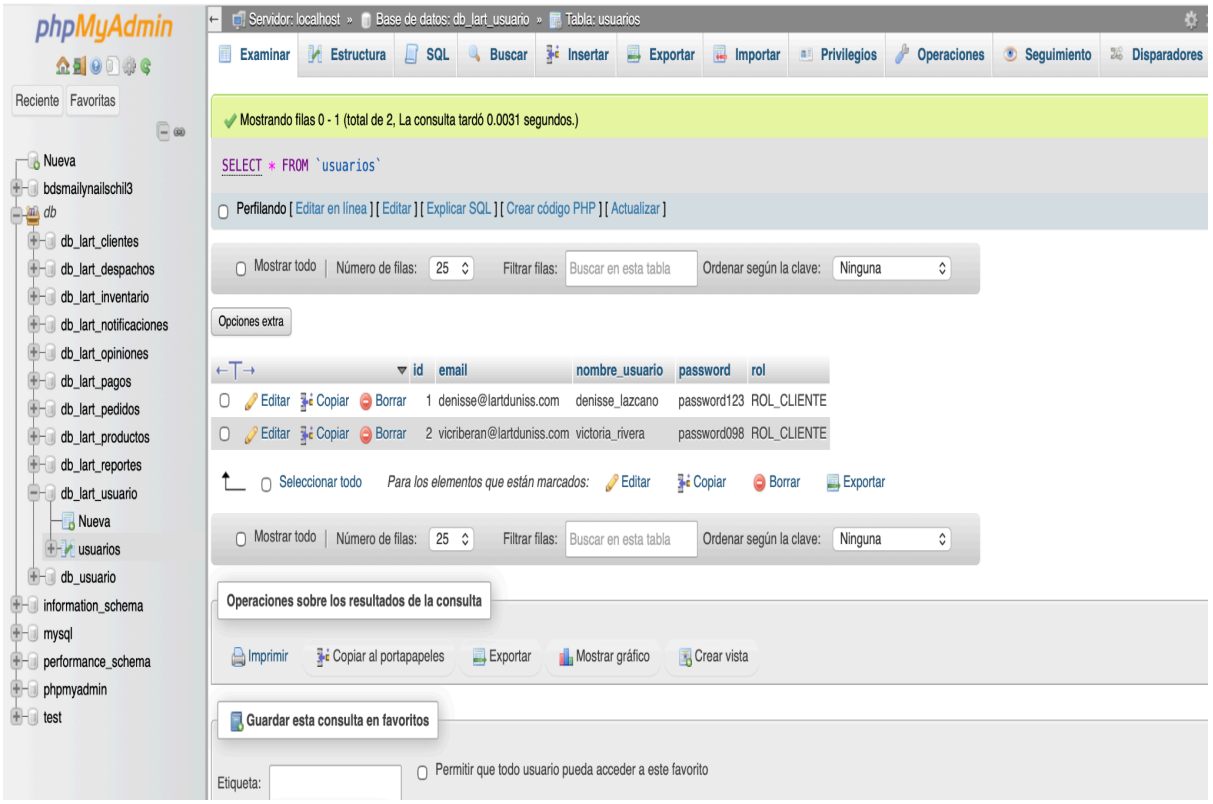


5.5. Evidencias de Cobertura de Pruebas Unitarias



5.6. Evidencias de Autenticador y token




phpMyAdmin
Reciente
Favoritas
Nueva
bdsmailynailschil3
db
db_lart_clientes
db_lart_despachos
db_lart_inventario
db_lart_notificaciones
db_lart_opiniones
db_lart_pagos
db_lart_pedidos
db_lart_productos
db_lart_reportes
db_lart_usuario
Nueva
usuarios
db_usuario
information_schema
mysql
performance_schema
phpmyadmin
test
Servidor: localhost » Base de datos: db_lart_usuario » Tabla: usuarios
Examinar
Estructura
SQL
Buscar
Insertar
Exportar
Importar
Privilegios
Operaciones
Seguimiento
Disparadores
Mostrando filas 0 - 1 (total de 2, La consulta tardó 0.0031 segundos.)
SELECT * FROM `usuarios`
Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]
Mostrar todo | Número de filas: 25 Filtrar filas: Buscar en esta tabla Ordenar según la clave: Ninguna
Opciones extra
id email nombre_usuario password rol
Editar Copiar Borrar 1 denisse@lartduniss.com denisse_lazcano password123 ROL_CLIENTE
Editar Copiar Borrar 2 vicriberan@lartduniss.com victoria_rivera password098 ROL_CLIENTE
Seleccionar todo Para los elementos que están marcados: Editar Copiar Borrar Exportar
Mostrar todo | Número de filas: 25 Filtrar filas: Buscar en esta tabla Ordenar según la clave: Ninguna
Operaciones sobre los resultados de la consulta
Imprimir Copiar al portapapeles Exportar Mostrar gráfico Crear vista
Guardar esta consulta en favoritos
Etiqueta:
Permitir que todo usuario pueda acceder a este favorito